Politechnika Wrocławska
Wydział Informatyki i Telekomunikacji

Kierunek: **Informatyka Stosowana (IST)**
Specjalność: **Inżynieria oprogramowania (IO)**

PRACA DYPLOMOWA MAGISTERSKA

Model oceny efektywności narzędzi wspomagających zarządzanie zmianami w bazach danych w kontekście CI/CD

inż. Adam Stypczyc

Opiekun pracy
dr inż. Marcin Kawalerowicz

Słowa kluczowe: ciągła integracja, ciągłe dostarczanie, inżynieria oprogramowania, zarządzanie bazami danych

WROCŁAW 2026

Streszczenie

Niniejsza praca magisterska ma na celu opracowanie modelu oceny efektywności narzędzi wspomagających zarządzanie zmianami w bazach danych w kontekście CI/CD. W ramach realizacji pracy stworzono aplikację demonstracyjną umożliwiającą wykonywanie migracji i rollbacków zarówno przy użyciu Flyway, jak i Liquibase, wyświetlającą wyniki badanych metryk oraz udostępniającą podgląd bieżącego schematu bazy danych. Ponadto zaimplementowano dwa potoki wdrożeniowe w środowisku Github Actions, które pozwoliły na przetestowanie narzędzi w warunkach ciągłej integracji i ciągłego dostarczania.

Zgromadzono i porównano metryki obu narzędzi w środowisku lokalnym oraz CI/CD, a następnie przeprowadzono analizę statystyczną czasu migracji w celu oceny istotności zaobserwowanych różnic. Na podstawie pozyskanych danych opracowano model oceny efektywności, uwzględniający normalizację danych oraz środowisko wykonania. Ostateczne wyniki modelu wskazały efektywność na poziomie $0,52$ dla narzędzia Flyway oraz $0,54$ dla narzędzia Liquibase.

Praca dowodzi, że wskazanie jednego uniwersalnego narzędzia spełniającego wszystkie wymagania organizacji jest złożonym zadaniem, jednakże opracowany model umożliwia świadome dobranie narzędzia zgodnie z indywidualnymi priorytetami. Dodatkowo wdrożenie potoków CI/CD i aplikacji demonstracyjnej potwierdziło, że możliwe jest praktyczne wdrożenie migracji w architekturze ciągłego dostarczania.

Abstract

This thesis aims to develop a model for evaluating the effectiveness of database change management tools in the context of CI/CD. As part of the thesis, a demonstration application was created to perform migrations and rollbacks using both Flyway and Liquibase, displaying the results of the metrics and providing a preview of the current database schema. In addition, two pipelines were implemented in a Github Actions environment, which allowed the tools to be tested under continuous integration and continuous delivery conditions.

Metrics of both tools in the local and CI/CD environment were collected and compared, followed by a statistical analysis of migration time to assess the significance of observed differences. Based on the acquired data, a model was developed to assess efficiency, taking into account data normalisation and the execution environment. The final model results indicated efficiencies of $0,52$ for the Flyway tool and $0,54$ for the Liquibase tool.

The thesis proves that identifying a universal tool that meets all the requirements of an organisation is a complex task, however, the developed model allows an aware selection of the tool according to individual priorities. Additionally, the implementation of CI/CD pipelines and a demo application confirmed that it is possible to practically implement migration in a continuous delivery architecture.

Spis treści

1 Wstęp **3**

2 Opis teoretyczny **5**
2.1 Przegląd literatury 5
2.2 Analiza potrzeb i wymagań 10
2.3 Krótki opis dostępnych narzędzi 11
2.4 Zarządzanie zmianami w bazach danych 12
2.4.1 Kontrola wersji 13
2.4.2 Migracja danych 13
2.4.3 Cofanie zmian i odzyskiwanie danych 13

3 Aplikacja demonstracyjna **15**
3.1 Opis aplikacji 15
3.1.1 Potoki wdrożeniowe Github Actions 15
3.1.2 Backend 19
3.1.3 Frontend 20
3.1.4 Baza danych 21
3.2 Scenariusze migracji 23
3.2.1 Scenariusz 1. 23
3.2.2 Scenariusz 2. 24
3.3 Porównanie wybranych narzędzi 26
3.3.1 Metryki 26
3.3.2 Flyway i Liquibase 27

4 Wyniki badań **37**
4.1 Scenariusz 1. 37
4.1.1 Środowisko lokalne 37
4.1.2 Środowisko CI/CD 42
4.2 Scenariusz 2. 46
4.2.1 Środowisko lokalne 46
4.2.2 Środowisko CI/CD 50
4.3 Zestawienie dla metryki - Złożoność skryptów migracyjnych 54

5 Model efektywności **59**
5.1 Normalizacja 60
5.2 Stopień efektywności 60
5.3 Stopień efektywności wybranych narzędzi 62

6 Wnioski **65**

7 Załączniki do pracy **69**

Bibilografia **69**

Rozdział 1

Wstęp

Współczesne procesy wytwarzania oprogramowania wymagają od zespołów IT szybkiego reagowania na zmiany i częstego wdrażania nowych funkcji. Ważną rolę odgrywają tutaj praktyki ciągłej integracji oraz ciągłego dostarczania, które automatyzują proces łączenia i testowania zmian oraz ich wdrażania w krótkich, powtarzalnych cyklach. Dzięki tym praktykom zespoły mogą szybciej publikować aktualizacje przy zachowaniu wysokiej jakości oprogramowania. W efekcie CI/CD umożliwia szybkie i powtarzalne wdrażanie zmian zarówno w kodzie aplikacji, jak i w infrastrukturze, co zwiększa zwinność i niezawodność procesu deweloperskiego.

W takich środowiskach wyzwaniem pozostaje synchronizacja zmian w kodzie aplikacji z ewolucją struktury bazy danych. Nagłe zmiany w wymaganiach biznesowych, dodawanie nowych funkcjonalności czy refaktoryzacje w kodzie mogą zmuszać do modyfikacji schematu bazy danych, czyli wykonywania migracji schematu. Ręczne lub nieprzemyślane migracje mogą prowadzić do poważnych problemów, takich jak chwilowa niedostępność aplikacji, spadki wydajności czy nawet utrata danych. Z tego powodu integracja z narzędziami wspomagającymi zarządzanie zmianami w bazach danych, które umożliwiają automatyzację procesów migracji, może zminimalizować ryzyko wystąpienia błędów. Pomimo szerokiego zastosowania CI/CD, zarządzanie zmianami w bazach danych w tych procesach wciąż stawia unikalne wyzwania, a wybór odpowiednich narzędzi oraz praktyk zarządzania zmianami w bazach danych jest kluczowy dla skutecznego wdrażania aplikacji w środowiskach CI/CD.

Celem niniejszej pracy jest przeprowadzenie analizy porównawczej narzędzi Flyway i Liquibase ze szczególnym uwzględnieniem ich przydatności w środowiskach CI/CD. Praca obejmuje analizę potrzeb i wymagań stawianych przed takimi narzędziami, porównanie funkcjonalności i wydajności obu rozwiązań oraz opracowanie modelu wieloaspektowej oceny ich efektywności. W ramach badania przygotowano aplikację demonstracyjną oraz skonfigurowano potoki CI/CD, co pozwoliło na zebranie metryk związanych z wydajnością, niezawodnością i skutecznością działania obu systemów migracji. Wyniki tego badania mogą przynieść korzyści dla zespołów DevOps i inżynierów danych, ponieważ będą oni mogli zyskać wskazówki jak podejść do optymalizacji procesów wdrażania zmian w bazach danych oraz będą wiedzieli w jaki sposób porównać narzędzia ułatwiające bezpieczną ewolucję schematu danych przy zachowaniu spójności i integralności.

Praca będzie miała na celu odpowiedź na poniższe pytania badawcze, które stanowią podstawę niniejszej pracy.

- RQ1: Jakie są specyficzne potrzeby i wymagania dotyczące zarządzania zmianami w bazach danych w kontekście aplikacji opartych na CI/CD?
- RQ2: Jakie parametry są kluczowe dla oceny efektywności narzędzi do zarządzania zmianami w bazach danych w środowiskach CI/CD?
- RQ3: Jak zdefiniować wieloaspektowy model efektywności narzędzi CI/CD do zarządzania zmianami w bazach danych, uwzględniający wydajność, niezawodność oraz skuteczność?

Rozdział 2

Opis teoretyczny

W niniejszym rozdziale zostanie przedstawiony przegląd literaturowy obejmujący teoretyczne podstawy oraz dostępne narzędzia do zarządzania zmianami w bazach danych. Rozdział ten jest kluczowy z kilku powodów. Po pierwsze, dostarcza teoretycznego kontekstu niezbędnego do zrozumienia mechanizmów działania narzędzi migracyjnych, co będzie stanowić podstawę do interpretacji wyników badań. Po drugie, umożliwia usystematyzowanie wiedzy na temat istniejących rozwiązań. Po trzecie, ułatwia sformułowanie kryteriów porównawczych i opracowanie modelu wieloaspektowej oceny efektywności, który zostanie zaprezentowany w dalszej części pracy.

2.1 Przegląd literatury

Współczesne systemy informatyczne stają się coraz bardziej złożone, ewoluują w kierunku wielowarstwowych architektur, które są oparte o mikroserwisy, konteneryzację i rozwiązania chmurowe. Integracja tych komponentów wymaga wdrożenia odpowiednich narzędzi DevOps, aby zapewnić ciągłą integrację i dostarczanie oprogramowania. Umożliwi to automatyzację procesów, dynamiczne skalowanie i zarządzanie w środowiskach rozproszonych. Systemy te muszą także sprostać wymaganiom dotyczącym wysokiej dostępności, elastyczności oraz bezpieczeństwa, co pociąga za sobą konieczność stosowania mechanizmów monitorowania, kontroli wersji oraz automatycznych procedur rollbacku, aby utrzymać stabilność operacyjną w dynamicznie zmieniającym się otoczeniu technologicznym. W obliczu tych wyzwań, niezbędne jest zastosowanie precyzyjnych strategii oraz narzędzi do zarządzania zmianami w bazach danych, które umożliwią utrzymanie spójności i integralności schematu w rozproszonych, szybko ewoluujących systemach. W dalszej części rozdziału zostanie wykonany przegląd literatury umożliwiający zrozumienie istniejących metod, narzędzi i strategii DevOps, a w szczególności mechanizmów CI/CD oraz narzędzi wspomagających migrację i zarządzanie zmianami w bazach danych, takich jak Liquibase i Flyway.

Przegląd literatury został przeprowadzony poprzez początkowe zdefiniowanie słów kluczowych *"CI/CD", "database change management", "Liquibase", "Flyway", "DevOps"*. Następnie do wyszukania artykułów wykorzystano wyszukiwarkę oraz bazy danych, takie jak Google Schoolar, arXiv oraz Scopus. Wyszukane prace poddano wstępnej selekcji poprzez analizę tytułów i streszczeń.

Autorzy książki *"The DevOps Handbook: How to Create World-Class Agility, Reliability, and Security in Technology Organizations"* [34] przedstawiają w jaki sposób praktyki DevOps mogą wpływać na poprawę szybkości, jakości i bezpieczeństwa wdrożeń. Na wstę-

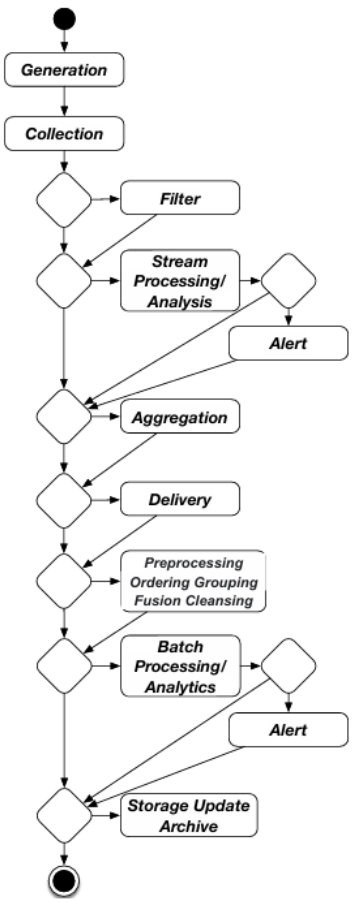
pie książki, autorzy podkreślają konieczność eliminacji konfliktów między zespołami developerskimi (Dev) a operacyjnymi (Ops). Zwracają uwagę, że sprzeczność pomiędzy zespołami prowadzi do lawinowego wzrostu długu technicznego, co obniża się poziom kultury pracy organizacyjnej, poziom efektywności. zarówno kulturę organizacyjną, jak i efektywność pracy. Zachęcają do wdrożenia kultury DevOps, opartej na hipotezach, współpracy oraz automatyzacji, co pozytywnie wpływa na jakość obsługi klienta i atmosferę w zespole. Książka składa się z sześciu części, gdzie pierwsze dwie wprowadzają czytelnika do wprowadzają czytelnika w tematykę DevOps, prezentując koncepcje dotyczące projektowania organizacji oraz jej wzorców. Dodatkowo dają wskazówki w jaki sposób rozpocząć implementację DevOps. Trzecia część omawia techniczne praktyki przepływu (*"The Technical Practices of Flow"*), które koncentrują się na przyspieszaniu procesów poprzez budowę potoków wdrożeniowych. Czwarta część skupia się na przyśpieszeniu przekazywania informacji zwrotnych, w tym tworzeniu skutecznej telemetrii produkcyjnej, co pozwoli na dostrzeganie problemów przed ich eskalacją, oraz na znaczeniu procesów przeglądu i komunikacji między zespołami Dev i Ops. Piąta część omawia praktyki ciągłego uczenia się i eksperymentowania, zwracając uwagę na rolę kultury organizacyjnej w przyspieszaniu procesu uczenia się, rezerwowanie czasu na rozwój i organizowanie procesów globalizacji wiedzy z pierwszych czterech części książki. Ostatnia część książki porusza zagadnienia związane z bezpieczeństwem informacji, zgodnością i zarządzaniem zmianami, podkreślając integrację bezpieczeństwa w procesie DevOps poprzez wykorzystanie współdzielonych repozytoriów kodu, poprawę telemetrii oraz ochronę potoków wdrożeniowych, równocześnie osiągając cele zarządzania zmianami. Podsumowując, *The DevOps Handbook* dostarcza praktycznych wskazówek umożliwiających organizacjom skuteczne wdrożenie praktyk DevOps, które prowadzą do poprawy wydajności, jakości i bezpieczeństwa procesów technologicznych.

Autorzy pracy *On the importance of CI/CD practices for database applications* [16] przeprowadzają analizę zastosowania praktyk ciągłej integracji (CI) i ciągłego dostarczania (CD) w kontekście rozwoju aplikacji bazodanowych. Podkreślają, że chociaż CI/CD jest standardem w inżynierii oprogramowania, implementacja tych praktyk w projektach bazodanowych jest ograniczona z powodu na trudności związanych ze złożonością techniczną, organizacyjną oraz koniecznością zarządzania ewoluującymi danymi. Autorzy opracowali trzy studia przypadków, które dotyczyły implementacji CI/CD w projektach z wykorzystaniem bazy danych Oracle. Badania wykazały, że wdrożenie automatyzacji przyniosło korzyści, tj. redukcja liczby nieudanych wdrożeń (nawet o 90% w środowiskach testowych), zwiększenie stabilności systemów czy poprawienie jakości kodu. Automatyzacja pozwoliła również na częstsze wdrożenia i zwiększyła niezależność zespołu. Wyniki potwierdzają, że implementacja CI/CD w projektach bazodanowych podnosi efektywność i jakość procesów inżynieryjnych, umożliwiając lepsze zarządzanie zmianami oraz dostosowanie systemów do dynamicznych wymagań.

Artykuł *Measuring the benefits of ci/cd practices for database application development* [15], gdzie ci sami badacze, również analizują zastosowanie praktyk CI/CD w kontekście rozwoju aplikacji bazodanowych. Z jednej strony wskazują oni na korzyści wynikające z implementacji CI/CD, takie jak przyspieszenie cyklu wdrożeniowego, eliminacja prac manualnych oraz wzrost pewności programistów w realizacji zmian. Z drugiej strony identyfikują wyzwania charakterystyczne dla aplikacji bazodanowych, w tym kwestie związane z ewolucją schematów baz danych, testowaniem oraz utrzymaniem spójności w procesach zautomatyzowanych. Autorzy rekomendują eliminację sztywnych harmonogramów wdrożeniowych, co może przyczynić się do skrócenia czasu dostarczania zmian oraz zwiększenia elastyczności procesów. Zwracają także uwagę na konieczność wdrożenia mechanizmów au-

tomatycznego przywracania środowisk, co zapewni większe bezpieczeństwo w przypadku niepowodzeń wdrożeń. Przejście z modelu pracy na gałęziach funkcjonalnych na trunk-based development wskazano jako sposób na minimalizację konfliktów scalania i ryzyka związanego z wdrożeniami. Dodatkowo, zastosowanie repozytoriów artefaktów do przechowywania przetestowanych wersji aplikacji mogłoby poprawić bezpieczeństwo procesu CI/CD. Badanie potwierdza, że praktyki CI/CD usprawniają rozwój aplikacji bazodanowych, zmniejszając liczbę błędów wdrożeniowych oraz podnosząc stabilność systemów. Zaproponowana architektura pipeline'u CI/CD może pełnić rolę wzorcowego rozwiązania w implementacjach tego typu, uwzględniając kluczowe warunki wstępne, takie jak strategie zarządzania wersjami, narzędzia migracyjne oraz systemy monitorowania. Pomimo wyzwań związanych z implementacją, automatyzacja procesów zarządzania zmianami w bazach danych została wskazana jako kluczowy obszar dalszych inwestycji i badań w tej dziedzinie.

W artykule *"From DevOps to DevDataOps: Data Management in DevOps processes"* [7] autorzy przedstawiają DevOps jako metodę zarządzającą cyklem życia oprogramowania. Zwrócono uwagę, że bazowym elementem tego podejścia jest zarządzanie danymi, które stanowią swego rodzaju "paliwo" całego procesu. Artykuł bada zagadnienia związane z zarządzaniem danymi w DevOps, omawiając wyzwania oraz rozwiązania, które czerpią inspirację z technologii BigData i podejścia DataOps. Dodatkowo jeden z rozdziałów skupia się na opracowaniu przepływu danych (*data flow pipeline*) w DevOps oraz implementacji podejścia DataOps. Na rysunku 2.1. przedstawiono opracowany przez autorów przepływ danych.



Rysunek 2.1 Potok przepływu danych DevOps [7]

Na diagramie przedstawionym na rysunku 2.1. pokazano kolejne etapy przetwarzania danych w potoku DevOps. Pierwszym elementem jest generacja danych przez każdy z modułów w ramach potoku DevOps. Każdy moduł jest odpowiedzialny za generowanie strumienia logów raportujących przebieg jego działania, powinno to być realizowane poprzez dedykowany monitorujący proces, który rejestruje wszelkie zdarzenia i operacje wykonywane przez dany moduł. Potem następuje zbieranie wcześniej wygenerowanych logów w jednym miejscu. Następnie na podstawie znaczników czasowych, logi są ponownie porządkowane i grupowane, co pozwala uzyskać migawkę stanu całego procesu DevOps w określonych przedziałach czasowych. Kolejno logi są poddawane filtrowaniu. Robi się to w celu usunięcia danych odstających, duplikatów lub obserwacji zawierających błędy. Dzięki strumieniowemu przetwarzaniu logów i ich analizie, która odbywa się niemalże w czasie rzeczywistym, możliwe jest szybkie wykrycie potencjalnych awarii czy błędów w konkretnym etapie pipeline'u. W przypadku wykrycia nieprawidłowości, system automatycznie generuje komunikaty ostrzegawcze lub błędów. Po takim przetworzeniu logi są agregowane, czyli łączone w formie podsumowania. Ma to na celu skompresowanie dużej ilości danych. Dzięki agregacji uzyskuje się bardziej zwięzłą i czytelną formę, ułatwiającą dalszą analizę statystyczną. Dopiero po wykonaniu powyższych kroków logi są udostępniane użytkownikom końcowym oraz aplikacjom. Po dostarczeniu dane są ponownie sortowane i grupowane, aby w każdym przedziale czasowym uzyskać pełny obraz działania systemu. Dane pochodzące z różnych źródeł są integrowane do jednolitego schematu. Umożliwia to uzyskanie spójnego dostępu do danych bez konieczności stosowania indywidualnych rozwiązań dla każdego formatu logów. Na samym końcu dane są organizowane, aktualizowane i przechowywane, co umożliwia ich późniejsze wykorzystanie. Zgromadzone logi poddawane są analizie, której celem jest wykrycie korelacji oraz przyczyn błędów, a także prognozowanie przyszłych zdarzeń i wspieranie podejmowania decyzji. Autorzy pracy dodatkowo dają cenne wskazówki w jaki sposób potok przepływu danych DevOps powinien zostać zaimplementowany. Na samym początku powinno się stworzyć łańcuch narzędziowy wykorzystujący rozwiązania do przetwarzania danych masowych, które umożliwiają definiowanie oraz koordynację przepływów informacji. Następnie cały system wdrażany jest przy użyciu narzędzi automatyzujących, np. Jenkins, w połączeniu z technologiami konteneryzacji (Docker, Kubernetes). Pozwali to na elastyczną implementację zarówno w środowiskach chmurowych, jak i lokalnych. Kolejnym etapem jest monitorowanie funkcjonowania systemu oraz infrastruktury.

Autorzy artykułu *Database Migration Service With A Microservice Architecture* [33] przedstawiają badanie nad migracją baz danych z wykorzystaniem architektury mikroserwisów. Zajmują się oni złożonościami migracji baz danych między różnymi systemami zarządzania bazami danych (DBMS) i wprowadzają nowatorski framework oparty na mikroserwisach, który radzi sobie z tymi wyzwaniami. Każde zadanie migracyjne w tym frameworku traktowane jest jako odrębny mikroserwis. Takie podejście umożliwia większą skalowalność i efektywność. W artykule zostały wskazane problemy na które można natrafić podczas migracji bazy danych, tj. niespójność danych, słaba organizacja procesu migracji, brak spójnej strategii migracji, niewłaściwe mapowanie danych, brak weryfikacji migracji. Stworzony przez autorów framework obniża koszty migracji oraz minimalizuje szanse utraty danych i niespójności podczas migracji. Warto wspomnieć, że stworzony przez autorów framework wspiera migrację między różnymi DBMS takimi jak Oracle, Microsoft SQL Server i MySQL. Artykuł podkreśla, że dzięki wykorzystaniu mikroserwisów, zaproponowany framework oferuje nowoczesne, elastyczne rozwiązanie dla migracji baz danych.

W przeglądzie literatury wskazano, że współczesne systemy informatyczne oparte na architekturze mikroserwisowej, konteneryzacji i rozwiązaniach chmurowych wymagają wdrożenia narzędzi DevOps w celu zapewnienia ciągłej integracji i dostarczania oprogramowania (CI/CD), automatyzacji procesów, dynamicznego skalowania oraz efektywnego zarządzania w środowiskach rozproszonych. Konieczne staje się przy tym utrzymanie wysokiej dostępności, elastyczności i bezpieczeństwa, co wiąże się z implementacją mechanizmów monitorowania, kontroli wersji oraz automatycznych procedur cofania wprowadzonych zmian, aby zachować stabilność operacyjną w szybko zmieniającym się otoczeniu technologicznym. Analiza praktyk DevOps wskazuje, że kultura współpracy między zespołami deweloperskimi i operacyjnymi, wsparta automatyzacją, redukuje dług techniczny oraz poprawia jakość i stabilność wdrożeń. Techniczne praktyki przepływu i przekazywania informacji zwrotnej przyspieszają cykl rozwoju oprogramowania, a integracja bezpieczeństwa na etapie tworzenia i wdrożeń minimalizuje ryzyko naruszeń. W kontekście aplikacji bazodanowych implementacja CI/CD znacząco ogranicza liczbę błędów wdrożeniowych i zapewnia spójność schematu danych. Podejście DevDataOps uwypukla rolę potoku przepływu danych, w którym generacja, zbieranie, filtrowanie i agregacja logów pozwalają na niemalże w czasie rzeczywistym monitorowanie stanu systemu oraz wczesne wykrywanie anomalii. W obszarze migracji baz danych architektura mikroserwisowa traktuje pojedyncze zadania migracyjne jako odrębne usługi, co zwiększa skalowalność, redukuje koszty i minimalizuje ryzyko niespójności podczas przenoszenia danych między różnymi DBMS, wspierając się konteneryzacją i narzędziami automatyzującymi.

Na podstawie powyższego przeglądu literaturowego zdefiniowano następujące pytania badawcze:

- RQ1: Jakie są specyficzne potrzeby i wymagania dotyczące zarządzania zmianami w bazach danych w kontekście aplikacji opartych na CI/CD?
- RQ2: Jakie parametry są kluczowe dla oceny efektywności narzędzi do zarządzania zmianami w bazach danych w środowiskach CI/CD?
- RQ3: Jak zdefiniować wieloaspektowy model efektywności narzędzi CI/CD do zarządzania zmianami w bazach danych, uwzględniający wydajność, niezawodność oraz skuteczność?

2.2 Analiza potrzeb i wymagań

W kontekście zarządzania zmianami w bazach danych, zwłaszcza w ramach praktyk CI/CD, istotne jest określenie wymagań, które musi spełniać narzędzie do migracji i zarządzania zmianami w bazach danych. Analiza potrzeb i wymagań jest kluczowa dla zapewnienia, że takie narzędzie będzie efektywne, bezpieczne i umożliwi łatwe wprowadzanie zmian w środowisku produkcyjnym i testowym.

Jednym z podstawowych wymagań, które musi spełniać narzędzie do zarządzania zmianami w bazach danych jest możliwość automatyzacji wszystkich etapów procesu migracji, w tym chociażby synchronizację schematów bazy danych. Automatyzacja pozwali na eliminację błędów związanych z manualnym zarządzaniem wersjami, przyspiesza procesy wdrożeniowe i minimalizuje ryzyko błędów ludzkich. Narzędzie powinno umożliwiać bezproblemową integrację z systemami CI/CD, zapewniając w ten sposób szybkie i kontrolowane dostarczanie zmian do baz danych.

Kolejnym podstawowym wymaganiem jest kontrola wersji, która pozwoli na śledzenie i zarządzanie każdą zmianą w bazie danych, zarówno w kontekście struktury, jak i danych. Dzięki wersjonowaniu możliwe jest zarządzanie historią zmian, co ułatwia identyfikację ewentualnych błędów oraz przywracanie poprzednich wersji w razie potrzeby.

Narzędzie do zarządzania bazami danych musi wspierać migrację danych między różnymi wersjami schematów, uwzględniając konwersję danych oraz tłumaczenie schematów między różnymi systemami bazodanowymi. Powinno to obejmować zarówno prostą migrację schematów, jak i bardziej zaawansowane techniki.

Kolejnym kluczowym aspektem jest bezpieczeństwo w każdym etapie procesu migracji. Narzędzie musi zapewniać mechanizmy ochrony danych podczas migracji oraz umożliwiać szybkie odzyskiwanie danych w przypadku awarii. Wymaga to integracji z systemami backupowymi i procedurami rollbacku.

Zarządzanie zmianami w bazach danych musi zapewnić spójność i integralność danych w czasie rzeczywistym. Narzędzie powinno wspierać synchronizację danych pomiędzy różnymi środowiskami oraz umożliwiać wykrywanie i rozwiązywanie konfliktów, które mogą wystąpić w przypadku równoczesnych zmian w różnych środowiskach.

Narzędzie musi umożliwiać przeprowadzanie testów w ramach procesu migracji danych oraz weryfikację, czy nowe zmiany w schemacie i danych są zgodne z oczekiwaniami.

Z racji, że współczesne środowiska IT są często oparte na chmurze i konteneryzacji (np. Docker, Kubernetes). Narzędzie do zarządzania zmianami w bazach danych musi być w stanie obsługiwać migracje baz danych w takich środowiskach, zapewniając pełną elastyczność i skalowalność.

Kolejnym istotnym wymaganiem jest możliwość monitorowania procesów migracyjnych oraz prowadzenie audytu. Narzędzie powinno zapewniać pełną widoczność zmian w bazach danych, umożliwiając śledzenie wprowadzonych modyfikacji, czas ich realizacji oraz identyfikację ewentualnych problemów. Monitoring oraz audyt pozwala na szybkie reagowanie w przypadku wystąpienia błędów oraz na zapewnienie zgodności z wymaganiami audytowymi i bezpieczeństwa.

2.3 Krótki opis dostępnych narzędzi

- **Flyway**

  Flyway [9] to narzędzie open-source do zarządzania migracjami bazy danych i wersjonowania zmian w schemacie bazy danych. Obsługuje szeroki wachlarz baz danych, takich jak PostgreSQL, MySQL, Oracle i SQL Server. Flyway pozwala na łatwe zarządzanie zmianami w schemacie bazy danych poprzez kontrolę wersji, zapewniając spójność migracji w różnych środowiskach. Narzędzie oferuje prosty interfejs wiersza poleceń, umożliwiający powtarzalne migracje oraz cofanie migracji w razie potrzeby. Flyway integruje się również z narzędziami do Continuous Integration (CI) i innymi systemami budowania.

- **Liquibase**

  Liquibase [9] to elastyczne, open-source'owe narzędzie, które ułatwia wersjonowanie i migrację schematu bazy danych w wielu relacyjnych bazach danych, takich jak MySQL, PostgreSQL, Oracle czy SQL Server. Obsługuje definicję zmian w bazie danych za pomocą składni XML lub YAML, umożliwiając rollback (cofanie zmian), migracje powtarzalne oraz wersjonowanie zmian. Liquibase integruje się z pipeline'ami CI oraz narzędziami do budowania, a także oferuje rozszerzenia do obsługi baz danych nierelacyjnych, takich jak Cassandra, MongoDB czy Couchbase.

- **Bytebase**

  Bytebase [6] pełni funkcję pośrednika pomiędzy użytkownikiem a systemem zarządzania bazą danych, oferując kompleksowe rozwiązanie typu *all-in-one* do zarządzania całym cyklem życia zmian w bazie danych. Narzędzie eliminuje konieczność korzystania z rozproszonych komponentów, konsolidując procesy w pojedynczym, spójnym przepływie pracy. W ramach tego przepływu zmiany w schemacie lub danych mogą być inicjowane, poddawane wstępnej weryfikacji, zatwierdzane i automatycznie wdrażane od etapu deweloperskiego aż po środowisko produkcyjne. Bytebase jest jedną z najpopularniejszych alternatyw dla narzędzi takich jak Liquibase i Flyway, a jego tempo rozwoju na platformie GitHub przewyższa oba konkurencyjne rozwiązania.

- **ApexSQL Diff**

  ApexSQL Diff [9] to narzędzie przeznaczone głównie do porównywania i synchronizowania zmian w schemacie i danych baz danych SQL Server. Obsługuje różne tryby porównań, takie jak porównanie obiekt po obiekcie, porównanie z migawką lub porównanie projektów. Narzędzie zapewnia interfejs wiersza poleceń do automatyzacji procesów oraz integrację z systemami kontroli wersji. ApexSQL Diff umożliwia generowanie szczegółowych raportów i logów, co jest przydatne w audytach i zapewnianiu zgodności z wymaganiami.

- **Fluent migrations**

  Fluent Migrations [9] to biblioteka open-source umożliwiająca zarządzanie migracjami schematu bazy danych za pomocą składni fluent API w kodzie. Obsługuje wiele baz danych, takich jak SQL Server, PostgreSQL, MySQL, SQLite, Oracle i Firebird. Narzędzie wspiera wersjonowanie i powtarzalne migracje, zapewniając spójność i strukturę przy stosowaniu zmian w bazie danych. Fluent Migrations posiada interfejs wiersza poleceń do wykonywania migracji i integracji z procesami budowy

i wdrożenia, a także oferuje rozbudowaną architekturę umożliwiającą dostosowanie funkcjonalności narzędzia do indywidualnych potrzeb.

- **Evolve**

  Evolve [9] to narzędzie open-source dla projektów opartych na .NET i .NET Core, które upraszcza proces aplikowania zmian w schemacie bazy danych. Inspirowane narzędziem Flyway, Evolve obsługuje bazy danych takie jak SQL Server, PostgreSQL, MySQL, SQLite, MariaDB, Cassandra czy CockroachDB. Migracje są definiowane za pomocą plików YAML lub JSON, a narzędzie zapewnia interfejs wiersza poleceń do wykonywania migracji. Evolve łatwo integruje się z narzędziami do budowania i wdrożeń, takimi jak Jenkins czy Octopus Deploy, i pozwala na rozszerzanie funkcjonalności, co umożliwia dostosowanie narzędzia do specyficznych potrzeb projektu.

- **Narzędzia do migracji baz danych NoSQL**

  Istnieje kilka narzędzi dedykowanych migracjom baz danych NoSQL [9]. MongoDB Database Migration Tool to narzędzie wiersza poleceń umożliwiające migrację danych między instancjami MongoDB. Amazon DynamoDB Data Pipeline to usługa AWS, która pozwala na migrację danych do Amazon DynamoDB oraz innych usług AWS. Cassandra Migrator to narzędzie do migracji danych między bazą danych Apache Cassandra a innymi systemami, a Redis Migrate Tool umożliwia migrację danych pomiędzy instancjami Redis, obsługując różne struktury danych i migracje w tle. Wszystkie te narzędzia ułatwiają migrację danych w środowiskach baz danych NoSQL.

2.4 Zarządzanie zmianami w bazach danych

Dane stanowią kluczowy zasób współczesnych systemów informacyjnych, będąc fundamentem działalności takich technologii jak smartfony, urządzenia IoT *(Internet of Things)* czy infrastruktura serwerowa [48]. Wpływają one na każdy aspekt funkcjonowania organizacji, dlatego skuteczne zarządzanie danymi jest niezbędne do zapewnienia ich wysokiej jakości oraz spójności. Zarządzanie zmianami w bazach danych (Database Change Management, DCM) [70] jest kluczowym procesem w cyklu życia baz danych, który obejmuje monitorowanie, dokumentowanie oraz kontrolowanie modyfikacji zarówno w strukturze bazy danych (takich jak tabele, kolumny, relacje między danymi), jak i w samych danych przechowywanych w bazie. Celem tego procesu jest utrzymanie spójności, integralności oraz kontroli nad rozwojem bazy danych w środowiskach produkcyjnych i testowych. Integracja zarządzania zmianami z praktykami CI/CD pozwala na automatyzację migracji baz danych, synchronizacji schematów oraz zapewnienia spójności danych w różnych środowiskach. Dzięki CI/CD zmiany w strukturze bazy danych (np. dodawanie nowych tabel, kolumn czy modyfikowanie relacji) mogą być wprowadzane w sposób kontrolowany, bezpieczny i zminimalizować ryzyko błędów, a cały proces wspierany jest przez automatyczne testy oraz monitorowanie, co umożliwia szybkie wdrażanie nowych wersji bazy danych w cyklu życia aplikacji. Z powyższego przeglądu literatury wiadomo, że integracja zarządzania zmianami w bazach danych z praktykami CI/CD wiąże się z wyzwaniami takimi jak problemy związane z wersjonowaniem schematów baz danych, migracje danych w cyklu życia aplikacji oraz synchronizacja zmian w strukturze bazy danych z kodem aplikacji. Jednakże istnieją techniki zarządzania zmianami w bazach danych, które są w stanie wprowadzić praktyki CI/CD.

2.4.1 Kontrola wersji

Kontrola wersji w kontekście procesów CI/CD w aplikacjach bazodanowych obejmuje zarządzanie oraz śledzenie zmian w elementach bazy danych, a także automatyzację oraz uporządkowanie procesu wdrażania tych zmian w sposób powtarzalny, spójny i bezpieczny. W klasycznych projektach pozbawionych warstwy bazodanowej kontrola wersji sprowadza się przede wszystkim do monitorowania zmian w kodzie źródłowym. Natomiast w aplikacjach, które operują na relacyjnych lub nierelacyjnych bazach danych, zachodzi konieczność zarządzania dodatkowym obszarem zmian, danymi i ich strukturą, które nie tylko ewoluują wraz z rozwojem logiki biznesowej, lecz także muszą pozostać spójne i integralne w trakcie aktualizacji oprogramowania.

2.4.2 Migracja danych

Migracja baz danych to proces, w którym przekształca się wszystkie istotne komponenty aplikacji bazodanowej z jednego środowiska do ich odpowiedników w środowisku docelowym. W tym procesie uwzględnia się przede wszystkim tłumaczenie schematu bazy danych oraz konwersję danych [46]. Tłumaczenie schematu bazy danych składa się z dwóch faz. Pierwszy etap, określany jako inżynieria wsteczna baz danych *(DataBase Reverse Engineering, DBRE)*, ma na celu rekonstrukcję schematu konceptualnego, takiego jak model związków encji (ERM), który odzwierciedla zarówno jawne, jak i ukryte semantyki danych w schemacie źródłowym. Jawna semantyka obejmuje takie elementy jak relacje, atrybuty, klucze oraz zależności danych. Ukryta semantyka, która nie jest przedstawiona w relacyjnych bazach danych, to na przykład związki pomiędzy danymi. Druga faza, określana jako inżynieria do przodu baz danych *(DataBase Forward Engineering, DBFE)*, koncentruje się na opracowaniu fizycznego schematu docelowego na podstawie schematu konceptualnego wygenerowanego w pierwszym etapie. Alternatywnie, możliwe jest bezpośrednie przekształcenie schematu źródłowego na schemat docelowy, bez konieczności stosowania reprezentacji pośredniej. W celu uzupełnienia brakujących semantyk lub dopracowania wynikowego schematu tak, aby w pełni wykorzystywał możliwości docelowego modelu bazy danych, może być konieczne zaangażowanie użytkownika lub odpowiedniego narzędzia.

2.4.3 Cofanie zmian i odzyskiwanie danych

Cofanie zmian polega na przywróceniu struktury lub zawartości bazy danych do stanu sprzed wykonania konkretnych operacji modyfikujących schemat lub dane. Mechanizm ten opiera się na zasadzie, zgodnie z którą każda zmiana jest zapisywana w odpowiednim dzienniku, co umożliwi późniejszą identyfikację, odwrócenie i powtórne odtworzenie operacji.

Odzyskiwanie danych stanowi uzupełnienie procesu cofania zmian, obejmując przywrócenie treści bazy danych do spójnego stanu operacyjnego. Proces ten wykorzystuje kopie zapasowe oraz punkty przywracania zgromadzone przez system zarządzania bazą danych [69]. W przypadku awarii lub niepoprawnego wdrożenia migracji, administrator systemu inicjuje procedurę przywracania bazy z ostatniego spójnego backupu, a następnie może zastosować techniki odtwarzania, aby uwzględnić zmiany zaakceptowane pomiędzy wykonaniem backupu, a momentem awarii.

Rozdział 3

Aplikacja demonstracyjna

Aplikacja demonstracyjna została zaprojektowana w celu przeprowadzenia eksperymentalnej oceny narzędzi Flyway oraz Liquibase w aspekcie migracji schematu bazy danych PostgreSQL w ramach procesu CI/CD. Wyboru tych dwóch rozwiązań dokonano ze względu na ich ugruntowaną pozycję rynkową, długoletnią historię rozwoju oraz rozbudowane wsparcie społeczności, co przekłada się na stabilność, bezpieczeństwo i dostępność bogatego ekosystemu wtyczek. Oba narzędzia oferują natywną kompatybilność ze wszystkimi powszechnie stosowanymi silnikami relacyjnymi, w tym PostgreSQL, MySQL, Oracle i SQL Server, oraz zapewniają spójne interfejsy CLI i API do automatyzacji zadań migracyjnych, co czyni je uniwersalnym punktem odniesienia niezależnie od wybranego środowiska bazodanowego. Pominięto takie rozwiązanie jak Bytebase z uwagi na fakt, że pomimo swojej rosnącej popularności jako kompleksowa platforma zarządzania cyklem życia zmian w bazie danych, oferuje funkcjonalności wykraczające poza samą migrację schematu, chociażby takie jak wbudowane GUI i rozbudowane zarządzanie uprawnieniami, oraz dysponuje wciąż rozwijającym się systemem wtyczek, nie uwzględniono jej w badaniu, aby uniknąć nadmiernego skomplikowania analizy. Eksperymenty zostały przeprowadzone wyłącznie na silniku PostgreSQL, co pozwoliło utrzymać jednolite warunki testowe i skoncentrować się na porównaniu efektywności, ergonomii oraz sposobu integracji tych narzędzi z procesem ciągłej integracji, eliminując przy tym zmienność wyników wynikającą z różnic między systemami bazodanowymi.

3.1 Opis aplikacji

Aplikacja demonstracyjna obejmuje wszystkie aspekty, które były potrzebne do przeprowadzenia badań. Zawiera zautomatyzowany potok wdrożeniowy, implementację backendu i frontendu oraz skrypty potrzebne do stworzenia bazy danych i obróbki danych. Cała struktura projektu widoczna jest w sekcji 7.1, a kod projektu jest dostępny w repozytorium GitHub [1].

3.1.1 Potoki wdrożeniowe Github Actions

Zaimplementowano dwa niezależne potoki CI/CD dedykowane warstwom frontend oraz backend. Potok `ci-frontend.yml` uruchamiany jest przy każdym zdarzeniu `push` lub `pull request` do gałęzi `main`, w przypadku modyfikacji plików w katalogu frontend. W pierwszym etapie tworzona jest maszyna wirtualna z systemem Ubuntu w wersji 24.04 [21], na której następnie wykonywane są kolejne kroki potoku: klonowanie repozytorium za pomocą akcji `actions/checkout v4` [22], konfiguracja środowiska Node.js 18 [26], instalacja

zależności przy użyciu `npm ci` oraz budowanie aplikacji Angular w trybie produkcyjnym poprzez `npm run build --configuration production` [3]. Ostatecznie, jeżeli w projekcie zdefiniowano skrypt `e2e`, są uruchamiane testy end-to-end. Kod potoku przedstawiono w listingu 3.1.

```
name: CI Frontend (Ubuntu)
on:
  push:
    branches: [main]
    paths:
      - 'frontend/**'
  pull_request:
    branches: [main]
    paths:
      - 'frontend/**'
jobs:
  build-and-test:
    runs-on: ubuntu-latest

    steps:
      - name: Checkout repo
        uses: actions/checkout@v4

      - name: Set up Node.js 18
        uses: actions/setup-node@v3
        with:
          node-version: '18'

      - name: Install dependencies
        working-directory: frontend
        run: npm ci

      - name: Build Angular app
        working-directory: frontend
        run: npm run build -- --configuration production

      - name: Run end-to-end tests (if configured)
        working-directory: frontend
        run: |
          if npm run | grep -q 'e2e'; then
            npm run e2e
          else
            echo "No e2e script defined-skipping."
          fi
```

Listing 3.1 Potok wdrożeniowy `ci-frontend.yml`

Analogicznie skonfigurowano potok `ci-backend.yml`, wyzwalany przy zmianach w katalogu backend na gałęzi `main`. Po inicjalizacji środowiska Ubuntu instalowany jest serwis PostgreSQL oraz definiowane są zmienne środowiskowe niezbędne do komunikacji z bazą danych w procesie CI [24]. Repozytorium pobierane jest ponownie przy użyciu `actions/checkout`, a następnie poprzez akcję `setup-java` instalowane jest i konfigurowane środowisko OpenJDK 17 (Temurin) [25], przy czym wartość zmiennej `JAVA_HOME` jest

dodawana jako zmienna środowiskowa. Kolejnym krokiem jest uruchomienie faz Maven, `clean` oraz `package` z opcją `-DskipTests` [30], co umożliwia kompilację oraz pakowanie artefaktu JAR bez wykonywania testów jednostkowych. Następnie akcja `setup-python` instaluje środowisko Python 3.11 [27], po czym instalowane są biblioteki `psycopg2` [28] oraz `faker` [31] do obsługi połączenia z bazą i generacji danych testowych. Przygotowywane są katalogi oraz pliki CSV służące do rejestracji metryk, po czym inicjalizowana jest baza danych. W dalszym etapie pobierana i instalowana jest wersja 10.20.1 narzędzia Flyway CLI z Maven Central, rozpakowywana do katalogu domowego, uprawnienia są nadawane w tym samym kroku, a ścieżka do wykonywalnego pliku Flyway dodawana jest do zmiennej `PATH` [66]. Następnie, z użyciem `flyway info -baselineOnMigrate=true`, inicjalizowana jest tabela historii schematu, po czym wykonywane są migracje do określonych wersji (`flyway migrate --target`) oraz cofnięcia zmian realizowane poprzez skrypty `UNDO` i ręczne usuwanie wpisów z tabeli historii. Czas każdego etapu, liczba linii w skryptach SQL, kod wyjścia oraz stopień sukcesu migracji są mierzone i zapisywane do plików CSV, które następnie za pomocą akcji `actions/upload-artifact@v4` trafiają do artefaktów CI [23]. W sposób analogiczny przeprowadzane są migracje i cofnięcia zmian przy użyciu Liquibase: instalacja narzędzia, wykonanie `liquibase update -- contexts` oraz `liquibase rollbackCount 1 -- contexts`, pomiar czasu operacji oraz rejestracja wyników w formacie CSV [40]. Przykładowe uruchomienie migracji w potoku wdrożeniowym `ci-backend.yml` przedstawiono w listingu 3.2.

```
- name: Run Flyway migrations & rollbacks
  shell: bash
  run: |
    for iter in {1..100}; do
      for ctx in {2..14}; do
        START=$(date +%s%N)
        $HOME/flyway-10.20.1/flyway \
          -url="$DB_URL" -user="$DB_USER" -password="$DB_PASS" \
          -baselineOnMigrate=true \
          -locations=filesystem:backend/src/main/resources/db/migration \
          migrate -target="${ctx}"
        EXIT=$?
        DURATION=$(( $(date +%s%N) - START ))
        LINES=$(wc -l backend/src/main/resources/db/migration/V${ctx}__*.sql |
    awk '{print $1}')
        if [ $EXIT -eq 0 ]; then
          SUCCESS_RATE=100.0
        else
          SUCCESS_RATE=0.0
        fi
        echo "Flyway,Scen1,${ctx},${DURATION},0,${EXIT},${LINES},0,0,${
    SUCCESS_RATE}" \
          >> metricsFlyway/flyway.csv
      done
    done
  env:
    PGHOST: localhost

- name: Run Liquibase migrations & rollbacks
  shell: bash
```

```
run: |
  set +e
  for iter in {1..100}; do
    for ctx in {1..13}; do
      START=$(date +%s%N)
      liquibase \
        --changeLogFile=backend/src/main/resources/db/changelog/db.changelog-
 master.xml \
        --url="$DB_URL" --username="$DB_USER" --password="$DB_PASS" \
        update --contexts="${ctx}"
      EXIT=$?
      DURATION=$(( $(date +%s%N) - START ))
      LINES=$(grep -c "changeSet id=\"${ctx}\"" \
        backend/src/main/resources/db/changelog/db.changelog-master.xml)
      if [ $EXIT -eq 0 ]; then
        SUCCESS_RATE=100.0
      else
        SUCCESS_RATE=0.0
      fi
      echo "Liquibase,Scen1,${ctx},${DURATION},0,${EXIT},${LINES},0,0,${
 SUCCESS_RATE}" \
        >> metricsLiquibase/liquibase.csv
    done
  done
env:
  JAVA_HOME: ${{ env.JAVA_HOME }}
```

Listing 3.2 Przykładowe uruchomienie migracji w potoku wdrożeniowym `ci-backend.yml`

3.1.2 Backend

Moduł backendu został zaimplementowany jako aplikacja Spring Boot [5], w której poszczególne warstwy funkcjonalne zostały wyodrębnione, a migracje baz danych są realizowane w sposób zautomatyzowany. W celu zapewnienia spójności środowiska kompilacji i uniknięcia różnic pomiędzy lokalnymi stanowiskami oraz systemami CI wykorzystano narzędzie Maven Wrapper [4]. Parametry konfiguracyjne zostały wyeksportowane do pliku `application.yml`, co umożliwia modyfikację zachowania aplikacji bez konieczności wprowadzania zmian w kodzie źródłowym [20]. Kod pliku konfiguracyjnego widoczny w listingu 3.3. Aplikacja wywołuje klasę główną, która uruchamia kontekst Spring oraz wbudowany serwer WWW, a udostępnianie punktów końcowych HTTP odbywa się na poziomie kontrolerów oznaczonych adnotacją `@RestController` [13]. Logika biznesowa zrealizowana została w komponentach oznaczonych jako `@Service`, odpowiedzialnych między innymi za koordynację procesów migracji, monitorowanie zużycia zasobów oraz agregację wyników testów. Dostęp do warstwy persystencji upraszcza klasa `JdbcTemplate`, wstrzykiwana wraz z obiektem `DataSource`, co pozwala na wydajne wykonywanie zapytań SQL.

```
spring:
  datasource:
    url: jdbc:postgresql://localhost:5432/mastersThesis
    username: postgres
    password: postgres
---
spring:
  config:
    activate:
      on-profile: flyway
  flyway:
    enabled: false
    clean-disabled: false
    locations: classpath:db/migration
    baseline-on-migrate: true
  liquibase:
    enabled: false
    change-log: classpath:db/changelog/db.changelog-master.xml
```

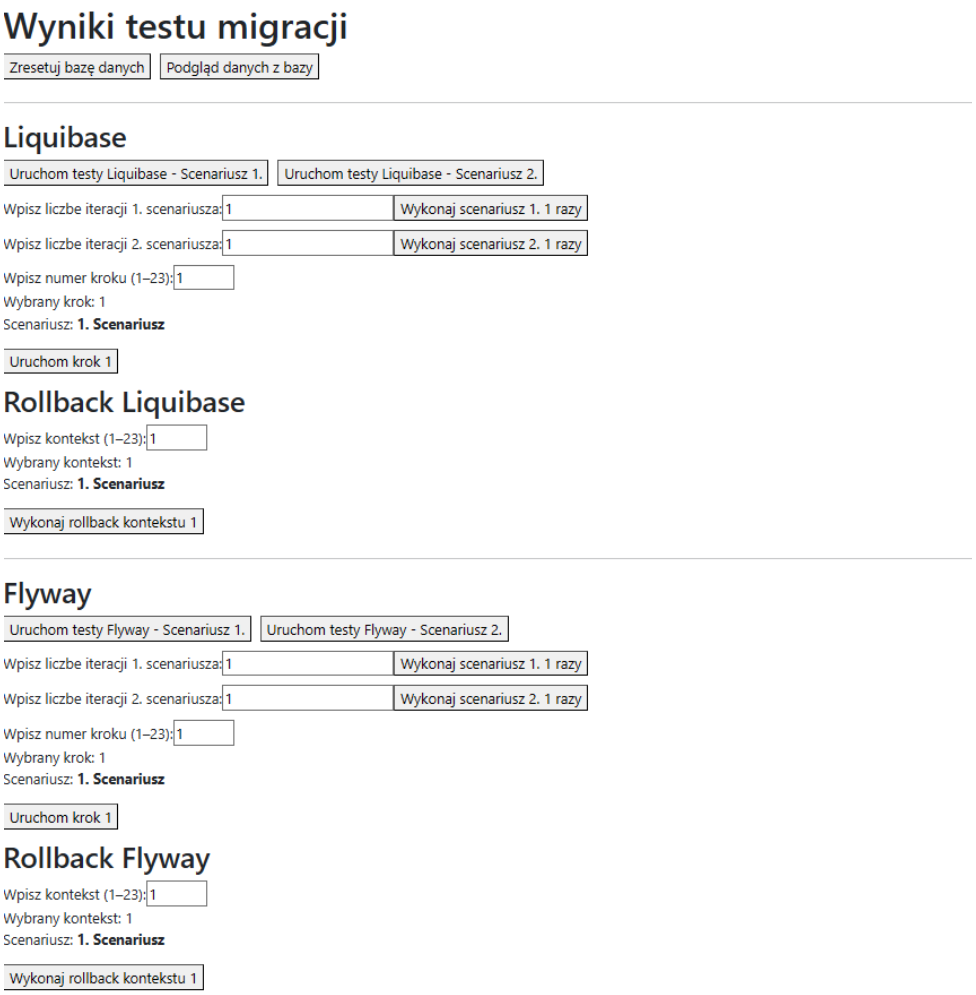
Listing 3.3 Kod pliku konfiguracyjnego `application.yml`

Zarządzanie schematem bazy danych realizowane jest przy użyciu dwóch niezależnych frameworków: Flyway oraz Liquibase. Konfiguracja Flyway odbywa się programowo, wskazywany jest katalog zawierający skrypty SQL, a każda migracja (oraz odpowiadające jej skrypty undo) jest wywoływana, mierząc czas wykonania oraz monitorując zużycie CPU i pamięci. Liquibase natomiast operuje na głównym changelogu w formacie XML, zawierającym zestawy zmian (change sets) pogrupowane według kontekstów. Operacje aktualizacji i wycofania są wykonywane za pomocą standardowych komend Liquibase, zaś równoległe zbieranie metryk dotyczących liczby linii zmian, czasu wykonania i wydajności pozwala na bezpośrednie porównanie obu podejść. Należy zaznaczyć, że w darmowej edycji Flyway skrypty undo muszą być uruchamiane bezpośrednio przez SQL, gdyż narzędzie nie udostępnia dedykowanej komendy do cofania migracji.

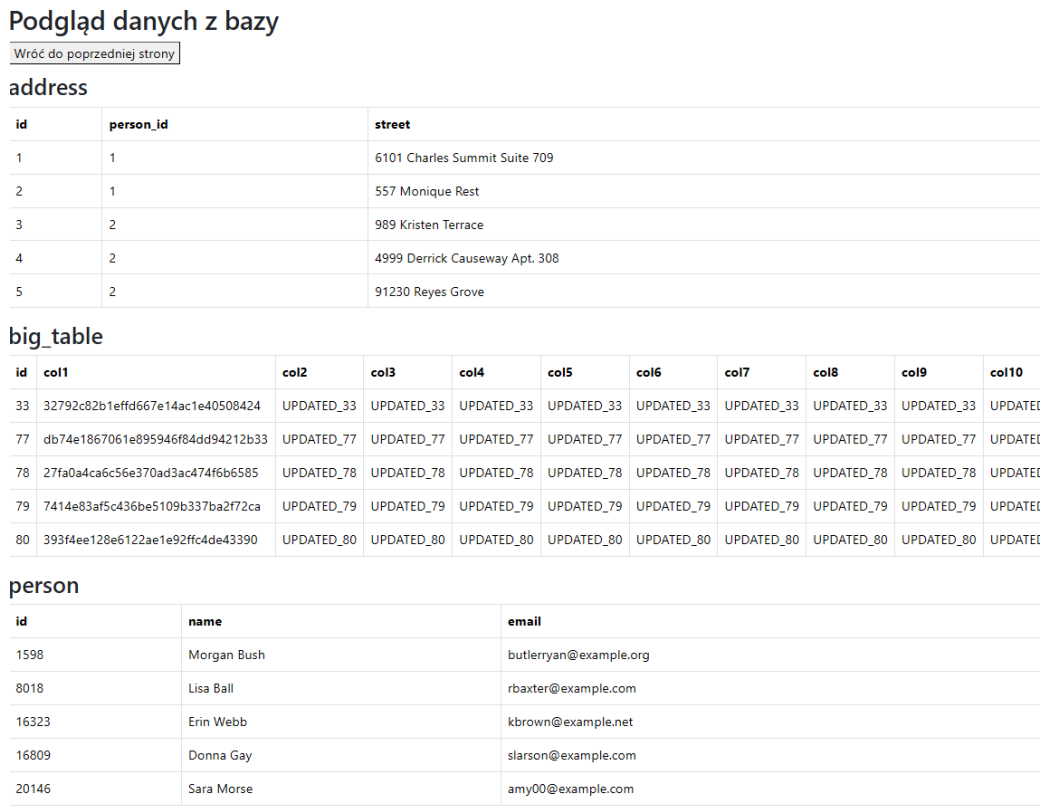
3.1.3 Frontend

Moduł frontend został zrealizowany jako jednostronicowa aplikacja internetowa (*Single Page Application, SPA*) oparta na frameworku Angular [2]. Konfiguracja tras jest uproszczona: domyślna ścieżka renderuje komponent MainComponent, natomiast przejście pod adres `/preview` wywołuje komponent TablePreviewComponent. Każdy z komponentów został zadeklarowany jako standalone, dzięki czemu importowane są jedynie niezbędne dyrektywy i moduły Angulara, CommonModule, FormsModule oraz elementy routingu, takie jak RouterLink i RouterOutlet, zamiast całych NgModules. Takie podejście redukuje rozmiar finalnego pakietu oraz przyspiesza cykl rekompilacji kodu podczas pracy deweloperskiej.

W ramach MainComponent zaimplementowano mechanizmy uruchamiania procesów migracji oraz rollbacków dla obu wspieranych narzędzi. Interfejs umożliwia inicjację pełnych scenariuszy migracyjnych z jednoczesnym pomiarem i wyświetlaniem wszystkich kluczowych metryk. Dodatkowo istnieje możliwość wywołania poszczególnych kroków migracji poprzez wskazanie odpowiedniego numeru kroku, analogicznie do wymuszenia cofnięcia zmian. W interfejsie przewidziano również elementy nawigacyjne pozwalające na przejście do innych sekcji aplikacji oraz funkcję resetu bazy danych. Sekcja pod adresem `/preview` prezentuje aktualny schemat bazy danych w trybie poglądu.



Rysunek 3.1 Strona główna aplikacji demonstracyjnej



Podgląd danych z bazy

Wróć do poprzedniej strony

address

id	person_id	street
1	1	6101 Charles Summit Suite 709
2	1	557 Monique Rest
3	2	989 Kristen Terrace
4	2	4999 Derrick Causeway Apt. 308
5	2	91230 Reyes Grove

big_table

id	col1	col2	col3	col4	col5	col6	col7	col8	col9	col10
33	32792c82b1effd667e14ac1e40508424	UPDATED_33	UPDATED_33	UPDATED_33	UPDATED_33	UPDATED_33	UPDATED_33	UPDATED_33	UPDATED_33	UPDATE
77	db74e1867061e895946f84dd94212b33	UPDATED_77	UPDATED_77	UPDATED_77	UPDATED_77	UPDATED_77	UPDATED_77	UPDATED_77	UPDATED_77	UPDATE
78	27fa0a4ca6c56e370ad3ac474f6b6585	UPDATED_78	UPDATED_78	UPDATED_78	UPDATED_78	UPDATED_78	UPDATED_78	UPDATED_78	UPDATED_78	UPDATE
79	7414e83af5c436be5109b337ba2f72ca	UPDATED_79	UPDATED_79	UPDATED_79	UPDATED_79	UPDATED_79	UPDATED_79	UPDATED_79	UPDATED_79	UPDATE
80	393f4ee128e6122ae1e92ffc4de43390	UPDATED_80	UPDATED_80	UPDATED_80	UPDATED_80	UPDATED_80	UPDATED_80	UPDATED_80	UPDATED_80	UPDATE

person

id	name	email
1598	Morgan Bush	butlerryan@example.org
8018	Lisa Ball	rbaxter@example.com
16323	Erin Webb	kbrown@example.net
16809	Donna Gay	slarson@example.com
20146	Sara Morse	amy00@example.com

Rysunek 3.2 Strona podglądu aktualnego schematu bazy danych

3.1.4 Baza danych

Baza danych dla pierwszego scenariusza składa się z trzech głównych tabel `person`, `address` oraz `orders`.

```
CREATE TABLE person (
    id SERIAL PRIMARY KEY,
    name VARCHAR(100),
    email VARCHAR(100) UNIQUE,
    legacy_name VARCHAR(100)
);
CREATE TABLE address (
    id SERIAL PRIMARY KEY,
    person_id INT NOT NULL REFERENCES person(id),
    street VARCHAR(200),
    city VARCHAR(100),
    zipcode VARCHAR(20)
);
CREATE TABLE orders (
    id SERIAL PRIMARY KEY,
    person_id INT NOT NULL REFERENCES person(id),
    order_date DATE,
    amount NUMERIC(10,2)
);
```

Listing 3.4 Schemat bazy danych wykorzystywanej w scenariuszu 1.

Tabela `person` została zaprojektowana z wykorzystaniem typu `SERIAL` jako klucza głównego w kolumnie `id`, natomiast unikalność wartości zapewniono poprzez nałożenie ograniczenia unikalności na kolumnę `email`. Dane osobowe są przechowywane zarówno w kolumnie `name`, jak i w kolumnie `legacy_name`, przy czym ta druga pełni funkcję pomocniczą w kontekście testów migracyjnych. Tabela `address` definiuje relację jeden-do-wielu (*one-to-many*) względem tabeli `person` poprzez klucz obcy `person_id` i zawiera standardowe atrybuty adresowe. Analogicznie, tabela `orders` referencyjnie powiązana jest z `person(id)` i rejestruje dane transakcyjne, takie jak data zamówienia oraz jego wartość w formacie `NUMERIC(10,2)`.

W celu automatycznego wypełnienia danych w powyższym schemacie przygotowano skrypt `addData.py`, którego zadaniem jest zarówno generowanie danych testowych, jak i reset struktury bazy. Skrypt akceptuje parametr uruchomieniowy `--count`, który określa liczbę generowanych rekordów (domyślnie: 100000). Dane są syntetyzowane przy użyciu biblioteki Faker [31], natomiast połączenie z instancją bazy PostgreSQL realizowane jest przez bibliotekę `psycopg2` [28]. Na etapie inicjalizacji skrypt usuwa istniejące tabele (`DROP TABLE IF EXISTS`) wchodzące w skład schematu oraz dodatkowe struktury powstające podczas procesów migracyjnych, takie jak `databasechangelog`, `flyway_schema_history` oraz `databasechangeloglock`. Następnie odtwarzane są trzy podstawowe tabele zgodnie z definicją przedstawioną w listingu 3.4. Podczas głównej fazy działania, w pętli generacyjnej wywoływane są metody `fake.name()` oraz `fake.unique.email()` w celu uzyskania realistycznych i unikalnych danych osobowych, które następnie wstawiane są do tabeli `person`. Po zakończeniu tego etapu skrypt pobiera wszystkie wygenerowane identyfikatory, po czym iteracyjnie przypisuje do każdego z nich od jednego do trzech adresów oraz od zera do pięciu rekordów zamówień, przy czym daty są losowane z przedziału ostatnich dwunastu miesięcy, a kwoty oscylują w zakresie od 10,00 do 1000,00. Całość operacji finalizowana jest poprzez jedno wywołanie `conn.commit()`, które zapisuje zmiany w bazie danych. Wykonanie skryptu sygnalizowane jest komunikatami wypisywanymi na standardowe wyjście (np. „Start...”, „Finished!”).

3.2 Scenariusze migracji

W ramach badania przewidziano dwa główne scenariusze migracyjne, realizowane zarówno w Flyway, jak i w Liquibase. Dzięki temu możliwe będzie bezpośrednie porównanie wyników obu narzędzi dla identycznych operacji.

3.2.1 Scenariusz 1.

Scenariusz migracji odzwierciedla etapową refaktoryzację modelu tabeli `person` przy użyciu wcześniej zadeklarowanych zestawów zmian (*change sets*), które umożliwią odwracalną i kontrolowaną ewolucję schematu bazy danych. W pierwszej fazie nowa kolumna `new_name` jest wprowadzana obok istniejącego kolumny `legacy_name`. Natychmiast po tym wykonywana jest masowa operacja `UPDATE`, która kopiująca zawartość z kolumny `legacy_name` do `new_name`. Następnie wdrażany jest mechanizm dwukierunkowej synchronizacji pól oparty na wyzwalaczu (*trigger*). Najpierw tworzona jest funkcja `PL/pgSQL person_name_sync`, a następnie wiązana jest z tabelą `person` za pomocą wyzwalacza `BEFORE INSERT OR UPDATE`, co gwarantuje, że modyfikacje pól `new_name` lub `legacy_name` będą konsekwentnie prpagowane na poziomie bazy danych.

Gdy aplikacja w pełni przełączy się na kolumnę `new_name`, starsza kolumna zostanie usunięta, a towarzyszące dotychczas wyzwalacze i funkcje zostaną deaktywowane poprzez ich usunięcie. To dwuetapowe podejście, najpierw usunięcie kolumny, a następnie eliminacja przestarzałych wyzwalaczy i funkcji, separuje zmianę struktury schematu od usuwania logiki biznesowej i minimalizuje ryzyko przerwania działania aplikacji, wzorzec zalecany w ewolucyjnym projektowaniu baz danych [18].

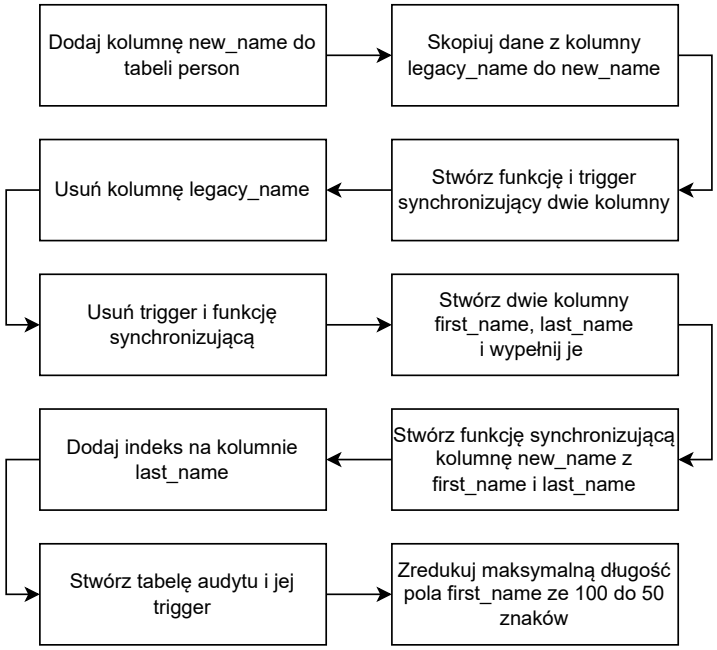
Kolejna faza restrukturyzuje kolumnę `new_name` dodając dwa nowe pola `first_name` i `last_name`. Wykorzystując funkcję `split_part` w SQL, skrypt uzupełniania dane i ustanawia drugą funkcję wyzwalacza `(person_name_sync_v2)` wraz z jej wyzwalaczem w celu utrzymania tych nowych pól przy przyszłych wstawieniach i aktualizacjach. Migracja ta zapewnia spójność bez konieczności stosowania kodu transformacji na poziomie aplikacji.

Następnie optymalizuje się mechanizm odczytu poprzez utworzenie indeksu na kolumnie `last_name` pod warunkiem, że indeks już nie istnieje, co poprawia wydajność zapytań filtrujących po nazwisku. Na samym końcu tworzona jest tabela `person_audit` i odpowiadający jej wyzwalacz `AFTER INSERT OR UPDATE OR DELETE` w celu rejestrowania historii zmian ze znacznikami czasu. Ostateczna migracja redukuje maksymalną długość pola `first_name` ze 100 do 50 znaków dopiero po potwierdzeniu za pomocą warunków wstępnych SQL, że żadne istniejące wartości nie przekraczają nowego limitu, zapewniając przerwanie migracji zamiast cichego obcinania danych.

Cały scenariusz może być stosowany w środowiskach programistycznych, przejściowych i produkcyjnych, umożliwiając bezprzerwową ewolucję modelu bazy danych. Poprzez stopniowe wprowadzanie kolumny `new_name` równolegle obok starszego pola, uzupełnianie danych i organizowanie dwukierunkowej synchronizacji za pomocą wyzwalaczy bazy danych, scenariusz pozwala równocześnie obsługiwać zarówno starsze, jak i nowe wersje aplikacji.

W praktyce zaleca się przeprowadzenie wstępnych etapów (dodanie nowej kolumny i kopiowanie danych) w klastrze deweloperskim lub przejściowym, aby zweryfikować, czy obie wersje aplikacji działają poprawnie. Po walidacji należy wdrożyć logikę wyzwalacza w fazie przejściowej, monitorować jej zachowanie pod obciążeniem (zapewniając brak spadku wydajności), a następnie wycofać starszą kolumnę i wyłączyć jej wyzwalacz w produkcji podczas zaplanowanego okna serwisowego w produkcji.

Takie podejście ilustruje zarządzanie migracjami, obejmujące rozszerzenie schematu, wypełnianie danych, synchronizację, wycofywanie, optymalizację wydajności oraz audyt historii zmian, co stanowi pełny cykl weryfikacji mechanizmu migracji od początku do końca. Diagram przedstawiający kroki scenariusza jest widoczny na rysunku 3.3.



Rysunek 3.3 Diagram - scenariusz 1.

3.2.2 Scenariusz 2.

Ten scenariusz migracji symuluje intensywny proces przetwarzania danych charakterystyczny dla środowisk OLTP (*Online Transaction Processing*) [53] lub procesów ETL (*Extract, Transform, Load*) [52].

W pierwszym kroku definiowana jest nowa tabela o nazwie `big_table` z kluczem podstawowym typu `SERIAL`, a zaraz potem wykonywany jest blok proceduralny `PL/pgSQL`, który zapętla się pięćdziesiąt razy, aby dodać kolumny od `col1` do `col50` jako `VARCHAR(50)` w jednej operacji. Ten wzorzec generowania kolumn symuluje zmieniające się wymagania biznesowe, w którym dokładna liczba atrybutów może być znana tylko w czasie wykonywania.

Następnie tabela jest wypełniana w pięciu partiach po 20000 wierszy każda, przy użyciu poniższej konstrukcji.

```
INSERT INTO big_table (col1)
SELECT md5(random()::text)
FROM generate_series(1,20000);
```

Listing 3.5 Zastosowana konstrukcja dodająca dane

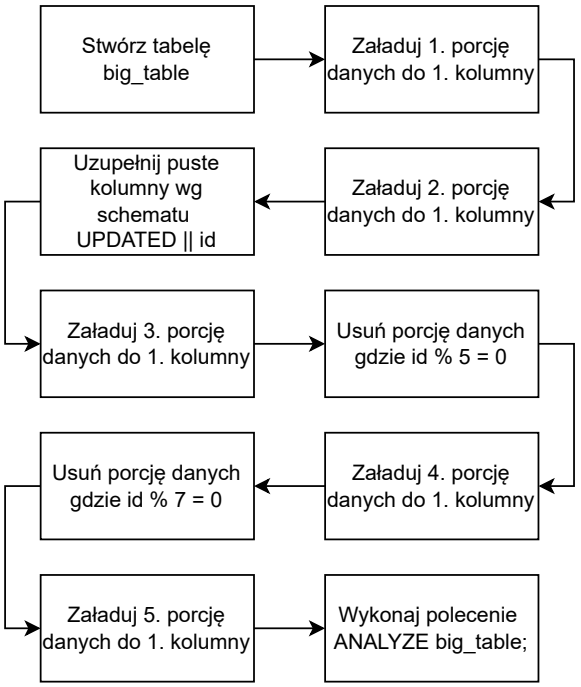
Dzieląc ładowanie na wiele pomniejszych partii, scenariusz umożliwia przyrostowe pozyskiwanie danych, które jest powszechne w potokach ETL, i zapewnia jednocześnie możliwość niezależnego wycofania każdej partii.

Pomiędzy tymi wstawieniami, uruchamiana jest masowa pętla `UPDATE` wypełniająca każdą kolumnę, w której obecnie utrzymuje się wartość `NULL`, poprzedzając każdą wartość prefiksem `UPDATED_ || id`. Reprezentuje to fazę transformacji po załadowaniu, umożliwiając wypełnianie danych pochodnych lub domyślnych bezpośrednio na poziomie bazy.

Kolejne dwa etapy usuwania danych realizują mechanizm usuwania wierszy, których `id` spełnia warunki modułu $id\%5 = 0$, a następnie $id\%7 = 0$, symulując egzekwowanie polityki przechowywania lub czyszczenie nieaktualnych partycji. Warto zauważyć, że te usunięcia nie mają definicji wycofania, co odzwierciedla scenariusze, w których czyszczenie danych jest nieodwracalne z założenia (np. RODO „prawo do bycia zapomnianym").

Po zakończeniu wszystkich modyfikacji wykonywane jest polecenie `ANALYZE big_table`, które odświeża statystyki planisty, zapewniając, że kolejne operacje odczytu i planowania będą korzystać z aktualnych optymalizacji planu wykonania.

W środowisku produkcyjnym ten scenariusz mógłby być wykonywany w środowiskach przeznaczonych do testów wydajnościowych, optymalizacji planów zapytań lub etapowych przepływów ETL. Poprzez parametryzację wsadowych obciążeń i wzbogacających operacji, wraz z przygotowanymi skryptami rollbacku dla krytycznych kroków, operatorzy mogą bezpiecznie weryfikować charakterystykę obciążenia, testować strategie partycjonowania i indeksowania oraz ćwiczyć procedury odzyskiwania po awarii zanim analogiczne działania zostaną zastosowane na danych produkcyjnych. Schemat przedstawiający kroki tego scenariusza przedstawiono na rysunku 3.4.



Rysunek 3.4 Diagram - scenariusz 2.

3.3 Porównanie wybranych narzędzi

Aby opracować model oceny efektywności narzędzi wspomagających zarządzanie zmianami w bazach danych w kontekście CI/CD, konieczne jest przeprowadzenie analizy porównawczej tych narzędzi przy wykorzystaniu metryk wydajnościowych i funkcjonalnych. W niniejszym podrozdziale skoncentrowano się na charakterystyce oraz definicji wprowadzonych wskaźników oceny, stanowiących podstawę do dalszej analizy porównawczej.

3.3.1 Metryki

- **Czas wykonania migracji oraz cofania zmian**

  Powyższe kryterium polega na zmierzeniu, jak szybko narzędzie wykonuje operacje wprowadzenia nowych zmian w bazie danych oraz jak efektywnie potrafi przywrócić poprzedni stan. Odzwierciedla to wydajność algorytmów zastosowanych przez narzędzie oraz stopień automatyzacji procesów migracyjnych. W praktyce, krótszy czas wykonania tych operacji wskazuje na lepszą optymalizację i mniejsze obciążenie systemu, co jest kluczowe w środowiskach o wysokiej dynamice zmian.

- **Wskaźnik sukcesu migracji**

  Ten parametr będzie mierzył. jak często operacje migracyjne kończą się powodzeniem. Wskaźnik ten pokaże jak niezawodne jest zastosowane narzędzie, dodatkowo będzie widoczna zdolność do prawidłowego zastosowania zmian na bazie danych. Im wyższy wskaźnik sukcesu, tym narzędzie skuteczniej radzi sobie z różnorodnymi scenariuszami migracyjnymi.

- **Złożoność skryptów migracyjnych**

  Ocena złożoności odnosi się do struktury i przejrzystości plików migracyjnych. Należy ocenić, jak rozbudowane są skrypty oraz czy są one łatwe do zrozumienia i utrzymania przez zespół deweloperski. Mniejsza złożoność sprzyja szybszemu wdrożeniu i ułatwia diagnozowanie problemów, co jest szczególnie ważne w dynamicznych środowiskach CI/CD.

- **Zużycie zasobów**

  Dzięki tej metryce będzie możliwe ocenienie efektywności narzędzi do zarządzania zmianami w bazach danych pod kątem wykorzystania zasobów systemowych, takich jak procesor (CPU) i pamięć operacyjna (RAM), podczas operacji migracyjnych. Nadmierne zużycie zasobów może prowadzić do obniżenia wydajności systemu, zwłaszcza w środowiskach o ograniczonych zasobach lub przy równoczesnym wykonywaniu wielu procesów. Monitorowanie tych parametrów pozwala na identyfikację wąskich gardeł i optymalizację procesów migracyjnych. W praktyce, efektywne narzędzia powinny minimalizować obciążenie systemu, zapewniając płynne i szybkie przeprowadzanie migracji bez negatywnego wpływu na inne komponenty systemu.

- **Łatwość integracji z CI/CD**

  Ten aspekt oceny dotyczy tego, jak prosto narzędzie do migracji może być włączone w istniejący pipeline CI/CD. Narzędzie powinno posiadać mechanizmy automatycznej konfiguracji, wsparcie dla standardowych formatów konfiguracyjnych oraz interfejsy API umożliwiające łatwą integrację z popularnymi narzędziami CI/CD.

Łatwość integracji wpływa na szybkość wdrożenia narzędzia oraz na minimalizację konieczności dostosowywania istniejących procesów do nowego rozwiązania.

- **Subiektywna ocena narzędzi**

  W tej metryce narzędzia zostaną ocenione na podstawie osobistych doświadczeń zdobytych w trakcie implementacji. Omówiona zostanie intuicyjność narzędzi i interfejsu, dostępność kompletnej dokumentacji oraz wsparcia technicznego.

3.3.2 Flyway i Liquibase

W tym podrozdziale skupiono się na szczegółowym opisie i porównaniu narzędzi Flyway i Liquibase, które służą do zarządzania zmianami w bazach danych. Narzędzia zostały wstępnie opisane w podrozdziale 2.3 wraz z innymi dostępnymi narzędziami, a w tym podrozdziale zostaną porównane ze sobą.

Pliki migracyjne

Flyway to narzędzie służące do kontrolowanej, wersjonowanej migracji schematów baz danych, oparte na tzw. podejściu wersjonowanych migracji (`versioned migrations approach`), w którym każda zmiana reprezentowana jest przez odpowiednio ponumerowany skrypt w języku SQL lub Java. Narzędzie to jest powszechnie klasyfikowane jako lekkie rozwiązanie migracyjne, szczególnie z uwagi na preferencję dla skryptów SQL, co przekłada się na prostotę użytkowania i minimalny próg wejścia. Kluczową rolę w działaniu Flyway odgrywa konwencja nazewnictwa plików migracyjnych [71], która zapewnia spójną i uporządkowaną kolejność wykonywanych migracji. Każdy plik migracyjny musi być nazywany zgodnie z poniższym schematem ze schematem przedstawionym na opisie 3.6.

```
<Prefix><Version>__<Description>.<ext>
```

Listing 3.6 Konwencja nazwy plików Flyway

Gdzie:

- `<Prefix>` - litera określająca typ migracji.
- `<Version>` - wersja migracji, która może przyjmować postać liczby naturalnej (np. `1`), liczby zmiennoprzecinkowej (np. `1.2`) lub znacznika czasowego (np. `20250522`).
- `<Description>` — tekstowy opis migracji o charakterze informacyjnym,
- `<ext>` — rozszerzenie pliku: `sql` lub `java`.

Domyślnie Flyway oczekuje skryptów migracyjnych w folderze `src/main/resources/db/migration/`.

Skrypty wersjonowane (*versioned migration*) [61] są wykonywane w docelowej bazie danych dokładnie raz. Każdy z nich jest identyfikowany na podstawie unikalnego numeru wersji oraz opisywany przez system w tabeli systemowej `flyway_schema_history` [62] która rejestruje nie tylko zastosowane wersje, ale również sumy kontrolne plików w celu detekcji zmian. Zaleceniem i dobrą praktyką jest niedokonywanie zmian w już zastosowanej migracji. Zamiast tego najlepiej lepiej utworzyć nowy wersjonowany skrypt z wyższym numerem wersji i tam zawrzeć wszelkie niezbędne zmiany. Migracje wersjonowane są zwykle używane do takich operacji jak tworzenie, modyfikacja i usuwanie tabel, aktualizacja

danych referencyjnych oraz korekt danych. Każdy plik migracji wersjonowanej zaczyna się od prefiksu `V` i może wyglądać w następujący sposób, np.: `V1__Description.sql`. Migracje stosowane są w sekwencyjnie w kolejności rosnącej, tak jak można by się tego spodziewać.

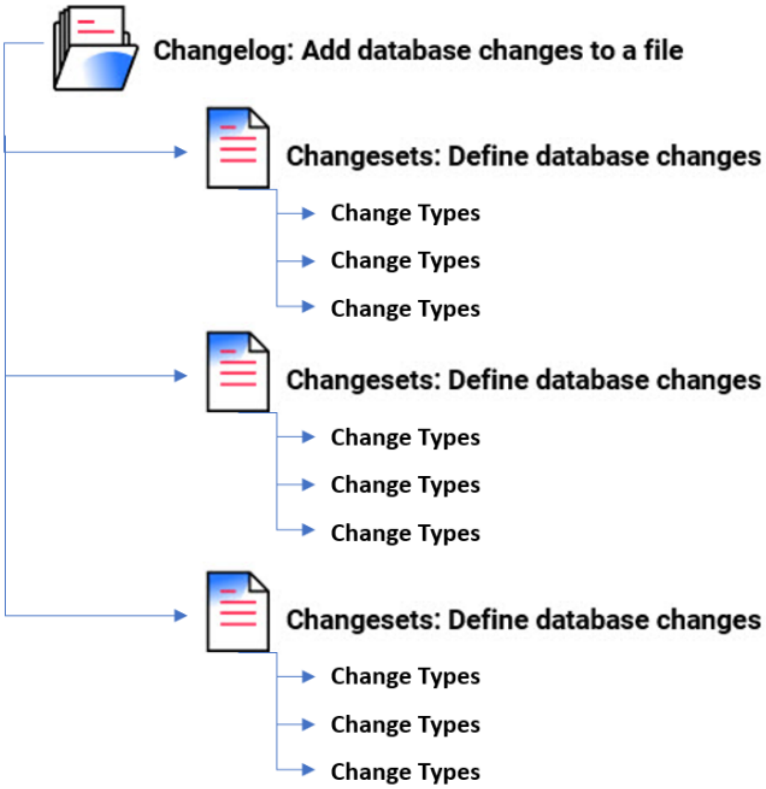
Migracje cofające (*undo migrations*) [60] są przeciwieństwem skryptów wersjonowanych i umożliwiają wycofanie zmian wprowadzonych przez skrypt wersjonowany o tej samej wersji. Są opcjonalne i niewymagane do wykonanaia zwyczajnej wersjonowanej migracji. Każdy taki plik zaczyna się od prefiksu `U` i mógłby wyglądać następująco, np.: `U1.__UndoDescription.sql`. Należy tutaj wspomnieć, że cofanie zmian za pomocą plików undo jest dostępne w Flyway Teams/Pro Edition, więc aby zastosować je w wersji oper-source należy wywoływać odpowiednie skrypty SQL, które mogą być nazwane wg tej samej konwencji.

Trzecią kategorią są migracje powtarzalne (*repeatable migrations*) [59], które są uruchamiane za każdym razem kiedy suma kontrolna bazy danych się zmieni. Stanowią one wygodny mechanizm do utrzymywania obiektów takich jak widoki, funkcje, procedury składowane, indeksy czy dane konfiguracyjne w jednym, kontrolowanym pliku. Migracje powtarzalne są zawsze wykonywane po zakończeniu wszystkich oczekujących migracji wersjonowanych w ramach pojedynczego cyklu migracji. Użytkownik musi upewnić się, że ta sama powtarzalna migracja może być zastosowana wiele razy. Nazwa pliku migracji powtarzalnych zaczyna się od prefiksu `R` bez następującej wersji i mogłaby wyglądać tak, np.: `R__SetDefaults.sql`.

Liquibase to narzędzie do zarządzania zmianami w bazach danych, które umożliwia szybsze i bezpieczniejsze wprowadzanie zmian schematu oraz danych. W przypadku tego narzędzia używany jest tekstowy plik dziennika zmian (*changelog*) [36], który wyświetla sekwencyjnie wszystkie zmiany, które zostaną wykonane na bazie danych. Domyślnie plik dziennika zmian znajduje się w folderze `src/main/resources/db/changelog/db.changelog-master.xml`.

Każdy plik dziennika składa się z jednej lub więcej jednostek zmian (*changeset*), w których zdefiniowane są konkretne operacje, takie jak `createTable`, `addColumn`, `dropPrimaryKey`, `sql` czy `sqlFile`. Uproszczony schemat struktury dziennika zmian przedstawiono na rysunku 3.5.

Podczas uruchomienia Liquibase porównuje zawartość pliku dziennika zmian z tabelą systemową `databasechangelog` i aplikuje tylko te zmiany, które nie zostały jeszcze zarejestrowane. Cały dziennik zmian składa się z jednej lub więcej jednostek zmian (*changeset*), w których zdefiniowane są konkretne operacje, takie jak `createTable`, `addColumn`, `dropPrimaryKey`, `sql` czy `sqlFile`. Uproszczony schemat struktury dziennika zmian przedstawiono na rysunku 3.5.



Rysunek 3.5 Uproszczony schemat dziennika zmian (*changelog*) [36]

Liquibase wspiera cztery formaty dziennika zmian, tj. SQL, XML, YAML, JSON, które są rozpoznawane automatycznie po rozszerzeniu pliku albo nagłówkowi dziennika zmian. Format realizowany jest według dwóch modeli:

- **Model SQL** (`.sql`): instrukcje SQL są wpisywane bezpośrednio w zestawy zmian, a plik jest formatowany przy użyciu komentarzy SQL.
- **Model niezależny od platformy** (`.xml`, `.yaml`, `.json`): każda operacja jest reprezentowana przez odpowiedni typ zmiany Liquibase, z którego narzędzie generuje odpowiadające im operacje SQL dla docelowej bazy danych. Dodatkowo, w tych formatach można osadzić nieprzetworzone polecenia SQL za pomocą typu zmiany `sql` lub odwołać się do zewnętrznego pliku SQL używając typu `sqlFile`.

Każdy zestaw zmian może zawierać wewnętrzną sekcję `<rollback>`, określającą sposób cofnięcia wprowadzonych operacji. W przypadku jej braku Liquibase stara się automatycznie wygenerować skrypt rollbacku dla prostych zmian (np. `createTable` → `dropTable`), jednak w przypadku bardziej złożonych operacji lub poleceń typu `dropTable`, należy samodzielnie zdefiniować element `<rollback>`. Przykładowy zestaw zmian przedstawiono poniżej w wykazie 3.7

```
<changeSet id="1-add-new-name" author="dev" context="1" runAlways="true">
    <preConditions onFail="MARK_RAN">
        <not><columnExists tableName="person" columnName="new_name"/></not>
    </preConditions>
    <addColumn tableName="person">
        <column name="new_name" type="text"/>
    </addColumn>
    <rollback>
        <dropColumn tableName="person" columnName="new_name"/>
    </rollback>
</changeSet>
```

Listing 3.7 Przykładowy zestaw zmian (*changeset*) w fomacie XML.

Wspierane bazy danych i platformy

W tabeli 3.2 przedstawiono zestawienie wszystkich wspieranych platform i baz danych przez narzędzia Flyway i Liquibase.

Tabele tworzone podczas migracji

Obydwa narzędzia podczas wykonywania migracji tworzą specjalne tabele w bazie danych, aby śledzić stan migracji i zapewnić spójność oraz kontrolę nad zmianami schematu. Utworzone tabele pełnią dzienników, umożliwiając narzędziom określenie, które migracje zostały już wykonane, a które są jeszcze do wykonania.

Flyway podczas pracy tworzy tabelę `flyway_schema_history` [62]. Dzięki temu śledzi, które migracje zostały zastosowane, kiedy i przez kogo. Śledzi ona także sumy kontrolne migracji oraz to, czy migracje zakończyły się powodzeniem. Nazwę tabeli można dostosować za pomocą opcji `table`. Schemat wg którego ma zostać utworzona tabela, można skonfigurować za pomocą `defaultSchema`, a przestrzeń w której ma zostać utworzona tabela historii zmian może zostać skonfigurowana za pomoca opcji `tablespace`. W tabeli 3.1 przedstawiono atrybuty domyślnie utworzonej tabeli `flyway_schema_history`.

Nazwa atrybutu	Typ	Opis
installed_rank	integer	Licznik zastosowanych migracji.
version	varchar(50)	Wersja migracji (z nazwy pliku skryptu).
description	varchar(50)	Opis migracji (wyciągnięty z nazwy skryptu)
type	varchar(20)	Sposób przeprowadzenia migracji (SQL lub Java)
script	varchar(1000)	Nazwa pliku lub klasy.
checksum	integer	Suma kontrolna migracji.
installed_by	varchar(100)	Użytkowanik bazy danych, który uruchomił migrację.
installed_on	timestamp	Czas ukończenia migracji bez strefy czasowej.
execution_time	integer	Czas trwania migracji [ms]
success	boolean	Wartość `true` lub `false`.

Tabela 3.1 Zawartość tabeli `flyway_schema_history` [64].

Baza Danych/Platofrma	Liquibase	Flyway
Amazon Aurora - MySQL	tak	tak
Amazon Aurora - PostgreSQL	tak	tak
Amazon DocumentDB	tak	tak
Amazon DynamoDB	tak	nie
Amazon RDS - MariaDB	tak	tak
Amazon RDS - MySQL	tak	tak
Amazon RDS - Oracle	tak	tak
Amazon RDS - PostgreSQL	tak	tak
Amazon RDS - SQL Server (MSSQL)	tak	tak
Amazon Redshift	tak	tak
Apache Cassandra	tak	tak
Apache Derby	tak	tak
ClickHouse	tak	tak
CockroachDB	tak	tak
Databricks	tak	tak
EnterpriseDB	tak	tak
Firebird	tak	tak
Google BigQuery	tak	tak
Google Cloud - MySQL	tak	tak
Google Cloud - PostgreSQL	tak	tak
Google Cloud - SQL Server (MSSQL)	tak	tak
Google Cloud - Spanner	tak	tak
Greenplum Database	tak	tak
H2	tak	tak
HarperDB	tak	nie
HyperSQL Database (HSQLDB)	tak	tak
IBM DB2 LUW	tak	tak
IBM DB2 for i	tak	tak
IBM Db2 for Z	tak	tak
IBM Informix	tak	tak
Ingres	tak	nie
InterSystems Cache	tak	tak
MariaDB	tak	tak
MariaDB SkySQL	tak	tak
Microsoft Azure Cosmos DB	tak	nie
Microsoft Azure Database - MySQL	tak	tak
Microsoft Azure Database - PostgreSQL Flexible Server	tak	tak
Microsoft Azure Database - PostgreSQL Single Server	tak	tak
Microsoft Azure SQL Database	tak	tak
Microsoft Azure SQL Managed Instance	tak	tak
Microsoft SQL Server (MSSQL)	tak	tak
MongoDB	tak	tak
MongoDB Atlas	tak	tak
MySQL	tak	tak
Neo4j Graph Database	tak	nie
Oracle Autonomous Database	tak	tak
Oracle Database	tak	tak
Percona Server - MySQL	tak	tak
Perocna XtraDB Cluster	tak	tak
PostgreSQL	tak	tak
SAP HANA	tak	tak
SAP MaxDB	tak	nie
SAP SQL Anywhere	tak	nie
SQLite	tak	tak
SingleStoreDB	tak	tak
Snowflake	tak	tak
Sybase (Enterprise/ASE - now SAP ASE)	tak	tak
Teradata	tak	nie
VMware vFabric SQLFire	tak	nie
Vertica	tak	tak
YugabyteDB	tak	tak
Oracle Exadata Database Service	tak	tak
TimescaleDB	tak	tak
Azure Synapse	nie	tak
Ignite	tak	tak
OceanBase	tak	tak
TiDB (Titanium DB)	tak	tak
TestContainers	tak	tak

Tabela 3.2 Zestawienie obsługiwanych baz danych i platform [45] [63].

Liquibase podczas wykonywania migracji tworzy dwie tabele, `databasechangelog` [37] oraz `databasechangelock` [38]. Pierwsza rejestruje każdy zestaw zmian, który został wykonany, umożliwiając śledzenie historii modyfikacji schematu. Druga służy do zarządzania mechanizmem blokady, gwarantując, że w danym momencie tylko jeden proces Liquibase wykonuje operacje migracyjne na danej bazie danych. Nazwy obydwu tabel można zmienić za pomocą argumentów odpowiednio `--database-changelog-table-name` oraz `--database-changelog-lock-table-name`. W tabelach 3.3 oraz 3.4 przedstawiono atrybuty tych tabel.

Nazwa atrybutu	Typ	Opis
id	varchar(255)	Wartość z atrybutu id zestawu zmian.
author	varchar(255)	Wartość z atrybutu author zestawu zmian.
filename	varchar(255)	Ścieżka do pliku dziennika zmian.
dateexecuted	timestamp	Data/godzina wykonania zestawu zmian bez strefy czasowej.
orderexecuted	integer	Kolejność wykonania zestawów zmian.
exectype	varchar(10)	Opis sposobu wykonania zestawu zmian. Możliwe wartości to EXECUTED, FAILED, SKIPPED, RERAN i MARK_RAN.
md5sum	varchar(35)	Suma kontrolna zestawu zmian w momencie jego wykonania.
description	varchar(255)	Krótki, automatycznie wygenerowany, czytelny dla człowieka opis zestawu zmian.
comments	varchar(255)	Wartość z atrybutu comments zestawu zmian.
tag	varchar(255)	Śledzi, które zestawy zmian odpowiadają operacjom na znacznikach.
liquibase	varchar(20)	Wersja Liquibase użyta do wykonania zestawu zmian.
contexts	varchar(255)	Konteksty użyte do wykonania zestawu zmian.
labels	varchar(255)	Etykiety używane do wykonania zestawu zmian.
deployment_id	varchar(10)	Unikalny identyfikator wszystkich wdrożonych za pomocą tej samej operacji zestawów zmian.

Tabela 3.3 Zawartość tabeli `databasechangelog` [37].

Nazwa atrybutu	Typ	Opis
id	integer	Identyfikator blokady.
locked	integer	Ustawione na 1 jeżeli Liquibase działa na bazie danych, w innym przypadku 0.
lockgranted	timestamp	Data i godzina przyznania blokady bez uwzględniania strefy czasowej.
lockedby	varchar(255)	Czytelny dla człowieka opis tego, komu blokada została przyznana.

Tabela 3.4 Zawartośc tabeli `databasechangeloglock` [38].

Licencje

Narzędzia Flyway oraz Liquibase oferowane są zarówno w bezpłatnych, jak i komercyjnych edycjach, dostosowanych do różnorodnych potrzeb zespołów deweloperskich i organizacji.

Flyway występuje w trzech głównych wariantach: Community Edition, Teams Edition oraz Enterprise Edition. Community Edition[67] rozpowszechniana jest na licencji Apache 2.0 i umożliwia nieograniczone, bezpłatne użytkowanie, również w środowiskach komercyjnych. Zapewnia ona podstawowy zestaw poleceń migracyjnych, takich jak `migrate`, `validate`, `repair`, `baseline`, `info` oraz `clean`. Są one dostępne zarówno z poziomu interfejsu CLI, jak i poprzez Java API, Maven czy Gradle. Wersja Teams, licencjonowana na użytkownika, rozszerza funkcjonalność o skrypty cofające (*undo scripts*), generowanie szczegółowych raportów SQL, a także graficzny interfejs użytkownika prezentujący czasy wykonania, listę oczekujących skryptów oraz ich zawartość. Ponadto Teams Edition gwarantuje dziesięcioletnie wsparcie nowych wersji baz danych oraz pomoc techniczną Redgate[65]. Dla organizacji i firm o najbardziej rozbudowanych wymaganiach Flyway Enterprise[68] udostępnia mechanizmy, takie jak funkcja `Generate` umożliwiająca automatyczne tworzenie różnorodnych skryptów migracyjnych, czy moduł `Check` wykrywający potencjalne błędy przed wprowadzeniem zmian do produkcji. Dodatkowo Enterprise Edition oferuje rozbudowane raporty detekcji dryfu środowiskowego oraz pulpity nawigacyjne, zapewniające pełną widoczność stanu migracji dla całego zespołu. Zgodnie z danymi rynkowymi[19], Community Edition jest dystrybuowana bezpłatnie, Teams Edition kosztuje około 600 USD rocznie, natomiast wycena Enterprise Edition uzależniona jest od indywidualnych ustaleń z producentem.

Liquibase, podobnie jak Flyway, udostępnia wersję open-source na licencji Apache 2.0, określaną jako Liquibase Community Edition [41]. Wersja Community oferuje bezpłatny dostęp do silnika migracji opartego na dzienniku zmian, w tym możliwość wycofywania wprowadzonych zmian według tagu, daty lub numeru zestawu zmian, a także automatyczne generowanie dziennika zmian za pomocą polecenia `diffChangeLog`. Wszystkie funkcje są dostępne zarówno z poziomu interfejsu CLI, jak i w integracjach z Maven, Gradle, Spring Boot oraz narzędziami typu GitHub Actions, Jenkins, czy GitLab CI/CD. Liquibase Teams (Pro) Edition [42] rozszerza funkcjonalność wersji Community poprzez automatyzację „inteligentnych" aktualizacji, które identyfikują i stosują wyłącznie niezastosowane zestawy zmian, a także umożliwiają ukierunkowane wykonywanie operacji, podgląd suchych przebiegów oraz zaawansowane scenariusze wycofywania zmian, w tym selektywne cofanie wyzwalane błędem. Wersja Teams oferuje ponadto natywne executory dla SQLPlus, PSQL, SQLCMD i MongoSH oraz narzędzia zarządzania migracjami, takie jak automatyczne wykrywanie różnic, warunki wstępne przed wdrożeniem, definiowanie przepływów pracy (workflows), szczegółowe rejestrowanie operacji i raporty eksploatacyjne. Integracja z systemami zarządzania sekretami (AWS Secrets Manager, HashiCorp Vault) oraz dystrybucja przez CLI, Docker, Maven/Gradle/Ant lub jako biblioteka, a także opcjonalne (na podstawie umoqy SLA) wsparcie 24×7, onboarding, szkolenia i dedykowany opiekun techniczny. Liquibase Enterprise Edition [43] jest dedykowana środowiskom o wysokiej krytyczności i dużym natężeniu transakcji. Wspierając wszystkie główne platformy RDBMS dla przedsiębiorstw (Oracle, SQL Server, IBM DB2, EnterpriseDB i PostgreSQL) Enterprise Edition egzekwuje reguły pre-deployment oraz kontrole zgodności. Patentowany silnik predykcyjny umożliwia symulację i weryfikację zmian schematu w docelowej bazie danych, eliminując ryzyko błędów ręcznych i kosztownego cofania zmian, zaś zespół może wstępnie zatwierdzić wdrożenia i wygenerować automatyczne skrypty cofania przed zastosowaniem jakiejkolwiek modyfikacji. Połączenie analizy

predykcyjnej, integracji CI/CD jednym kliknięciem oraz wbudowanej gotowości do cofnięcia migracji przyspiesza i ubezpiecza proces wydań baz danych, pozwalając skupić się na rozwoju aplikacji. Zgodnie z informacjami [42], Liquibase Community Edition jest dostępna bezpłatnie, Liquibase Teams (Pro) Edition kosztuje 500 USD rocznie, natomiast cena Liquibase Enterprise Edition ustalana jest indywidualnie.

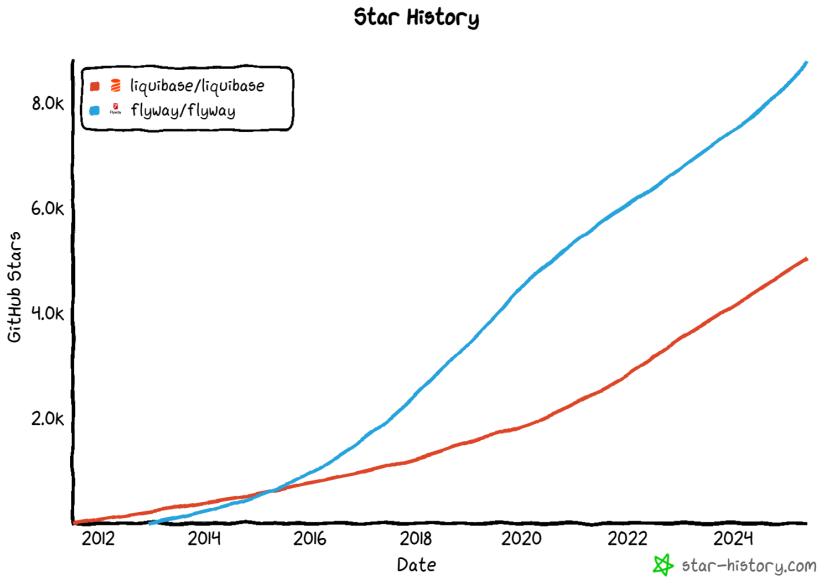
Społeczność

Wsparcie społeczności odgrywa kluczową rolę w rozwoju i utrzymaniu narzędzi, nie tylko takich jak Flyway i Liquibase, gdyż umożliwia szybkie rozwiązywanie problemów przez wymianę doświadczeń, zgłaszanie defektów oraz proponowanie usprawnień. Dzięki aktywnej współpracy użytkowników narzędzia zyskują na niezawodności, kompletności dokumentacji oraz lepszym dopasowaniu do rzeczywistych potrzeb.

Społeczność Flyway skoncentrowana jest przede wszystkim wokół platformy GitHub [57]. W maju 2025 roku obejmuje ona siedem publicznych repozytoriów, z których główne (flyway/flyway) zgromadziło około 8 800 gwiazdek, 173 obserwujących oraz 1 500 rozgałęzień (*forks*). Nad zawartością repozytoriów czuwają inżynierowie Redgate oraz rosnąca grupa zewnętrznych współtwórców, którzy zgłaszają problemy oraz utrzymują społecznościowe wtyczki. Poza GitHubem, użytkownicy Flyway aktywnie wymieniają się wiedzą na Stack Overflow pod tagiem „flyway"[49], a Redgate uzupełnia ofertę edukacyjną o materiały szkoleniowe dostępne na platformie Redgate University[58].

Podobnie społeczność Liquibase rozwija się na GitHubie [39], gdzie główne repozytorium zgromadziło w maju 2025 roku około 5 000 gwiazdek, 140 obserwujących oraz 1900 forków. Użytkownicy Liquibase dzielą się wiedzą i rozwiązaniami zarówno na Stack Overflow [50], jak i na dedykowanym forum [8], a bezpłatne kursy udostępniane są w ramach Liquibase University [44]. Na rysunku 3.6 przedstawiono porównanie popularności obu narzędzi w oparciu o liczbę gwiazdek na GitHubie.

Na rysunku 3.6 przedstawiono wykres popularności narzędzi wg gwiazdek na platformie Github.



Rysunek 3.6 Wykres popularności Flyway i Liquibase wg ilości gwiazdek na platformie Github [29].

Integracja z CI/CD

W środowiskach DevOps zarówno Flyway, jak i Liquibase można łatwo włączyć do potoków CI/CD, dzięki czemu migracje bazy danych są uruchamiane razem z kodem aplikacji na każdym etapie wdrożenia.

W przypadku Flyway wystarczy zainstalować CLI albo użyć oficjalnego obrazu Dockera na agentach budowania, a potem w skryptach potoku wywoływać polecenia takie jak `migrate`, `validate` czy `repair`. Redgate udostępnia krok po kroku instrukcje konfiguracji Flyway w CI/CD [56], przykładowe definicje potoków [54], a także gotową akcję na GitHub Actions, która pakuje te polecenia w wielokrotnego użytku workflow i ułatwia integrację z repozytoriami [55]. Dzięki temu użycie Flyway w dowolnym systemie CI wygląda bardzo podobnie i nie wymaga dużych zmian w konfiguracji.

Liquibase działa nieco inaczej, ponieważ w zmianach bazy danych trzyma spis zmian (`changelog`) w formatach SQL, YAML, JSON lub XML, który jest wersjonowany razem z kodem. Gdy pipeline CI/CD zaczyna pracę, Liquibase porównuje aktualny stan changelogu z tabelą `databasechangelog`, wykonuje tylko brakujące zmiany, a po każdym uruchomieniu zbiera dane o przebiegu migracji, metryki i komunikaty błędów. Dzięki wbudowanym raportom można na bieżąco sprawdzać, gdzie migracja utknęła lub ile czasu trwała [35]. Podobnie jak Flyway, Liquibase oferuje gotową akcję na GitHub Actions, która pozwala uruchamiać migracje w ramach potoków bez dodatkowego kodowania [17].

Rozdział 4

Wyniki badań

W tym rozdziale zostaną przedstawione wyniki migracji przedstawionych w podrozdziale 3.2. Testy zostały przeprowadzone w środowisku lokalnym oraz w środowisku Github Actions. W środowisku lokalnym były mierzone wszystkie metryki, poza wskaźnikiem sukcesu migracji, które były wspomniane w podrozdziale 3.3.1. W środowisku CI/CD nie były mierzone zużycie procesora i zużycie pamięci, ponieważ GitHub Actions nie udostępnia natywnych mechanizmów monitorowania zasobów systemowych kontenerów wykonawczych, ale za to mierzony był wskaźnik sukcesu migracji. Z racji, że skrypty migracyjne były takie same dla środowiska lokalnego i środowiska CI/CD ich skomplikowanie zostanie przedstawione w osobnych podrozdziałach bez podziału na środowiska. Wszystkie przedstawione wartości zostały uzyskane na podstawie stu iteracyjnych przebiegów migracji oraz rollbacku dla obu narzędzi, przeprowadzonych zarówno w środowisku lokalnym, jak i w potokach CI/CD.

4.1 Scenariusz 1.

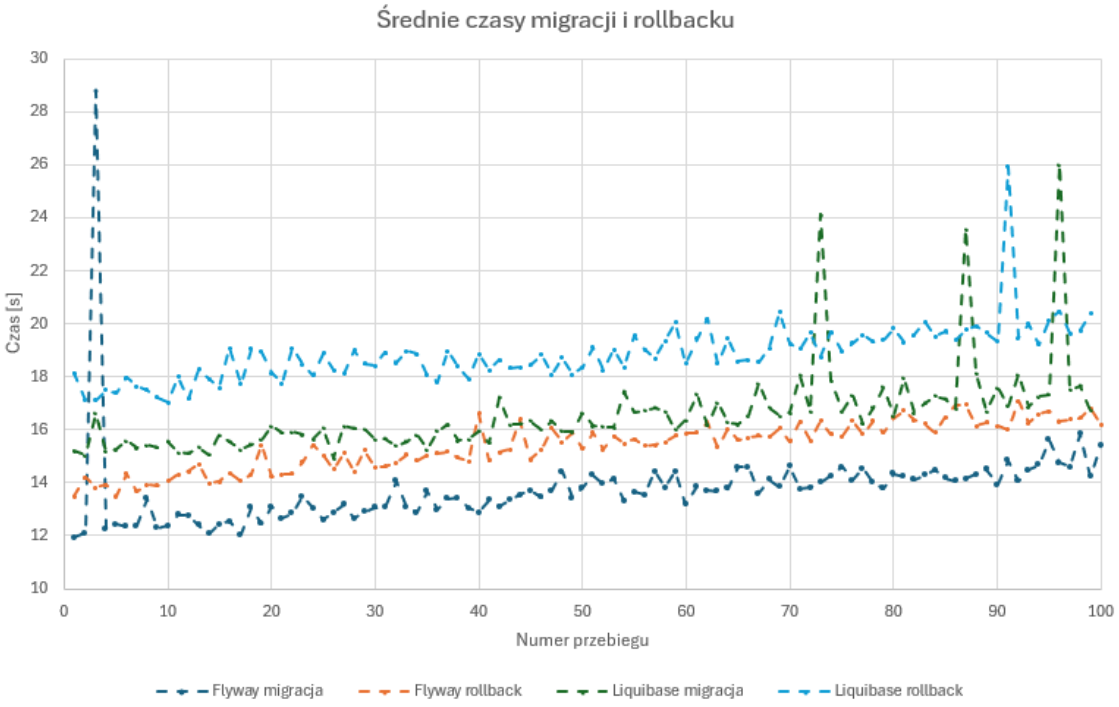
Opis scenariusza 1. został przedstawiony w podrozdziale 3.2.1.

4.1.1 Środowisko lokalne

Fragment tabeli wynikowej został przedstawiony w tabeli 4.1, natomiast opracowane wyniki ze scenariusza 1. zamieszczono w tabeli 4.2. Wartości oznaczone kolorem zielonym odpowiadają lepszym rezultatom, zaś kolorem czerwonym gorszym. Z analizy danych wynika, że Flyway uzyskał lepsze wyniki niż Liquibase w badanych metrykach.

Suma średnich czasów migracji poszczególnych kontekstów, stanowiąca średni czas migracji całego scenariusza, dla Flyway wyniosła $13,743s$, natomiast dla Liquibase $16,399s$, co oznacza przewagę Flyway na poziomie $16,20\%$. Średni czas cofania zmian (*rollback*) był również niższy dla Flyway ($15,392s$) względem Liquibase ($18,673s$), co przekłada się na przyspieszenie o $17,57\%$. Różnice pomiędzy czasami rollbacku i migracji wynosiły odpowiednio $1,648s$ (Flyway) i $2,274s$ (Liquibase). Zestawienie czasów migracji i cofania zmian dla każdego przebiegu testowego przedstawiono na wykresie 4.1. Piki widoczne na wykresie mogą być spowodowane chwilowym ograniczeniem I/O dysku (np. gdy w tle działa antywirus lub inne zadania odczytują/zapisują dane).

Jeśli chodzi o średnie obciążenie procesora podczas migracji, narzędzie Flyway zużywało średnio $1,021\%$, a Liquibase $1,071\%$ CPU. W trakcie rollbacku wartości te wyniosły odpowiednio $0,985\%$ (Flyway) i $0,994\%$ (Liquibase). Różnica obciążenia CPU między



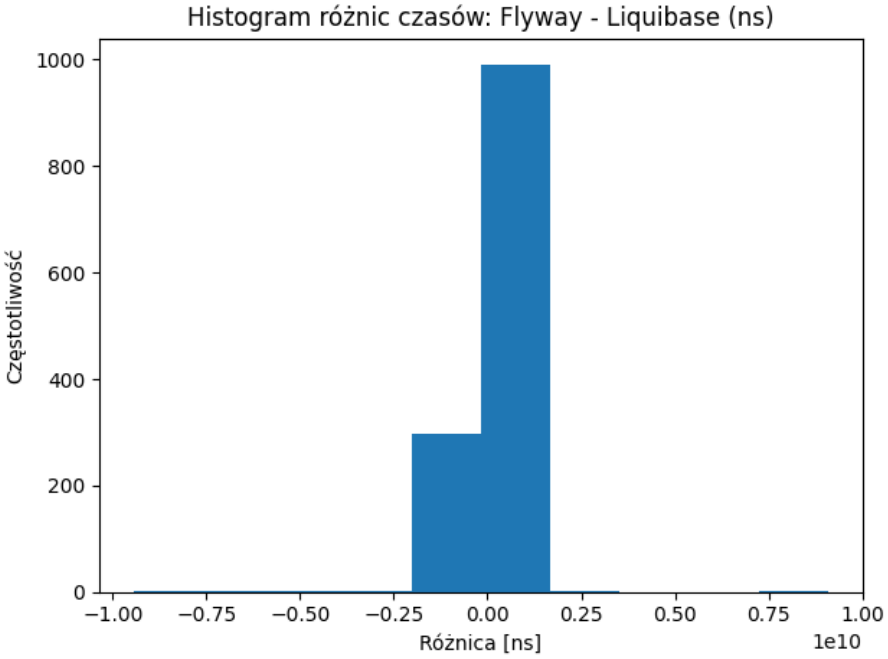
Rysunek 4.1 Zestawienie średnich czasów migracji i cofania zmian dla każdego z przebiegów.

narzędziami w czasie migracji wynosiła$0,049\%$, natomiast w czasie rollbacku $0,009\%$. Natomiast różnica wykorzystania CPU pomiędzy migracją i rollbackiem wynosiła $0,036\%$ dla Flyway oraz $0,077\%$ dla Liquibase.

W kontekście zużycia pamięci w trakcie migracji Flyway zużywał średnio $50,845MB$, a podczas rollbacku $50,866MB$ (różnica $0,021MB$). Liquibase w analogicznym scenariuszu zużywał odpowiednio $52,197MB$ oraz $52,476MB$ (różnica $0,279MB$). Flyway zużywał o około $2,59\%$ mniej pamięci niż Liquibase podczas migracji oraz o około $3,07\%$ mniej podczas rollbacku.

Analizę statystyczną przeprowadzono tylko dla czasów migracji, ponieważ funkcja migracji była udostępniona przez obydwa narzędzia. W ramach przeprowadzonej analizy statystycznej wykorzystano środowisko Python oraz biblioteki `pandas`[51] oraz `scipy`[72]. Biblioteka `pandas` posłużyła do wczytania, agregacji i wstępnej obróbki danych (grupowanie po kontekście, obliczanie średnich wartości czasów migracji, itp.). Z kolei z `scipy.stats` zastosowano funkcję `shapiro`[10], jest to test Shapiro–Wilka użyty do weryfikacji normalności rozkładu różnic czasów. Po weryfikacji skorzystano z funkcji `ttest_rel`[11] oraz `wilcoxon`[12]. Pierwsza z nich służy do przeprowadzenia dwustronnego testu t dla prób zależnych, w celu oceny istotności statystycznej różnic pomiędzy czasami migracji w ramach tych samych kontekstów dla dwóch narzędzi. Druga funkcja to test rang par Wilcoxona, który został zastosowany jako nieparametryczna alternatywa dla testu t, w sytuacji gdy rozkład różnic nie spełnia założeń normalności. Test został wykonany dla prób zależnych, ponieważ dla każdego kontekstu i powtórzenia porównywane są czasy migracji uzyskane przy użyciu dwóch narzędzi. Oznacza to, że każda para obserwacji jest logicznie powiązana i pochodzi z tych samych warunków eksperymentalnych, co uzasadnia zastosowanie testu prób zależnych.

- Test Shapiro–Wilka
  W przeprowadzonym teście Shapiro–Wilka wyliczono statystykę W, która jest miarą stopnia, w jakim badany wektor obserwacji zgadza się z rozkładem normalnym. Była ona równa $W = 0.4672$. Wartość W mieści się w przedziale $(0, 1]$. Im bliżej jedności, tym lepiej dane pasują do rozkładu normalnego. W tym przypadku wartość ta wskazuje na istotne odchylenie od kształtu normalnego, co potwierdza wykres różnic pokazany na rysunku 4.2.



Rysunek 4.2 Histogram różnic [ns] scenariusza 1. w środowisku lokalnym.

Drugim wyliczonym elementem jest wartość p, która wyraża prawdopodobieństwo uzyskania statystyki W co najmniej tak ekstremalnej jak zaobserwowana, zakładając, że próbka pochodzi z rozkładu normalnego. Tutaj wartość była równa $p = 4.087 \cdot 10^{-52}$. Taka wartość p oznacza, że przy założeniu normalności nieprawdopodobne byłoby uzyskanie tak małej (odległej od 1) wartości W. W konsekwencji odrzucamy hipotezę o normalności rozkładu różnic na rzecz hipotezy alternatywnej, że rozkład różnic istotnie odbiega od normalnego.

- Test t dla par zależnych
  W tym teście obliczana jest statystyka t oraz wartość p, jednakże z testu Shapiro–Wilka wynika, że rozkład różnic nie jest normalny, a więc nie spełnia wymagań testu t.

- Test Wilcoxona
  W teście Wilcoxona dla par zależnych wyliczono statystykę testową *stat*, która jest sumą rang przypisanych do tych różnic, które mają określony znak (np. sumą rang różnic dodatnich lub ujemnych). Tutaj $stat = 109236$, co oznacza, że po usunięciu zerowych różnic i przypisaniu rang według wielkości bezwzględnej, suma rang różnic tej samej orientacji wynosi około 109236. Do tego obliczono wartość p, która wyraża prawdopodobieństwo uzyskania statystyki co najmniej tak ekstremalnej jak *stat*, zakładając, że mediana rozkładu różnic jest równa zero. Tu $p = 1.29 \cdot 10^{-117}$. Z racji,

że wartość p jest znacznie mniejsza niż 0,05, istnieje dowód, że mediana różnic jest różna od zera, czyli czasy migracji dla obu narzędzi istotnie się różnią.

tool	identifier	context	migration_ns	rollback_ns	exit_code	script_lines	cpu_pct	mem_bytes
Flyway	1,fwd,1	1	59734500	0	0	2	0,261531865	44026096
Flyway	1,fwd,2	2	4365862300	0	0	2	0,800052017	47723048
Flyway	1,fwd,3	3	46460100	0	0	18	0,772049296	50962136
Flyway	1,fwd,4	4	49115500	0	0	5	0,772049296	53569784
Flyway	1,fwd,5	5	46823500	0	0	2	0,951411404	56769584
Flyway	1,fwd,6	6	40055100	0	0	2	0,951411404	59350744
Flyway	1,fwd,7	7	4199055700	0	0	4	0,887212024	61970013
Flyway	1,fwd,8	8	32662900	0	0	16	1,028325582	37439592
Flyway	1,fwd,9	9	31681700	0	0	4	0,237269767	40628864
Flyway	1,fwd,10	10	474494500	0	0	1	0,387838108	43171392
Flyway	1,fwd,11	11	45146600	0	0	8	0,839543132	46300048
Flyway	1,fwd,12	12	30748200	0	0	21	0,471417298	49319408
Flyway	1,fwd,13	13	2511127400	0	0	2	0,667336023	51842257
Flyway	1,rbk,13	13	0	10494400	0	1	0,249931155	51847384
Flyway	1,rbk,12	12	0	7790800	0	2	0,249931155	52281336
Flyway	1,rbk,11	11	0	10827900	0	1	0,703777092	52346136
Flyway	1,rbk,10	10	0	13529000	0	1	0,703777092	52660400
Flyway	1,rbk,9	9	0	9501000	0	1	0,703777092	53070120
Flyway	1,rbk,8	8	0	8348100	0	1	1,710333222	53216376
Flyway	1,rbk,7	7	0	4087448800	0	3	0,817162533	53342055
Flyway	1,rbk,6	6	0	6940700	0	22	1,004299882	53346984
Flyway	1,rbk,5	5	0	5132858700	0	4	0,899340815	53468796
Flyway	1,rbk,4	4	0	6717600	0	1	0,973580375	53822488
Flyway	1,rbk,3	3	0	1339900	0	1	0,973580375	53883104
Flyway	1,rbk,2	2	0	4158183200	0	2	0,972177242	54005323
Flyway	1,rbk,1	1	0	10718700	0	2	1,101975587	54405568
Liquibase	1,fwd,1	1	95896100	0	0	9	1,402004818	60019696
Liquibase	1,fwd,2	2	5138709600	0	0	4	1,032855972	45336426
Liquibase	1,fwd,3	3	84816500	0	0	26	1,053096272	51231480
Liquibase	1,fwd,4	4	44877500	0	0	7	1,053096272	60278296
Liquibase	1,fwd,5	5	53921500	0	0	8	0,944983342	43415208
Liquibase	1,fwd,6	6	59978300	0	0	46	0,944983342	52141664
Liquibase	1,fwd,7	7	5287483700	0	0	19	1,053199399	62312350
Liquibase	1,fwd,8	8	77474300	0	0	26	1,2233808	42988640
Liquibase	1,fwd,9	9	57822700	0	0	9	1,212540078	52356600
Liquibase	1,fwd,10	10	603673900	0	0	9	1,022932331	62000217
Liquibase	1,fwd,11	11	87289900	0	0	16	1,563199778	43806736
Liquibase	1,fwd,12	12	52697400	0	0	31	1,563199778	54012000
Liquibase	1,fwd,13	13	3564925900	0	0	10	0,891545587	40538358
Liquibase	1,rbk,13	13	0	106130500	0	10	1,126926305	45324712
Liquibase	1,rbk,12	12	0	57222100	0	31	1,126926305	53832272
Liquibase	1,rbk,11	11	0	64650900	0	16	0,872870016	60681896
Liquibase	1,rbk,10	10	0	79161900	0	9	0,872870016	42682088
Liquibase	1,rbk,9	9	0	75753200	0	9	1,506051055	50412904
Liquibase	1,rbk,8	8	0	73298800	0	26	1,506051055	57527400
Liquibase	1,rbk,7	7	0	5382854100	0	19	1,08729953	41012293
Liquibase	1,rbk,6	6	0	53096400	0	46	1,242896835	46682240
Liquibase	1,rbk,5	5	0	6591715300	0	8	0,939155552	56294063
Liquibase	1,rbk,4	4	0	77044700	0	7	1,471957537	61934360
Liquibase	1,rbk,3	3	0	52262300	0	26	1,471957537	44396088
Liquibase	1,rbk,2	2	0	5436104800	0	4	1,16519005	52615812
Liquibase	1,rbk,1	1	0	73127300	0	9	1,24283467	58366640

Tabela 4.1 Przykładowe wyniki scenariusza 1. na maszynie lokalnej

Migracja			
Kontekst	Flyway		
	Średni czas migracji [s]:	Średni procent zużycia procesora[%]:	Średnia zużycia pamięci[MB]:
1	0,080	1,072	51,000
2	4,771	0,952	50,868
3	0,065	1,022	50,573
4	0,120	1,075	50,610
5	0,045	1,101	51,142
6	0,041	1,011	51,134
7	4,857	0,958	50,931
8	0,057	0,999	50,822
9	0,052	1,098	50,884
10	0,533	0,971	50,815
11	0,071	1,039	51,143
12	0,045	1,062	50,671
13	3,005	0,918	50,386
Suma lub średnia	13,743	1,021	50,845
Kontekst	Liquibase		
	Średni czas migracji [s]:	Średni procent zużycia procesora[%]:	Średnia zużycia pamięci[MB]:
1	0,063	1,090	52,533
2	5,594	0,973	52,087
3	0,070	1,023	52,887
4	0,133	1,097	52,570
5	0,067	1,138	52,445
6	0,063	1,084	53,112
7	5,688	0,975	50,747
8	0,072	1,122	51,585
9	0,062	1,168	52,668
10	0,590	0,993	51,341
11	0,101	1,121	52,552
12	0,065	1,190	52,517
13	3,830	0,943	51,521
Suma lub średnia	16,399	1,071	52,197
Rollback			
Kontekst	Flyway		
	Średni czas rollbacku [s]:	Średni procent zużycia procesora[%]:	Średnia zużycia pamięci[MB]:
1	0,084	0,964	50,820
2	4,547	0,967	50,952
3	0,007	1,014	50,776
4	0,008	1,014	50,613
5	5,929	0,970	50,498
6	0,021	0,969	50,843
7	4,712	0,966	51,226
8	0,007	1,090	51,043
9	0,008	1,069	51,180
10	0,022	0,990	51,036
11	0,014	0,937	50,910
12	0,008	0,923	50,762
13	0,023	0,935	50,603
Suma lub średnia	15,392	0,985	50,866
Kontekst	Liquibase		
	Średni czas rollbacku [s]:	Średni procent zużycia procesora[%]:	Średnia zużycia pamięci[MB]:
1	0,195	1,054	52,733
2	5,300	1,002	52,616
3	0,055	1,160	53,240
4	0,067	1,091	52,124
5	7,103	0,982	51,658
6	0,073	1,058	52,333
7	5,500	1,004	52,693
8	0,054	0,946	52,386
9	0,068	0,926	52,750
10	0,056	0,949	52,545
11	0,060	0,926	52,256
12	0,056	0,891	52,045
13	0,086	0,933	52,815
Suma lub średnia	18,673	0,994	52,476

Tabela 4.2 Wyniki Scenariusza 1.

4.1.2 Środowisko CI/CD

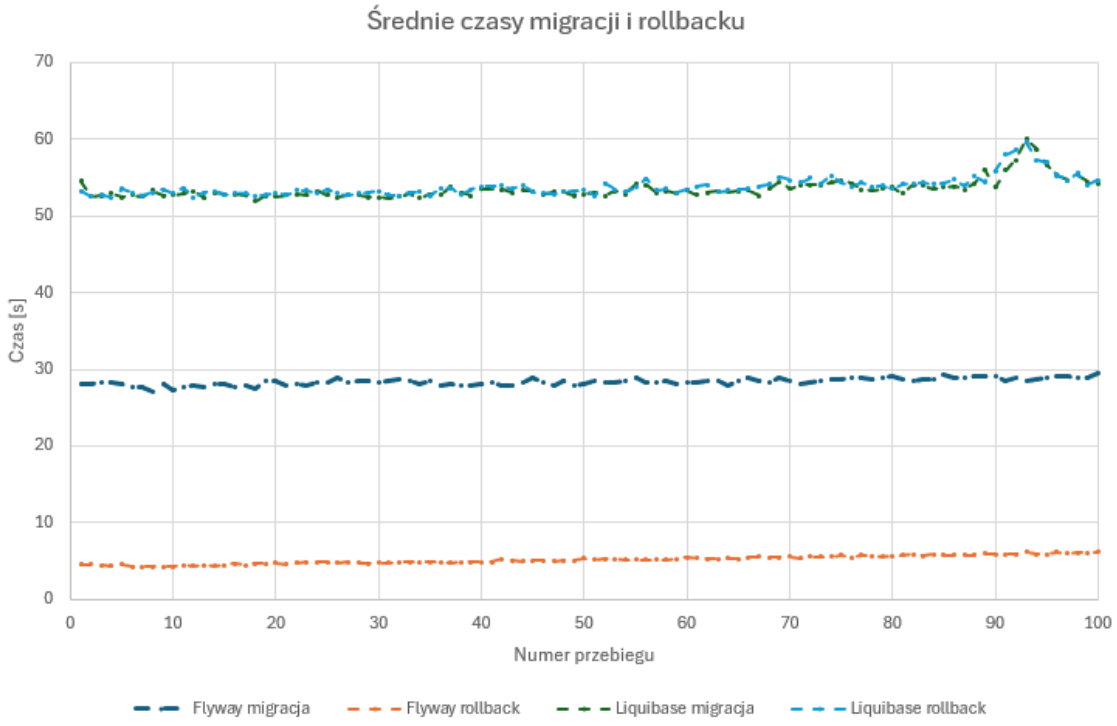
Część tabeli wynikowej została przedstawiona w tabeli 4.3. Opracowane wyniki ze scenariusza 1. zamieszczono w tabeli 4.4. Kolorami zielonym, czerwonym i niebieskim oznaczono odpowiednio lepsze, gorsze i takie same wyniki. Z analizy danych wynika, że Flyway również w środowisku CI/CD uzyskał lepsze wyniki niż Liquibase w badanych metrykach.

Średni czas wykonania migracji scenariusza w przypadku narzędzia Flyway wynosił $28,37s$, a w przypadku Liquibase wynosił $53,46s$. Flyway tutaj był znacznie szybszy od Liquibase będąc szybszym o około $46,94\%$.

Średni czas cofania zmian dla narzędzia Flyway wynosił $5,188s$, a dla narzędzia Liquibase $53,78s$. Między narzędziami jest ogromna różnica, ponieważ Flyway jest szybszy o około $90,35\%$. Taka różnica nie może być przypadkowa i może wynikać z tego, że Flyway w darmowej wersji nie udostępnia opcji wycofywania zmian, więc aby wykonać rollback wykonywane są bezpośrednio skrypty SQL, bez użycia narzędzia.

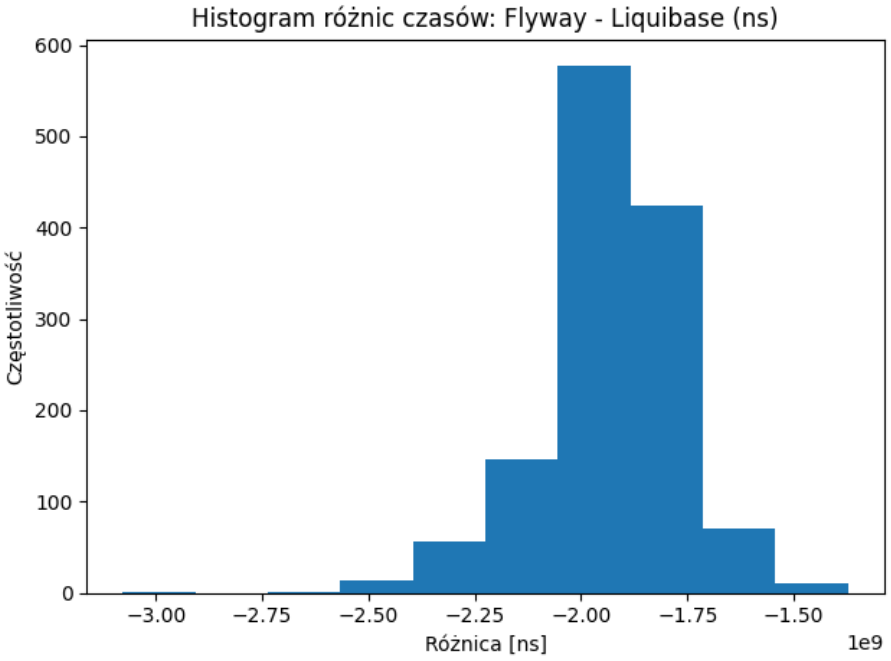
Różnica pomiędzy czasem migracji i czasem cofania zmian wynosi dla Flyway około $28,181s$, a dla Liquibase $0,32s$. Z tego widać, że Liquibase jest bardziej stabilny. Być może czas cofania dla Flyway byłby zbliżony do czasu wykonania migracji, jeżeli byłby wykonywany przez narzędzie, a nie przez skrypty SQL.

Zestawienie czasów jest widoczne na wykresie 4.3.



Rysunek 4.3 Zestawienie średnich czasów migracji i cofania zmian dla każdego z przebiegów w środowisku CI/CD.

- Test Shapiro–Wilka
  Wyniki testy Shapiro–Wilka, $W = 0,9606$, $p = 2,87 \cdot 10^{-18}$, wskazują, że pomimo faktu, że rozkład wygląda na normalny, bo wartość W jest zbliżona do 1, to należy odrzucić hipotezę o normalności, ponieważ wartość p jest dużo mniejsza od 0,05. Duża ilości obserwacji ma bardzo dużą moc testu, więc nawet niewielkie odchylenia od rozkładu normalnego potrafią dać ekstremalnie małą wartość p. Rozkład jest widoczny na rysunku 4.4.



Rysunek 4.4 Histogram różnic [ns] scenariusza 1. w środowisku CI/CD.

- Test t dla par zależnych
  Pominięto z racji na stwierdzenie nienormalności rozkładu.

- Test Wilcoxona
  Wartość statystyki testowej stat jest równa $stat = 0$, co oznacza, że wszystkie różnice mają taki sam znak. Wartość $p = 4.04 \cdot 10^{-214}$. Z racji, że wartość p jest znacznie mniejsza niż 0,05, istnieje dowód, że mediana różnic jest różna od zera, czyli czasy migracji istotnie się różnią.

tool	scenario	ctx	migrationTimeNs	rollbackTimeNs	exitCode	scriptLines	cpuUsage	memoryUsage	successRate
Flyway	Scen1	2	1999250522	0	0	2	0	0	100.0
Flyway	Scen1	3	2959680241	0	0	2	0	0	100.0
Flyway	Scen1	4	1883645961	0	0	19	0	0	100.0
Flyway	Scen1	5	1860269854	0	0	5	0	0	100.0
Flyway	Scen1	6	1902237825	0	0	2	0	0	100.0
Flyway	Scen1	7	2051094194	0	0	2	0	0	100.0
Flyway	Scen1	8	3442892217	0	0	4	0	0	100.0
Flyway	Scen1	9	1816282862	0	0	18	0	0	100.0
Flyway	Scen1	10	1852579451	0	0	4	0	0	100.0
Flyway	Scen1	11	2081383172	0	0	1	0	0	100.0
Flyway	Scen1	12	1959138974	0	0	8	0	0	100.0
Flyway	Scen1	13	1928867731	0	0	22	0	0	100.0
Flyway	Scen1	14	2340918319	0	0	2	0	0	100.0
Flyway-R	Scen1	14	0	82198095	0	1	0	0	100.0
Flyway-R	Scen1	13	0	75073355	0	2	0	0	100.0
Flyway-R	Scen1	12	0	75642146	0	1	0	0	100.0
Flyway-R	Scen1	11	0	77931016	0	1	0	0	100.0
Flyway-R	Scen1	10	0	76877204	0	1	0	0	100.0
Flyway-R	Scen1	9	0	77682655	0	1	0	0	100.0
Flyway-R	Scen1	8	0	1252782588	0	3	0	0	100.0
Flyway-R	Scen1	7	0	79434686	0	24	0	0	100.0
Flyway-R	Scen1	6	0	1393026232	0	5	0	0	100.0
Flyway-R	Scen1	5	0	77425973	0	1	0	0	100.0
Flyway-R	Scen1	4	0	75475098	0	1	0	0	100.0
Flyway-R	Scen1	3	0	1160489111	0	2	0	0	100.0
Flyway-R	Scen1	2	0	75119423	0	2	0	0	100.0
Liquibase	Scen1	1	5077171488	0	0	0	0	0	100.0
Liquibase	Scen1	2	4894235656	0	0	0	0	0	100.0
Liquibase	Scen1	3	3847833843	0	0	0	0	0	100.0
Liquibase	Scen1	4	3848721640	0	0	0	0	0	100.0
Liquibase	Scen1	5	3896331805	0	0	0	0	0	100.0
Liquibase	Scen1	6	3736435722	0	0	0	0	0	100.0
Liquibase	Scen1	7	5077439028	0	0	0	0	0	100.0
Liquibase	Scen1	8	3950818993	0	0	0	0	0	100.0
Liquibase	Scen1	9	3791792780	0	0	0	0	0	100.0
Liquibase	Scen1	10	4112357985	0	0	0	0	0	100.0
Liquibase	Scen1	11	3851563591	0	0	0	0	0	100.0
Liquibase	Scen1	12	3947877677	0	0	0	0	0	100.0
Liquibase	Scen1	13	4466702753	0	0	0	0	0	100.0
Liquibase-R	Scen1	13	0	3756228683	0	0	0	0	100.0
Liquibase-R	Scen1	12	0	3790445799	0	0	0	0	100.0
Liquibase-R	Scen1	11	0	4063505538	0	0	0	0	100.0
Liquibase-R	Scen1	10	0	3829011985	0	0	0	0	100.0
Liquibase-R	Scen1	9	0	3874085521	0	0	0	0	100.0
Liquibase-R	Scen1	8	0	3817089878	0	0	0	0	100.0
Liquibase-R	Scen1	7	0	4994274522	0	0	0	0	100.0
Liquibase-R	Scen1	6	0	3768449340	0	0	0	0	100.0
Liquibase-R	Scen1	5	0	5013506954	0	0	0	0	100.0
Liquibase-R	Scen1	4	0	3778974456	0	0	0	0	100.0
Liquibase-R	Scen1	3	0	3837482047	0	0	0	0	100.0
Liquibase-R	Scen1	2	0	5004092185	0	0	0	0	100.0
Liquibase-R	Scen1	1	0	3650734582	0	0	0	0	100.0

Tabela 4.3 Przykładowe wyniki scenariusza 1. na Github Actions

Migracja		
Kontekst	Flyway	
	Średni czas migracji [s]:	Średni wskaźnik sukcesu migracji [%]
1	1,91	100
2	3,22	100
3	1,90	100
4	1,91	100
5	1,89	100
6	1,91	100
7	3,23	100
8	1,89	100
9	1,90	100
10	2,14	100
11	1,90	100
12	1,92	100
13	2,64	100
Suma lub średnia	28,37	100
Kontekst	Liquibase	
	Średni czas migracji [s]:	Średni wskaźnik sukcesu migracji [%]
1	3,84	100
2	5,08	100
3	3,83	100
4	3,83	100
5	3,85	100
6	3,82	100
7	5,10	100
8	3,85	100
9	3,83	100
10	4,05	100
11	3,93	100
12	3,85	100
13	4,60	100
Suma lub średnia	53,46	100
Rollback		
Kontekst	Flyway	
	Średni czas rollbacku [s]:	Średni wskaźnik sukcesu migracji [%]
1	0,080	100
2	1,316	100
3	0,077	100
4	0,078	100
5	1,681	100
6	0,079	100
7	1,413	100
8	0,077	100
9	0,077	100
10	0,077	100
11	0,077	100
12	0,078	100
13	0,079	100
Suma lub średnia	5,188	100
Kontekst	Liquibase	
	Średni czas rollbacku [s]:	Średni wskaźnik sukcesu migracji [%]
1	3,81	100
2	5,04	100
3	3,79	100
4	3,79	100
5	5,44	100
6	3,81	100
7	5,22	100
8	3,79	100
9	3,79	100
10	3,83	100
11	3,86	100
12	3,80	100
13	3,81	100
Suma lub średnia	53,78	100

Tabela 4.4 Wyniki Scenariusza 1. w środowisku Github Actions.

4.2 Scenariusz 2.

Opis scenariusza 2. został przedstawiony w podrozdziale 3.2.2.

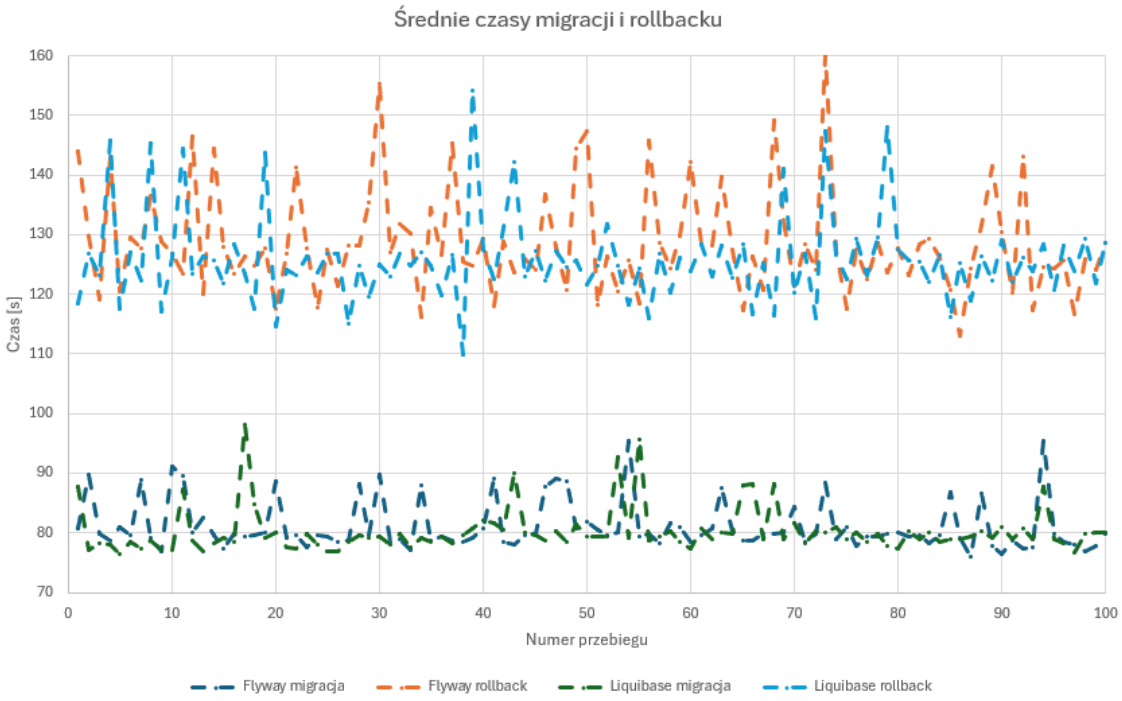
4.2.1 Środowisko lokalne

Część wyników uzyskanych w środowisku lokalnym dla scenariusza 2. przedstawiono w tabeli 4.5. Zestawienie opracowanych wyników zamieszczono w tabeli 4.6. Wartości oznaczone kolorem zielonym i czerwonym odpowiadają odpowiednio lepszym i gorszym rezultatom, analogicznie jak w przypadku scenariusza 1.

Z analizy danych zawartych w tabeli 4.6 wynika, że średni czas migracji w scenariuszu 2. za pomocą Flyway wynosił $81,08s$, natomiast dla Liquibase $80,26s$. Różnica na korzyść Liquibase wyniosła $0,82s$, co stanowi przyspieszenie o około $1,01\%$. W trakcie migracji Flyway generował średnie obciążenie procesora na poziomie $0,94\%$, podczas gdy Liquibase $1,05\%$, co daje różnicę $0,13\%$. W zakresie zużycia pamięci podczas migracji Flyway odnotował przeciętnie $79,78MB$, a Liquibase $80,96MB$, co oznacza oszczędność pamięci przez Flyway na poziomie $1,18MB$ (ok. $1,46\%$).

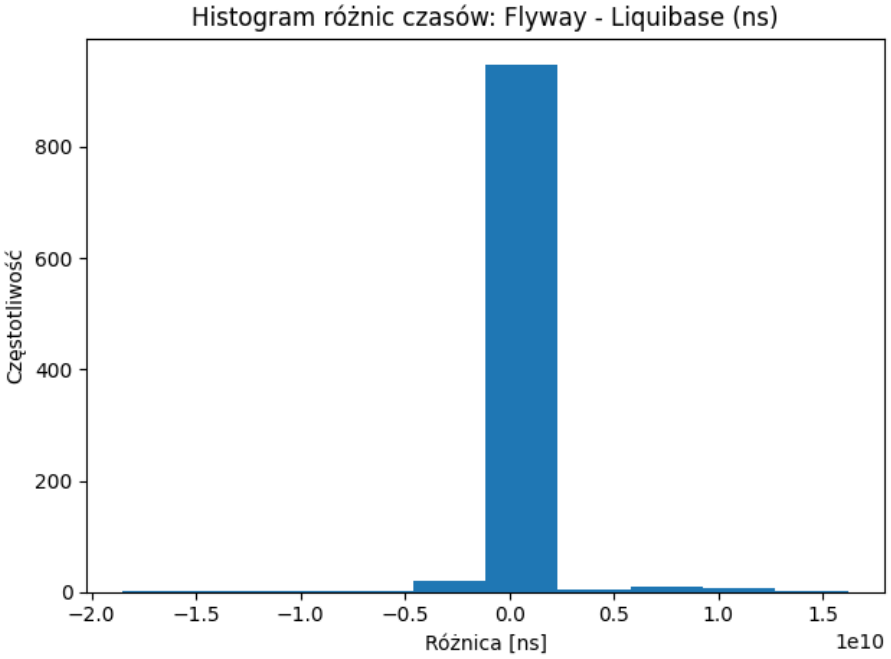
Jeżeli chodzi o średni czas cofania zmian, Liquibase ponownie okazał się bardziej efektywny, wykonując operację w $125,81s$, podczas gdy Flyway potrzebował $128,64s$. Przewaga Liquibase w tym zakresie wyniosła $2,83s$ (tj. $2,20\%$). Średnie zużycie procesora w trakcie rollbacku było zbliżone do wartości odnotowanych podczas migracji: Flyway zużywał $0,93\%$, a Liquibase $1,10\%$ CPU (różnica $0,17\%$). W kontekście zużycia pamięci Flyway wykorzystał średnio $79,52MB$, a Liquibase $80,77MB$, co przekłada się na oszczędność pamięci przez Flyway o $1,25MB$.

Zestawienie czasów migracji oraz rollbacku dla poszczególnych przebiegów testowych zilustrowano na wykresie 4.5.



Rysunek 4.5 Zestawienie średnich czasów migracji i cofania zmian dla każdego z przebiegów.

- Test Shapiro–Wilka
  W tym teście wartości wynosiły $W = 0,3770$, $p = 8,11 \cdot 10^{-50}$. Wskazuje to, że rozkład odbiega od normalnego. Wartość p jest znacznie mniejsza od 0,05, dlatego należy odrzucić hipotezę o normalności rozkładu. Rozkład różnic przedstawiono na rysunku 4.6.



Rysunek 4.6 Histogram różnic [ns] scenariusza 2. w środowisku lokalnym.

- Test t dla par zależnych
  Pominięto z racji na stwierdzenie nienormalności rozkładu.

- Test Wilcoxona
  Wartość statystyki *stat* była równa $stat = 241132$, a wartość p wynosiła $p = 0.3182$. Z racji, że wartość $p > 0,05$ nie ma powodu, aby odrzucić hipotezę, że mediana różnic jest równa zero, czyli dane nie różnią się od siebie istotnie statystycznie.

tool	identifier	context	migration_ns	rollback_ns	exit_code	script_lines	cpu_pct	mem_bytes
Flyway	1,fwd,14	14	236233700	0	0	11	0,049399418	98524080
Flyway	1,fwd,15	15	218052100	0	0	3	0,446778506	102495072
Flyway	1,fwd,16	16	186944100	0	0	3	0,844157595	105498008
Flyway	1,fwd,17	17	71286667200	0	0	10	0,786554966	109966837
Flyway	1,fwd,18	18	420888000	0	0	6	0,582915114	78535942
Flyway	1,fwd,19	19	1800921200	0	0	2	0,999521331	48356149
Flyway	1,fwd,20	20	204694700	0	0	3	0,694754007	51106768
Flyway	1,fwd,21	21	636393200	0	0	2	0,769469907	54651592
Flyway	1,fwd,22	22	224738600	0	0	3	0,819454785	57393472
Flyway	1,fwd,23	23	5540442600	0	0	1	1,068046257	61015266
Flyway	1,rbk,23	23	0	22971800	0	1	0,774099795	61867824
Flyway	1,rbk,22	22	0	33541800	0	2	0,788352942	61993680
Flyway	1,rbk,21	21	0	2402700	0	1	0,788352942	62123704
Flyway	1,rbk,20	20	0	26140100	0	2	1,473447764	62724488
Flyway	1,rbk,19	19	0	834900	0	1	1,473447764	62850344
Flyway	1,rbk,18	18	0	24681300	0	1	1,473447764	62978296
Flyway	1,rbk,17	17	0	1,40243E+11	0	50	0,929552349	64188196
Flyway	1,rbk,16	16	0	1063529000	0	1	0,962656416	65362976
Flyway	1,rbk,15	15	0	1362339400	0	1	0,844202613	65997390
Flyway	1,rbk,14	14	0	1308530100	0	1	1,222644233	66124374
Liquibase	1,fwd,14	14	75551300	0	0	25	1,405995037	46577472
Liquibase	1,fwd,15	15	213382600	0	0	7	0,201832111	58130548
Liquibase	1,fwd,16	16	239243900	0	0	7	0,746711389	66266336
Liquibase	1,fwd,17	17	78599167500	0	0	27	0,84388718	75978320
Liquibase	1,fwd,18	18	329955100	0	0	8	0,719504498	83709362
Liquibase	1,fwd,19	19	1708509100	0	0	6	0,835061759	92767446
Liquibase	1,fwd,20	20	240969800	0	0	7	1,293781654	99383592
Liquibase	1,fwd,21	21	710045600	0	0	6	1,210170773	108981944
Liquibase	1,fwd,22	22	192565000	0	0	7	0,950091474	47618972
Liquibase	1,fwd,23	23	5465838200	0	0	4	1,212776287	57527413
Liquibase	1,rbk,23	23	0	49883300	0	4	0,827144834	63325400
Liquibase	1,rbk,22	22	0	60174400	0	7	0,727113763	70578488
Liquibase	1,rbk,21	21	0	45572400	0	6	0,727113763	77861080
Liquibase	1,rbk,20	20	0	86322200	0	7	1,078678276	85422944
Liquibase	1,rbk,19	19	0	79075500	0	6	1,078678276	92071176
Liquibase	1,rbk,18	18	0	56244300	0	8	1,265669187	100210536
Liquibase	1,rbk,17	17	0	1,14182E+11	0	27	0,783312237	109108203
Liquibase	1,rbk,16	16	0	982212900	0	7	0,808149916	47028067
Liquibase	1,rbk,15	15	0	1196964800	0	7	0,648939732	54629044
Liquibase	1,rbk,14	14	0	1763826200	0	25	1,032724986	61906306
Liquibase	2,rbk,23	23	0	42530900	0	4	1,033828263	83600176

Tabela 4.5 Przykładowe wyniki scenariusza 2. na maszynie lokalnej

Migracja			
Kontekst	Flyway		
	Średni czas migracji [s]:	Średni procent zużycia procesora[%]:	Średnia zużycia pamięci[MB]:
14	0,09	1,17	79,99
15	0,20	0,85	80,92
16	0,21	0,90	80,20
17	71,71	0,83	79,63
18	0,48	0,93	80,09
19	1,94	0,90	79,16
20	0,31	0,95	79,15
21	0,71	1,04	79,23
22	0,26	0,92	79,88
23	5,17	0,93	79,59
Suma lub średnia	81,08	0,94	79,78
Kontekst	Liquibase		
	Średni czas migracji [s]:	Średni procent zużycia procesora[%]:	Średnia zużycia pamięci[MB]:
14	0,08	1,32	82,81
15	0,21	1,10	80,45
16	0,23	1,21	80,30
17	70,77	0,85	81,20
18	0,59	1,01	81,71
19	1,89	0,94	79,00
20	0,29	0,94	79,69
21	0,73	1,17	83,46
22	0,28	0,97	81,61
23	5,18	0,99	79,32
Suma lub średnia	80,26	1,05	80,96
Rollback			
Kontekst	Flyway		
	Średni czas rollbacku [s]:	Średni procent zużycia procesora[%]:	Średnia zużycia pamięci[MB]:
14	1,26	1,16	79,16
15	1,17	0,84	78,95
16	0,99	0,85	78,80
17	125,00	0,86	79,52
18	0,07	0,97	79,16
19	0,01	0,95	80,32
20	0,06	0,98	80,14
21	0,01	0,91	79,93
22	0,06	0,88	79,72
23	0,02	0,85	79,53
Suma lub średnia	128,64	0,93	79,52
Kontekst	Liquibase		
	Średni czas rollbacku [s]:	Średni procent zużycia procesora[%]:	Średnia zużycia pamięci[MB]:
14	1,30	1,32	82,34
15	1,23	0,93	78,61
16	1,10	0,94	79,96
17	121,69	0,85	82,23
18	0,12	1,26	80,17
19	0,06	1,22	79,29
20	0,09	1,22	81,69
21	0,07	1,21	82,57
22	0,08	1,13	81,73
23	0,07	0,96	79,17
Suma lub średnia	125,81	1,10	80,77

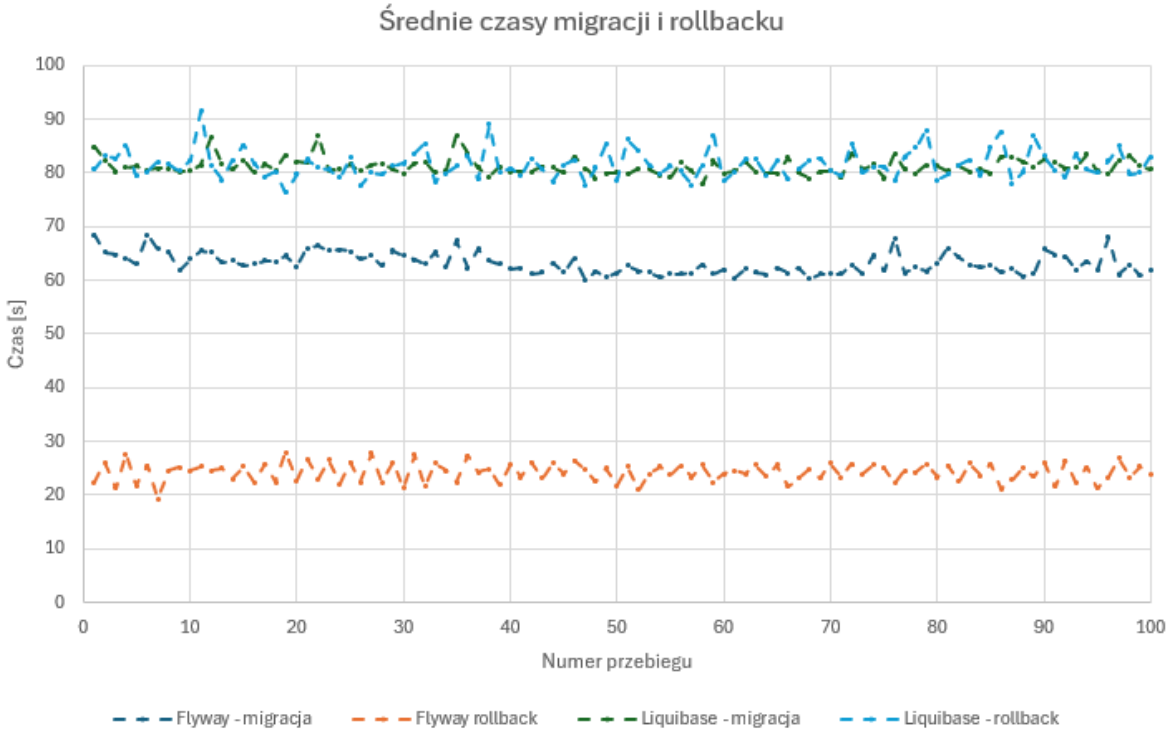
Tabela 4.6 Wyniki Scenariusza 2. w środowisku lokalnym.

4.2.2 Środowisko CI/CD

Część tabeli wynikowej badań w środowisku CI/CD została przedstawiona w tabeli 4.7. Opracowane wyniki ze scenariusza 2. w środowisku CI/CD zamieszczono w tabeli 4.8.

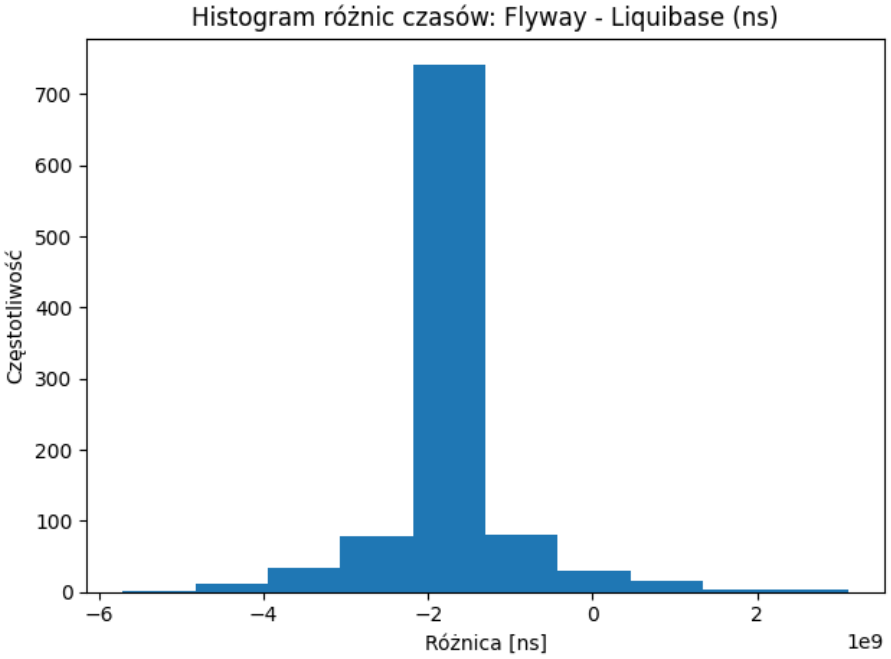
Z analizy danych zawartych w tabeli 4.8 wynika, że Flyway uzyskał krótsze czasy zarówno migracji, jak i rollbacku w porównaniu z Liquibase. Średni czas migracji za pomocą Flyway wyniósł 63,13 s, podczas gdy Liquibase potrzebował 81,15 s, co przekłada się na przyspieszenie o 18,02 s (około 22,21 %). W przypadku cofania zmian Flyway zakończył operację w 24,22 s, natomiast Liquibase wymagał 81,54 s, dając przewagę Flyway na poziomie 57,32 s (około 70,30 %). Należy przy tym uwzględnić, że rollback w Flyway realizowany jest poprzez bezpośrednie uruchamianie skryptów SQL, co może wpłynąć na obserwowaną różnicę wydajności.

Zestawienie czasów migracji i cofania zmian dla każdego przebiegu testowego przedstawiono na wykresie 4.7.



Rysunek 4.7 Zestawienie średnich czasów migracji i cofania zmian dla każdego z przebiegów w środowisku CI/CD.

- Test Shapiro–Wilka
  W tym teście wartość W była równa $W = 0.7832$, a wartość p wynosiła $p = 2.0 \cdot 10^{-34}$. Z racji, że wartość $p < 0,05$ można odrzucić hipotezę o normalności rozkładu, mimo że rozkład może w pewnym stopniu przypominać rozkład normalny. Rozkład różnic przedstawiono na rysunku 4.8.



Rysunek 4.8 Histogram różnic [ns] scenariusza 2. w środowisku CI/CD.

- Test t dla par zależnych
  Pominięto z racji na stwierdzenie nienormalności rozkładu.

- Test Wilcoxona
  Wartość statystyki $stat$ była równa $stat = 6257$, a wartość p wynosiła $p = 3.79 \cdot 10^{-157}$. Z racji, że wartość p jest dużo niższa od $0,05$ należy odrzucić hipotezę, że mediana różnic jest równa zero, czyli dane różnią się od siebie istotnie statystycznie.

tool	scenario	ctx	migrationTimeNs	rollbackTimeNs	exitCode	scriptLines	cpuUsage	memoryUsage	successRate
Flyway	Scen2	15	5553193539	0	0	13	0	0	100
Flyway	Scen2	16	2029872632	0	0	3	0	0	100
Flyway	Scen2	17	2170088971	0	0	3	0	0	100
Flyway	Scen2	18	41294001549	0	0	10	0	0	100
Flyway	Scen2	19	2142947409	0	0	6	0	0	100
Flyway	Scen2	20	4185818591	0	0	2	0	0	100
Flyway	Scen2	21	2369362412	0	0	3	0	0	100
Flyway	Scen2	22	2351712269	0	0	2	0	0	100
Flyway	Scen2	23	2041037608	0	0	3	0	0	100
Flyway	Scen2	24	4290297217	0	0	1	0	0	100
Flyway-R	Scen2	24	0	97992925	0	2	0	0	100
Flyway-R	Scen2	23	0	88282758	0	2	0	0	100
Flyway-R	Scen2	22	0	75833748	0	2	0	0	100
Flyway-R	Scen2	21	0	86072263	0	2	0	0	100
Flyway-R	Scen2	20	0	75822914	0	2	0	0	100
Flyway-R	Scen2	19	0	105677087	0	1	0	0	100
Flyway-R	Scen2	18	0	21304110641	0	54	0	0	100
Flyway-R	Scen2	17	0	99040660	0	1	0	0	100
Flyway-R	Scen2	16	0	114257954	0	1	0	0	100
Flyway-R	Scen2	15	0	298219744	0	1	0	0	100
Liquibase	Scen2	14	4239420441	0	0	0	0	0	100
Liquibase	Scen2	15	3930226408	0	0	0	0	0	100
Liquibase	Scen2	16	4070444522	0	0	0	0	0	100
Liquibase	Scen2	17	43169157674	0	0	0	0	0	100
Liquibase	Scen2	18	4103925137	0	0	0	0	0	100
Liquibase	Scen2	19	6658877659	0	0	0	0	0	100
Liquibase	Scen2	20	4055856521	0	0	0	0	0	100
Liquibase	Scen2	21	4307633041	0	0	0	0	0	100
Liquibase	Scen2	22	4137347928	0	0	0	0	0	100
Liquibase	Scen2	23	6201519141	0	0	0	0	0	100
Liquibase-R	Scen2	23	0	3751844436	0	0	0	0	100
Liquibase-R	Scen2	22	0	3792186184	0	0	0	0	100
Liquibase-R	Scen2	21	0	3831708915	0	0	0	0	100
Liquibase-R	Scen2	20	0	3841476034	0	0	0	0	100
Liquibase-R	Scen2	19	0	4443230063	0	0	0	0	100
Liquibase-R	Scen2	18	0	3879862922	0	0	0	0	100
Liquibase-R	Scen2	17	0	44647608291	0	0	0	0	100
Liquibase-R	Scen2	16	0	4206109821	0	0	0	0	100
Liquibase-R	Scen2	15	0	4258353115	0	0	0	0	100
Liquibase-R	Scen2	14	0	4060786622	0	0	0	0	100

Tabela 4.7 Przykładowe wyniki scenariusza 2. na Github Actions

Migracja		
Kontekst	Flyway	
	Średni czas migracji [s]:	Średni wskaźnik sukcesu migracji [%]
14	2,01	100
15	2,08	100
16	2,07	100
17	40,51	100
18	2,29	100
19	3,46	100
20	2,36	100
21	2,29	100
22	2,32	100
23	3,73	100
Suma lub średnia	63,13	100
Kontekst	Liquibase	
	Średni czas migracji [s]:	Średni wskaźnik sukcesu migracji [%]
14	3,92	100
15	3,93	100
16	3,94	100
17	41,30	100
18	4,25	100
19	5,40	100
20	4,26	100
21	4,23	100
22	4,19	100
23	5,72	100
Suma lub średnia	81,15	100
Rollback		
Kontekst	Flyway	
	Średni czas rollbacku [s]:	Średni wskaźnik sukcesu migracji [%]
14	0,75	100
15	0,15	100
16	0,12	100
17	22,62	100
18	0,12	100
19	0,08	100
20	0,10	100
21	0,08	100
22	0,10	100
23	0,09	100
Suma lub średnia	24,22	100
Kontekst	Liquibase	
	Średni czas rollbacku [s]:	Średni wskaźnik sukcesu migracji [%]
14	4,74	100
15	4,46	100
16	4,33	100
17	44,54	100
18	3,98	100
19	3,95	100
20	3,92	100
21	3,92	100
22	3,86	100
23	3,84	100
Suma lub średnia	81,54	100

Tabela 4.8 Wyniki Scenariusza 2. w środowisku Github Actions.

4.3 Zestawienie dla metryki - Złożoność skryptów migracyjnych

Złożoność skryptów migracyjnych obliczono za pomocą dwóch skryptów, osobno dla Flyway i Liquibase. Pierwszy z nich oblicza metryki dla narzędzi Flyway. Szuka plików z rozszerzeniem `.sql`. Pliki zaczynające się od `V` klasyfikuje jako migracje, a pliki zaczynające się od `U` jako skrypty cofania zmian. Numer wersji wyodrębniany jest z nazwy pliku, a na jego podstawie ustalany jest kontekst analizy. Dla każdego pliku obliczana jest liczba niepustych i niekomentarzowych linii kodu (LOC) [32], liczba wystąpień operacji takich jak `CREATE, ALTER, DROP, INSERT, UPDATE` i `DELETE` [14], przybliżona głębokość zagnieżdżeń wyznaczana na podstawie poziomu wcięć (4 spacje = 1 poziom), metryka cyklomatyczna w ujęciu McCabe'a (tj. 1 plus liczba punktów decyzyjnych takich jak `IF, ELSIF, LOOP, CASE` czy `WHEN`) [47]. Drugi skrypt został przygotowany dla Liquibase. Struktura skryptu XML jest parsowana z uwzględnieniem przestrzeni nazw Liquibase. Każdy element `<changeSet>` jest traktowany jako niezależna jednostka, a zawartość SQL (wewnątrz elementów `<sql>` i `CDATA`) łączona jest z reprezentacją XML całego zestawu zmian. Pozwala to uwzględnić zarówno deklaratywne operacje takie jak `<addColumn>` czy `<dropColumn>`, jak i dowolny kod SQL. Obliczane są analogiczne metryki jak w przypadku skryptów Flyway, przy czym głębokość zagnieżdżeń logicznych w ramach `preConditions (AND, OR, NOT)` mierzy się rekurencyjnie na podstawie struktury drzewa XML. Konteksty 1-13 odpowiadają scenariuszowi 1., a konteksty 14-23 scenariuszowi 2.. Wyniki skomplikowania dla Flyway są widoczne w tabeli 4.10, a dla Liquibase w tabeli 4.9.

Kontekst	changeSet	LOC	Instrukcje	Maks. głęb. zagn.	Metryka cyklomatyczna
1	1-add-new-name	11	0	1	1
2	2-copy-data	8	4	0	1
3	3-create-function	44	7	0	19
4	4-create-trigger	14	11	0	5
5	5-drop-legacy	11	3	0	1
6	6-disable-legacy-trigger	69	16	3	23
7	7-split-name-columns	26	4	2	1
8	8-create-name-sync-function	40	13	0	15
9	9-create-name-sync-trigger	16	11	0	5
10	10-add-indexes	11	0	1	1
11	11-create-audit-table	18	1	1	1
12	12-audit-trigger	56	34	0	15
13	13-change-type-error	11	0	0	1
14	14-create-big-table	35	3	1	7
15	15-insert-batch-1	12	5	0	1
16	16-insert-batch-2	12	5	0	1
17	17-update-extra-cols	42	5	0	9
18	18-insert-batch-3	14	5	0	1
19	19-delete-batch-1	8	3	0	1
20	20-insert-batch-4	12	5	0	1
21	21-delete-batch-2	8	3	0	1
22	22-insert-batch-5	12	5	0	1
23	23-analyze	6	0	0	1

Tabela 4.9 Metryki dla poszczególnych changeSet w narzędziu Liquibase.

Kontekst	Typ skryptu	LOC	Instrukcje	Maks. gł. zagn.	Metryka cyklomatyczna
1	Migracja	2	1	1	1
1	Rollback	2	2	0	2
2	Migracja	2	1	0	1
2	Rollback	2	1	0	1
3	Migracja	18	2	3	9
3	Rollback	1	1	0	2
4	Migracja	5	4	6	2
4	Rollback	1	1	0	2
5	Migracja	2	2	0	1
5	Rollback	4	2	1	1
6	Migracja	2	2	0	3
6	Rollback	22	6	6	10
7	Migracja	4	3	4	3
7	Rollback	3	5	0	3
8	Migracja	16	3	3	7
8	Rollback	1	1	0	2
9	Migracja	4	4	6	2
9	Rollback	1	1	0	2
10	Migracja	1	1	0	2
10	Rollback	1	1	0	2
11	Migracja	8	1	1	2
11	Rollback	1	1	0	2
12	Migracja	21	15	2	6
12	Rollback	2	2	0	3
13	Migracja	2	2	0	1
13	Rollback	1	2	0	1
14	Migracja	11	2	1	5
14	Rollback	1	1	0	2
15	Migracja	3	1	0	1
15	Rollback	1	1	0	1
16	Migracja	3	1	0	1
16	Rollback	1	1	0	1
17	Migracja	10	1	1	3
17	Rollback	50	50	0	1
18	Migracja	6	1	1	1
18	Rollback	1	1	0	1
19	Migracja	2	1	0	1
19	Rollback	1	0	0	1
20	Migracja	3	1	0	1
20	Rollback	2	1	0	1
21	Migracja	2	1	0	1
21	Rollback	1	0	0	1
22	Migracja	3	1	0	1
22	Rollback	2	1	0	1
23	Migracja	1	0	0	1
23	Rollback	1	0	0	1

Tabela 4.10 Metryki skryptów migracyjnych i cofających zmiany dla narzędzia Flyway.

W narzędziu Flyway metryki dotyczące migracji i rollbacku są odrębne w ramach każdego kontekstu, w celu umożliwienia bezpośredniego porównania z Liquibase, gdzie takich rozróżnień nie ma, wartości czasów migracji oraz rollbacku zostaną dla każdego kontekstu zsumowane, osobno dla każdego z kontekstów.

Jako ujednoliconą miarę złożoności zarówno skryptów Flyway i dzienników zmian Liquibase, przyjęto tzw. wskaźnik złożoności zdefiniowany wg wzoru 4.1.

$$CI_i = w_{LOC} \cdot \frac{LOC_i}{dLOC} + w_{STMT} \cdot \frac{STMT_i}{dSTMT} + w_{DEPTH} \cdot \frac{DEPTH_i}{dDEPTH} + w_{CYC} \cdot \frac{CYC_i}{dCYC} \quad (4.1)$$

Gdzie:

- i - numer kontekstu.
- LOC - liczba istotnych linii kodu.
- $STMT$ – liczba instrukcji.
- $DEPTH$ – maksymalna głębokość zagnieżdżeń.
- CYC – metryka McCabe'a.
- $dLOC$, $dSTMT$, $dDEPTH$, $dCYC$ - średnie wartości metryk.
- w_{LOC}, w_{STMT}, w_{DEPTH}, w_{CYC} - wagi dobierane zgodnie z ważnością poszczególnych składników (suma wag = 1), wartości: $w_{CYC} = 0.40$, $w_{STMT} = 0.30$, $w_{LOC} = 0.20$, $w_{DEPTH} = 0.10$.

Największą wartość wagi nadano metryce cyklomatycznej $w_{CYC} = 0.40$, gdyż mierzy ona liczbę punktów decyzyjnych i najściślej wiąże się z trudnością testowania oraz ryzykiem błędów. Druga pod względem znaczenia jest liczba instrukcji $w_{STMT} = 0.30$, ponieważ wyznacza stopień „rozproszenia" logiki operacji w kodzie i wpływa na złożoność migracji bazy. Trzecią metryką pod względem wartości jest rozmiar kodu (LOC) $w_{LOC} = 0.20$, choć istotny, jest bardziej ogólną miarą objętości niż jakościowej struktury. Najmniejszą, ale wciąż znaczącą wagę $w_{DEPTH} = 0.10$ przypisano głębokości zagnieżdżeń, gdyż duże zagnieżdżenia utrudniają czytelność i utrzymanie kodu, jednak ich wpływ jest mniejszy niż wpływ złożoności decyzji czy skali operacji.

Na podstawie wyników z tabeli 4.11 można stwierdzić, że wartości poszczególnych wskaźników $CI_1...CI_{23}$ odzwierciedlają stopień złożoności każdego kontekstu migracji, wartość powyżej jedynki oznacza złożoność powyżej średniej, czyli bardzo złożony lub skomplikowany skrypt. Natomiast wartość poniżej jedynki wskazuje na skrypt o relatywnie niewielkiej złożoności.

Średnia arytmetyczna wszystkich 23 wskaźników została znormalizowana do jedności, dzieki czemu służy jako punkt odniesienia.

Mediana rzędu 0.57 dla Flyway i 0.44 dla Liquibase sygnalizuje, że połowa skryptów ma złożoność poniżej 57% maksymalnej złożoności, zaś połowa dzienników Liquibase utrzymuje się poniżej 44%. Oznacza to, że większość jego dzienników jest relatywnie mniej skomplikowana niż skrypty Flyway.

Odchylenie standardowe obliczone zarówno jako miara populacyjna (0.94 dla Flyway, 1.02 dla Liquibase), jak i próbki (0.96 dla Flyway, 1.05 dla Liquibase) dostarcza informacji o rozproszeniu wartości CI wokół średniej, przy czym odchylenie populacyjne opisuje zmienność w pełnym zbiorze istniejących kontekstów, a odchylenie próbki pozwala uogólnić wnioski na przyszłe, niezaobserwowane jeszcze migracje.

Wskaźnik	Flyway	Liquibase
CI_1	0,575	0,44
CI_2	0,37	0,35
CI_3	1,73	2,29
CI_4	1,13	1,07
CI_5	0,57	0,33
CI_6	2,47	4,05
CI_7	1,36	1,03
CI_8	1,56	2,22
CI_9	1,11	1,09
CI_{10}	0,51	0,44
CI_{11}	0,71	0,55
CI_{12}	2,29	3,38
CI_{13}	0,45	0,18
CI_{14}	1,10	1,29
CI_{15}	0,37	0,43
CI_{16}	0,37	0,43
CI_{17}	4,26	1,36
CI_{18}	0,49	0,45
CI_{19}	0,29	0,30
CI_{20}	0,39	0,43
CI_{21}	0,29	0,30
CI_{22}	0,39	0,43
CI_{23}	0,22	0,14
CI - średnia	1	1
CI - mediana	0,57	0,44
CI - odchylenie standardowe populacji	0,94	1,02
CI - odchylenie standardowe próbki	0,96	1,05

Tabela 4.11 Wyniki wskaźnika skomplikowania skryptów.

Wyższe odchylenie standardowe obserwowane w przypadku Liquibase w porównaniu z Flyway oznacza większą zmienność złożoności poszczególnych dzienników. Część z nich jest relatywnie niewielka i prosta, a inne znacznie rozbudowane, co przekłada się na mniej jednorodny profil migracji.

Gdy CI_{Flyway} jest dużo wyższe niż $CI_{Liquibase}$, to najczęściej znaczy, że sporo logiki i zabezpieczeń zostało wpisane bezpośrednio w skrypty SQL, a w odpowiadających im plikach XML tego nie ma. Natomiast odwrotne relacje sugerują intensywniejsze użycie mechanizmów `preConditions` lub `trigger` w Liquibase.

Podsumowując można stwierdzić, że na podstawie porównania wartości mediany oraz odchylenia standardowego skrypty Flyway cechują się wyższą typową złożonością przy stosunkowo jednorodnym rozkładzie, podczas gdy zestawy Liquibase wykazują niższą przeciętną złożoność kosztem większego zróżnicowania. W konsekwencji Flyway można uznać za rozwiązanie generujące bardziej przewidywalnie skomplikowane migracje, natomiast Liquibase za narzędzie o zmiennym profilu złożoności, w którym prostsze i znacznie bardziej rozbudowane dzienniki współistnieją w jednym procesie. Dzięki temu badaniu można podjąć decyzję, które narzędzie w danym projekcie będzie skuteczniejsze, bardziej spójne i przewidywalne (Flyway) czy też dopuszczające większą elastyczność (Liquibase kosztem większego rozrzutu złożoności).

Rozdział 5

Model efektywności

Model porównania efektywności narzędzi wspomagających zarządzanie zmianami w bazach danych stanowi kluczowy element analizy porównawczej tych rozwiązań. W ramach opracowanego modelu uwzględnione zostaną wszystkie metryki opisane w rozdziale 3.3.1, czyli:

- T_{mig} - czas migracji,
- T_{rbc} - czas cofania zmian,
- $CPLX$ - mediana stopnia skomplikowania skryptów migracyjnych,
- CPU - średnia średniego zużycia procesora podczas migracji i cofania zmian w scenariuszu 1. i 2.,
- RAM - średnia średniego zużycia pamięci podczas migracji i cofania zmian w scenariuszu 1. i 2.,
- $SUBJ$ - subiektywna ocena narzędzia $(1 - 5)$,
- R - dostępność funkcji cofania zmian w darmowej wersji.

Wskaźnik sukcesu migracji SR zostanie uwzględniony w modelu z wagą zerową, gdyż w warunkach testowych wszystkie migracje zakończyły się powodzeniem. Metryka ta nabiera znaczenia w środowisku produkcyjnym, gdzie istnieje ryzyko wystąpienia niepowodzeń, dlatego w praktycznym zastosowaniu możliwe będzie przypisanie jej odpowiedniej wagi w zależności od wymagań operacyjnych.

5.1 Normalizacja

Aby umożliwić porównanie metryk o różnych jednostkach i skalach, każda zmierzona wartość M_i zostanie poddana procedurze normalizacji. Proces normalizacji realizowany będzie w dwóch wariantach w zależności od interpretacji kierunku „lepszości" wartości.

1. Metryki, w których mniejsza wartość oznacza lepszy wynik („mniej = lepiej"):

$$\bar{M_i} = 1 - \frac{M_i - M_{min}}{M_{max} - M_{min}}, \tag{5.1}$$

   gdzie $M_{\min}$ i $M_{\max}$ to minimalna i maksymalna wartość danej metryki w całym zbiorze wynikowym badań ($M_{\min}$ i $M_{\max}$ są takie same dla środowiska lokalnego i CI/CD).

2. Metryki, w których większa wartość oznacza lepszy wynik („więcej = lepiej"):

$$\bar{M_i} = \frac{M_i - M_{\min}}{M_{\max} - M_{\min}}. \tag{5.2}$$

5.2 Stopień efektywności

Ogólny stopień efektywności zostanie wyrażony jako suma składowa stopnia efektywności dla środowiska lokalnego i środowiska CI/CD. Wersja lokalna zakłada, że wszystkie wagi już są znormalizowane i sumują się do 1. Wersja CI/CD bierze pod uwagę tylko podzbiór metryk. Ich wagi (w oryginale dobrane dla modelu lokalnego) nie sumują się do 1, więc dodanie dzielenia przez ich sumę zapewnia, że wynik stopnia efektywności dla środowiska CI/CD też będzie w skali $[0, 1]$ i będzie porównywalny ze stopniem efektywności dla środowiska lokalnego.

Stopień efektywności dla środowiska lokalnego zostanie wyrażony wg wzoru 5.3.

$$\begin{aligned} E_{LOCAL} = w_{T_{mig}} \cdot \bar{T_{migL}} + w_{T_{rbk}} \cdot \bar{T_{rbkL}} + w_{CPLX} \cdot \bar{CPLX} + w_{CPU} \cdot \bar{CPU} \\ + w_{RAM} \cdot \bar{RAM} + w_{SUBJ} \cdot \bar{SUBJ} + w_R \cdot \bar{R} + w_{SR} \cdot SR \end{aligned} \tag{5.3}$$

Stopień efektywności dla środowiska CI/CD zostanie wyrażony wg wzoru 5.4.

$$E_{CI} = \frac{w_{T_{mig}} \cdot \bar{T_{migCI}} + w_{T_{rbk}} \cdot \bar{T_{rbkCI}} + w_{CPLX} \cdot \bar{CPLX} + w_{SUBJ} \cdot \bar{SUBJ} + w_R \cdot \bar{R} + w_{SR} \cdot \bar{SR}}{w_{T_{mig}} + w_{T_{rbk}} + w_{CPLX} + w_{SUBJ} + w_R + w_{SR}} \tag{5.4}$$

Ostateczny stopień efektywności zostanie wyrażony wg wzoru 5.5.

$$E = \alpha \cdot E_{LOCAL} + (1 - \alpha) \cdot E_{CI} \tag{5.5}$$

Gdzie:

- E_{LOCAL} - Wskaźnik efektywności narzędzia w środowisku lokalnym.
- $w_{T_{mig}}$ - Waga metryki „czas migracji".
- $\bar{T_{migL}}$ - Znormalizowany do $[0, 1]$ czas wykonania migracji w środowisku lokalnym.
- $w_{T_{rbk}}$ - Waga metryki „czas rollbacku".
- $\bar{T_{rbkL}}$ - Znormalizowany do $[0, 1]$ czas cofnięcia zmian w środowisku lokalnym.
- w_{CPLX} - Waga metryki „złożoność skryptów migracyjnych".
- $\bar{CPLX}$ - Znormalizowana do $[0, 1]$ wartość złożoności skryptów.
- w_{CPU} - Waga metryki „zużycie CPU".
- $\bar{CPU}$ - Znormalizowane do $[0, 1]$ zużycie CPU w środowisku lokalnym.
- w_{RAM} - Waga metryki „zużycie pamięci RAM" podczas migracji/rollbacku w środowisku lokalnym.
- $\bar{RAM}$ - Znormalizowane do $[0, 1]$ zużycie pamięci RAM w środowisku lokalnym.
- w_{SUBJ} - Waga metryki „subiektywna ocena narzędzia" (ocena zespołu w skali 1–5).
- $\bar{SUBJ}$ - Znormalizowana do $[0, 1]$ wartość subiektywnej oceny.
- w_R - Waga metryki „dostępność funkcji rollbacku".
- $\bar{R}$ - Znormalizowana do $[0, 1]$ wartość automatyzacji rollbacku.
- w_{SR} - Waga metryki „wskaźnik sukcesu migracji".
- SR - Znormalizowany wskaźnik sukcesu migracji.
- E_{CI} - Wskaźnik efektywności narzędzia w środowisku CI/CD.
- $\bar{T_{migCI}}$ - Znormalizowany do $[0, 1]$ czas migracji w środowisku CI/CD.
- $\bar{T_{rbkCI}}$ - Znormalizowany do $[0, 1]$ czas rollbacku w środowisku CI/CD.
- (Mianownik w E_{CI}) - Suma wag $w_{T_{mig}} + w_{T_{rbk}} + w_{CPLX} + w_{SUBJ} + w_R + w_{SR}$, pozwalająca na normalizację E_{CI} do skali $[0, 1]$.
- E - Globalny wskaźnik efektywności narzędzia, uwzględniający oba środowiska (lokalne i CI/CD).
- α - Współczynnik wagowy ($0 \leqslant \alpha \leqslant 1$) określający względne znaczenie środowiska lokalnego w obliczeniu E, waga środowiska CI/CD wynosi $(1 - \alpha)$.

5.3 Stopień efektywności wybranych narzędzi

Metryki		
metryka	Flyway	Liquibase
$\bar{T_{migL}}$	0,50	0,49
$\bar{T_{rbkL}}$	0,14	0,13
$\bar{CPLX_L}$	0,89	0,93
$\bar{CPU}$	0,77	0,27
$\bar{RAM}$	0,52	0,48
$\bar{SUBJ_L}$	0,00	1,00
$\bar{R}$	0,00	1,00
E_{LOCAL}	0,44	0,50
$\bar{T_{migCI}}$	0,53	0,21
$\bar{T_{rbkCI}}$	0,92	0,49
$\bar{CPLX_{CI}}$	0,89	0,93
$\bar{SUBJ_{CI}}$	0,00	1,00
$\bar{R_{CI}}$	0,00	1,00
E_{CI}	0,58	0,56
α	0,4	
E	0,52	0,54

Tabela 5.1 Obliczone znormalizowane wartości wraz ze stopniami efektywności.

Z tabeli 5.1 można odczytać, że $\bar{T_{migL}}$ dla Flyway wynosił 0,50, a dla Liquibase 0,49. Obie wartości praktycznie się pokrywają, oznacza to, że w testach lokalnych Flyway i Liquibase radziły sobie bardzo podobnie pod względem czasu, jaki zajmuje zastosowanie migracji. Bardzo niewielka przewaga Flyway wskazuje, że był minimalnie szybszy.

Czas cofnięcia zmian w środowisku lokalnym $\bar{T_{rbkL}}$ wynosiły dla Flyway 0,14, a dla Liquibase 0,13. Tak niskie wartości wskazują, że czasy cofnięcia zmian w testach lokalnych były stosunkowo długie. Flyway uzyskał nieznacznie lepszy wynik niż Liquibase.

Dla złożoności skryptów $\bar{CPLX}$ (w tabeli 5.1 $\bar{CPLX_{CI}} = \bar{CPLX_L}$) Liquibase osiągnął 0,93, a Flyway 0,89, co oznacza, że skrypty w Liquibase były odrobinę prostsze. W każdym razie oba narzędzia znajdują się w górnej części skali, co znaczy, że złożoność skryptów migracji nie stanowi dużego problemu dla żadnego z nich.

Zużycie CPU w testach lokalnych $\bar{CPU}$ wynosiło dla Flyway 0,77, a dla Liquibase 0,27. Różnica jest tu wyraźna. W praktyce znaczy to, że Liquibase zużywało w testach lokalnych więcej zasobów procesora niż Flyway, który w tej samej operacji wykonywał zmiany z mniejszym apetytem na CPU. Wysoka wartość 0,77 dla Flyway oznacza, że w skali porównawczej jego zużycie CPU było bliskie CPU_{min}, natomiast 0,27 dla Liquibase pokazuje, że tutaj był bliżej CPU_{max}.

Zużycie pamięci ram $\bar{RAM}$ w środowisku lokalnym dla Flyway wynosiło 0,52, a dla Liquibase 0,48. Obie wartości oscylują w środku skali, co oznacza umiarkowane różnice. Flyway zużywał nieco mniej pamięci niż Liquibase, ale różnica jest niewielka. Oba narzędzia w ramach lokalnych testów nie obciążały znacząco pamięci, raczej plasowały się w okolicach mediany.

Subiektywna ocena narzędzi $\bar{SUBJ}$ ($\bar{SUBJ_L} = \bar{SUBJ_{CI}}$) jest miarą absolutną. Metryka dla Flyway wynosiła 0, ponieważ narzędzie zostało ocenione na 3,5, a Liquibase dostał ocenę 4,5, co przełożyło się na maksymalną wartość metryki, tj. 1. Liquibase uzy-

skał nieznacznie wyższą ocenę ze względu na obsługę większej liczby formatów plików oraz możliwość precyzyjnego i intuicyjnego wykonywania poszczególnych etapów migracji poprzez definiowanie i wywoływanie odpowiednich kontekstów. Choć Flyway również umożliwia uruchamianie konkretnych skryptów na podstawie nazewnictwa plików, mechanizm ten jest mniej bezpośredni i mniej elastyczny w porównaniu z podejściem stosowanym przez Liquibase.

Metryka $\bar{R}$ jest dokładnym odzwierciedleniem faktu, że w darmowej wersji Flyway nie posiada wbudowanego, deklaratywnego rollbacku, konieczne było uruchamianie ręcznych skryptów SQL, co z punktu widzenia modelu jest traktowane jako „brak automatyzacji" (0). Natomiast Liquibase ma mechanizm rollback wbudowany od razu w silnik migracji, więc uzyskuje maksymalną notę (1).

Metryka E_{LOCAL} to zsumowany (wagami) wskaźnik opisujący ogólną efektywność każdego narzędzia w testach lokalnych. Wartość 0,50 dla Liquibase oznacza, że przy uwzględnieniu wszystkich powyższych metryk (czasy, złożoności, zużycia zasobów i subiektywnych ocen) uzyskało ono nieco lepszy wynik niż Flyway (0,44), mimo że Flyway był lepszy w niektórych metrykach, to przewaga Liquibase w subiektywnej ocenie i wbudowanej automatyzacji rollbacku ostatecznie dała Liquibase lepszy rating w środowisku lokalnym.

Metryka $\bar{T}_{migCI}$ pokazuje wyraźną przewagę Flyway. Jego migracja w środowisku CI/CD była na tyle szybka, że jego wynik znormalizowany wyniósł około 0,53, co plasuje go w górnej części skali. Dla porównania, Liquibase osiągnął zaledwie 0,21, co oznacza, że jego migracje w tym samym środowisku trwały znacznie dłużej. Taka różnica mocno faworyzuje Flyway w kontekście szybkości wdrożeń w CI.

Dla metryki $\bar{T}_{rbkCI}$ Flyway osiągnął wartość 0,92, co świadczy o istotnie krótszym czasie wykonania operacji rollback w porównaniu do Liquibase, który zanotował wynik 0,49. Ponieważ wartość 0,49 znajduje się nieznacznie poniżej progu 0,50, wskazuje to, że mechanizm rollback w CI/CD Liquibase wymagał zauważalnie więcej czasu. Różnicę tę tłumaczy fakt, że Flyway wykonuje skrypty rollback bezpośrednio w SQL, podczas gdy Liquibase uruchamia je za pośrednictwem własnego narzędzia.

Wskaźnik E_{CI} to zsumowana efektywność w CI. Flyway osiągając 0,58 wygrywa nieznacznie nad Liquibase z wynikiem 0,56. Różnica jest niewielka, ale jednak Flyway uzyskał trochę wyższą ocenę, głównie dzięki świetnym wynikom w $\bar{T}_{migCI}$ oraz $\bar{T}_{rbkCI}$. Natomiast słabsze oceny w subiektywnym rankingu oraz całkowitym braku autorskiej automatyzacji rollbacku nie zaważyły tak mocno, bo w CI/CD czasy migracji/rollbacku miały relatywnie większy udział wag.

Wskaźnik α mówi na ile odniesienie do wyników lokalnych jest ważne w porównaniu z pipeline CI/CD. Jeśli $\alpha = 0,4$, to 40 % wagi bierze pod uwagę oceny lokalne, a 60 % przydziela się wynikom z CI/CD.

Porównanie ostatecznego stopnia efektywności E, gdzie Flyway uzyskał $0,52$, a Liquibase $0,54$, ukazuje, że po wzięciu pod uwagę obu środowisk Liquibase osiąga niewielką przewagę 0,54 nad Flyway 0,52. Chociaż Flyway był widocznie lepszy w CI, to jednak globalnie Liquibase wygrywa, bo w ważonej mieszance, przy założonych wagach, przewaga Liquibase w środowisku lokalnym i przewaga Flyway w CI/CD niemal się równoważą, ale jednak Liquibase ma minimalnie lepszy bilans (wynika to m.in. z wyższej oceny w lokalnym środowisku). Jednakże jeśli wskaźnik $\alpha \leqslant 0,2$, czyli przyłożono by większą wagę do środowiska CI/CD, Flyway uzyskał by lepszy wynik.

Analiza wskaźnika efektywności E wskazuje, że Flyway osiągnął wartość $0,52$, a Liquibase $0,54$. Po uwzględnieniu obu środowisk Liquibase uzyskuje niewielką przewagę nad Flyway. Mimo że Flyway wykazał lepsze wyniki w środowisku CI/CD, to przy założonych wagach efektywności uzyskane przez Liquibase korzyści w środowisku lokalnym

oraz relatywnie wyrównana pozycja Flyway w CI/CD skutkują ostatecznym, nieznacznym zwycięstwem Liquibase. Należy jednak zaznaczyć, iż dla wartości parametru $\alpha \leqslant 0,2$ (tj. w sytuacji, gdy środowisko CI/CD ma znaczenie większe znaczenie w porównaniu z lokalnym), Flyway uzyskałby wyższą wartość wskaźnika E niż Liquibase.

Rozdział 6

Wnioski

Zarządzanie zmianami w bazach danych jest niezwykle ważnym zagadnieniem, zwłaszcza w obliczu rosnącej dynamiki rozwoju aplikacji i konieczności ciągłego dostarczania nowych funkcjonalności. Dzięki zastosowaniu odpowiednich narzędzi możliwe jest systematyczne i bezpieczne modyfikowanie schematu bazy danych, minimalizacja ryzyka błędów oraz zachowanie spójności danych w całym cyklu życia oprogramowania. Aby efektywnie zarządzać zmianami, należy najpierw jasno sprecyzować potrzeby i oczekiwania stawiane narzędziom oraz zespołom deweloperskim. Kluczowe jest zrozumienie, które aspekty mają największe znaczenie z perspektywy użytkowników biznesowych oraz zespołów IT.

W kontekście CI/CD rola narzędzi do migracji i wersjonowania schematów baz danych staje się jeszcze bardziej istotna. Środowisko CI/CD wymaga, aby zmiany w bazie danych były nie tylko poprawne technicznie, ale również możliwe do zautomatyzowania, przetestowania i wdrożenia w dowolnym momencie bez ingerencji manualnej. Właśnie dlatego metryki wydajnościowe oraz niezawodnościowe stanowią fundament oceny przydatności konkretnego narzędzia. Bez tych danych trudno jest podejmować świadome decyzje o wyborze technologii w projekcie, zwłaszcza gdy zespół dąży do skrócenia czasu wprowadzania zmian i jednoczesnego utrzymania wysokiej jakości.

Równocześnie nie można pominąć kwestii skalowalności i przepustowości zmian. W miarę jak aplikacja zyskuje nowych użytkowników i rozrasta się jej baza danych, rośnie również liczba migracji, które trzeba wykonać. W takich warunkach kluczowe staje się, aby narzędzie radziło sobie ze zwiększonym obciążeniem zarówno w zakresie liczby jednorazowo aplikowanych zmian, jak i ich skomplikowania (np. rozbudowane zmiany w dużych tabelach). Przykładowo, scenariusz wielokrotnego uruchamiania serii migracji w pętli (od 1 do 100 razy) pozwala ocenić, czy narzędzie zachowuje stałą wydajność, czy też wraz ze wzrostem liczby operacji wzrasta ryzyko błędów lub gwałtownego wzrostu czasu wykonywania.

W niniejszej pracy starano się odpowiedzieć na poniższe pytania badawcze.

- RQ1: Jakie są specyficzne potrzeby i wymagania dotyczące zarządzania zmianami w bazach danych w kontekście aplikacji opartych na CI/CD?
- RQ2: Jakie parametry są kluczowe dla oceny efektywności narzędzi do zarządzania zmianami w bazach danych w środowiskach CI/CD?
- RQ3: Jak zdefiniować wieloaspektowy model efektywności narzędzi CI/CD do zarządzania zmianami w bazach danych, uwzględniający wydajność, niezawodność oraz skuteczność?

Praca odpowiada na pytanie RQ1 w rozdziale 2.2 i wskazuje, że w kontekście aplikacji funkcjonujących w modelu CI/CD zarządzanie zmianami w bazach danych wymaga spełnienia zbioru specyficznych kryteriów wynikających z potrzeby pełnej automatyzacji, kontroli oraz zapewnienia niezawodności w środowiskach o stałym cyklu rozwojowo-wdrożeniowym. Przede wszystkim, narzędzie do migracji musi umożliwiać pełną automatyzację procesu wdrożeń od momentu przygotowania skryptów po ich zastosowanie w docelowych środowiskach. Oznacza to, że wszelkie etapy, od synchronizacji schematów, przez wykrywanie różnic między wersjami, aż po wykonanie właściwych skryptów migracyjnych, muszą działać bez interwencji manualnej, co eliminuje ryzyko błędów ludzkich oraz przyspiesza cykl wydawniczy. Konieczne jest również wsparcie pełnej kontroli wersji zmian zarówno w strukturze, jak i w danych bazy. Powinna też być możliwość migracji danych pomiędzy różnymi wersjami schematów, uwzględniając konwersję danych oraz tłumaczenie schematów między różnymi systemami bazodanowymi. W trakcie procesu migracji należy zagwarantować bezpieczeństwo danych na każdym etapie. Narzędzie powinno także umożliwiać synchronizację danych i schematów pomiędzy środowiskami deweloperskim, testowym oraz produkcyjnym, jednocześnie wykrywając i rozwiązując konflikty, które mogą pojawić się w wyniku równoległych zmian w różnych gałęziach kodu. Biorąc pod uwagę fakt, iż współczesne środowiska IT coraz częściej operują w modelu chmurowym i konteneryzowanym, narzędzie migracyjne musi być zdolne do obsługi baz danych w takich ekosystemach, zapewniając elastyczność przy skalowaniu instancji oraz zgodność z dynamicznie przydzielanymi zasobami. Ostatecznie w procesie zarządzania zmianami nie można zapomnieć o pełnej widoczności zmian w bazach danych, umożliwiając śledzenie wprowadzonych modyfikacji, czas ich realizacji oraz identyfikację ewentualnych problemów.

Na pytanie RQ2 starano się odpowiedzieć w rozdziale 3.3.1. Stwierdzono, że kluczowe elementy dla oceny efektywności narzędzi do zarządzania zmianami w bazach danych w środowiskach CI/CD to czas wykonywania migracji oraz cofania wprowadzonych zmian, wskaźnik sukcesu migracji, złożoność skryptów migracyjnych, zużycie zasobów oraz łatwość integracji z CI/CD. Można też wziąć pod uwagę subiektywną ocenę narzędzi.

Aby odpowiedzieć na pytanie RQ3 zbadano narzędzia Flyway i Liquibase za pomocą metryk wspomnianych w rozdziale 3.3.1, przeanalizowano wyniki w rozdziale 4 oraz opracowano model porównania efektywności w rozdziale 5. W celu stworzenia wieloaspektowego modelu efektywności narzędzi CI/CD do zarządzania zmianami w bazach danych, uwzględniającego kryteria wydajności, niezawodności oraz skuteczności, niezbędne jest najpierw zebranie metryk odpowiadających wymienionym aspektom. Kolejnym krokiem jest normalizacja metryk, co umożliwi porównanie wskaźników o różnych skalach i jednostkach miary. Normalizację należałoby przeprowadzić według wzorów 5.1 dla metryk, w przypadku których wartość niższa oznacza lepszy rezultat, oraz 5.2 dla metryk, w których większa wartość przekłada się na wyższą ocenę. Po normalizacji, na podstawie wzorów 5.3 i 5.4 wylicza się stopnie efektywności narzędzi w środowisku lokalnym oraz w kontekście CI/CD. Wreszcie, oba stopnie efektywności łączone są z wykorzystaniem wzoru 5.5, w którym uwzględniany jest współczynnik wagowy α, określający względne znaczenie środowiska lokalnego w końcowym obliczeniu wskaźnika efektywności globalnej. Dzięki tak skonstruowanemu modelowi uzyskuje się możliwość bezpośredniego porównania narzędzi Flyway i Liquibase. Najważniejszą zaletą tego podejścia jest jego elastyczność wynikająca z zastosowania wag istotności poszczególnych metryk. Użytkownik może w dowolnym momencie zmodyfikować wartości wag, tak aby odzwierciedlić własne priorytety. W przeprowadzonym badaniu, po zastosowaniu wszystkich etapów normalizacji i obliczeń, Flyway osiągnął ostateczny współczynnik efektywności równy $0,52$, podczas

gdy Liquibase uzyskał wartość 0,54. Pomimo że Flyway generalnie charakteryzował się lepszymi czasami wykonania migracji, łatwość implementacji Liquibase oraz wbudowane wsparcie dla automatycznego cofania zmian już w wydaniu otwartoźródłowym przeważyły na korzyść Liquibase w finalnym porównaniu. Opracowany model może być stosowany zarówno do porównywania wydań darmowych, jak i płatnych, gdyż obydwa narzędzia w wersjach komercyjnych oferują mechanizmy automatycznego wycofywania zmian. Ponadto model pozwala na analizę w środowisku testowym, gdzie zakłada się, że wszystkie migracje zakończą się pomyślnie, jak i w środowisku produkcyjnym, w którym mogą wystąpić różnorodne nieprzewidziane problemy. Elastyczność w doborze wag istotności oraz możliwość uwzględniania specyfiki środowiska wdrożeniowego sprawiają, że model ten jest wysoce konfigurowalny i może zostać zaadaptowany do szerokiego spektrum scenariuszy użycia.

Poza odpowiedzią na powyższe pytania w ramach pracy opracowano oraz wdrożono potoki CI/CD, a także przygotowano aplikację demonstracyjną, umożliwiającą ręczne uruchamianie poszczególnych etapów procesu migracji oraz podgląd bieżącego stanu schematu bazy danych. Zarówno aplikacja, jak i potoki wdrożeniowe stanowiły fundament do przeprowadzenia zaplanowanych badań.

W ramach dalszego rozwoju projektu uwzględnia się możliwość rozszerzenia zestawu scenariuszy testowych migracji, co pozwoli na jeszcze bardziej reprezentatywną ocenę porównawczą obu narzędzi. Ponadto można uwzględnić dodatkowe trzecie narzędzie, aby ocenić, jak Flyway i Liquibase wypadają na tle innego rozwiązania. W kolejnych etapach badania warto również przetestować płatne edycje wszystkich rozważanych narzędzi (np. Flyway Teams/Enterprise, Liquibase Pro/Enterprise), aby zweryfikować dodatkowe funkcjonalności, takie jak wbudowane mechanizmy cofania zmian, zaawansowane raportowanie czy dedykowane wsparcie techniczne. Dodatkowo rekomenduje się zwiększenie liczby powtórzeń testów migracyjnych, zamiast dotychczasowych 100 iteracji, przeprowadzenie badań przy 1 000 lub nawet 10 000 cyklach, o ile zasoby czasowe i infrastrukturalne na to pozwolą, co przełoży się na uzyskanie bardziej statystycznie miarodajnych wyników.

Podsumowując, badania przeprowadzone w ramach pracy magisterskiej wykazały, że wskazanie jednego uniwersalnego narzędzia spełniającego wszystkie wymagania każdej organizacji jest zadaniem trudnym. Flyway i Liquibase z jednej strony oferują porównywalny zestaw funkcji, z drugiej zaś różnią się podejściem do zarządzania zmianami oraz wbudowanymi mechanizmami zapewniającymi automatyzację. Przedstawiony model efektywności pozwala na świadome dobranie narzędzia w oparciu o indywidualne priorytety, a realizacja potoków CI/CD i aplikacji demonstracyjnej dowodzi, że praktyczne wdrożenie migracji w architekturze ciągłego dostarczania jest możliwe i przynosi wymierne korzyści. Jednocześnie warto prowadzić badania w rzeczywistym środowisku produkcyjnym, gdzie istnienie ryzyka nieudanej migracji umożliwia weryfikację odporności narzędzi oraz procedur cofania zmian w warunkach zbliżonych do produkcyjnych. Ostatecznie, należy podkreślić, że w obszarze zarządzania zmianami bazodanowymi w środowiskach CI/CD znaczenie ma nie tylko dobór konkretnego narzędzia, ale również dbałość o spójność procesów, regularne testowanie oraz stały nadzór operacyjny, co umożliwia szybkie reagowanie na nieprzewidziane sytuacje i zapewnia ciągłość działania systemów.

Rozdział 7

Załączniki do pracy

```
|---.github
|   \---workflows
|---backend
|   |---.mvn
|   |   \---wrapper
|   |
|   \---src
|       |---main
|       |   |---java
|       |   |   \---org
|       |   |       \---mastersthesis
|       |   |           \---backend
|       |   |               |---controller
|       |   |               \---service
|       |   |
|       |   \---resources
|       |       \---db
|       |           |---changelog
|       |           \---migration
|       |
|       \---test
|           \---java
|               \---org
|                   \---mastersthesis
|                       \---backend
|---frontend
|   |---.vscode
|   |---public
|   \---src
|       \---app
|           |---components
|           |   |---main
|           |   \---table-preview
|           \---services
|
\---sql
```

Listing 7.1 Struktura projektu

Literatura

[1] a. Adam Stypczyc. mastersthesis. https://github.com/astypczy/mastersThesis, 2025. [Dostęp: 1 czerwca 2025].

[2] Angular. Angular Overview. https://angular.dev/overview. [Dostęp: 6 czerwca 2025].

[3] Angular Team. Build environments angular. https://angular.dev/tools/cli/environments, 2025. [Dostęp: 20 maja 2025].

[4] Apache Maven Project. The maven wrapper. https://maven.apache.org/wrapper/, 2024. [Dostęp: 20 maja 2025].

[5] S. Boot. Spring Boot Documentation. https://docs.spring.io/spring-boot/index.html. [Dostęp: 6 czerwca 2025].

[6] Bytebase. What is bytebase? https://www.bytebase.com/docs/introduction/what-is-bytebase/, 2025. [Dostęp: 1 czerwca 2025].

[7] A. Capizzi, S. Distefano, M. Mazzara. From devops to devdataops: data management in devops processes. *Software Engineering Aspects of Continuous Development and New Paradigms of Software Production and Deployment: Second International Workshop, DEVOPS 2019, Château de Villebrumier, France, May 6–8, 2019, Revised Selected Papers 2*, strony 52–62. Springer, 2020.

[8] L. Community. Liquibase forum. https://forum.liquibase.org, 2025. [Dostęp: 24 maja 2025].

[9] K. Damirchi. Database change management. https://medium.com/@KeivanDamirchi/database-change-management-946af9c02169 [Dostęp: 29.11.24], 2023.

[10] S. Developers. scipy.stats.shapiro. https://docs.scipy.org/doc/scipy/reference/generated/scipy.stats.shapiro.html, 2025. [Dostęp: 25 maja 2025].

[11] S. Developers. scipy.stats.ttest_rel. https://docs.scipy.org/doc/scipy/reference/generated/scipy.stats.ttest_rel.html, 2025. [Dostęp: 25 maja 2025].

[12] S. Developers. scipy.stats.wilcoxon. https://docs.scipy.org/doc/scipy/reference/generated/scipy.stats.wilcoxon.html, 2025. [Dostęp: 25 maja 2025].

[13] R. Fadatare. Spring boot architecture: Controller, service, repository, database & architecture flow. *Medium*, 2025. [Dostęp: 20 maja 2025].

[14] N. E. Fenton, S. L. Pfleeger. *Software Metrics: A Rigorous and Practical Approach*. PWS Publishing, Boston, MA, 1997.

[15] J. Fluri, F. Fornari, E. Pustulka. Measuring the benefits of ci/cd practices for database application development. *2023 IEEE/ACM International Conference on Software and System Processes (ICSSP)*, strony 46–57, 2023.

[16] J. Fluri, F. Fornari, E. Pustulka. On the importance of ci/cd practices for database applications. *Journal of Software: Evolution and Process*, strona e2720, 2024.

[17] L. A. for the GitHub Actions Marketplace. Liquibase github action. `https://github.com/liquibase-github-actions/`. [Dostęp: 24 maja 2025].

[18] M. Fowler. Zero-downtime database migrations. *martinfowler.com*, 2020. [Dostęp 21 maja 2025].

[19] G2. Flyway pricing overview. `https://www.g2.com/products/flyway/pricing`, 2024. Dostęp 24 maja 2025.

[20] GeeksforGeeks. Spring boot – externalized configuration. *GeeksforGeeks*, 2024. [Dostęp: 20 maja 2025].

[21] GitHub. Actions: new images and ubuntu-latest changes. `https://github.blog/changelog/2024-09-25-actions-new-images-and-ubuntu-latest-changes/`, September 2024. Dostęp: 2025-05-20.

[22] GitHub. actions/checkout: Github action for checking out a repository. `https://github.com/marketplace/actions/checkout`, 2025. Dostęp: 20 maja 2025.

[23] GitHub. actions/upload-artifact@v4. `https://github.com/actions/upload-artifact`, 2025. [Dostęp: 20 maja 2025].

[24] GitHub. Creating postgresql service containers. `https://docs.github.com/en/actions/use-cases-and-examples/using-containerized-services/creating-postgresql-service-containers`, 2025. [Dostęp: 20 maja 2025].

[25] GitHub. Setup java jdk. `https://github.com/marketplace/actions/setup-java-jdk`, 2025. [Dostęp: 20 maja 2025].

[26] GitHub. Setup node.js environment. `https://github.com/marketplace/actions/setup-node-js-environment`, 2025. [Dostęp: 20 maja 2025].

[27] GitHub. Setup python. `https://github.com/marketplace/actions/setup-python`, 2025. [Dostęp: 20 maja 2025].

[28] F. D. Gregorio, D. Varrazzo. psycopg2: Postgresql database adapter for python. `https://pypi.org/project/psycopg2/`, 2024. [Dostęp: 20 maja 2025].

[29] S. History. Star history: Liquibase vs. flyway. `https://www.star-history.com/#liquibase/liquibase&flyway/flyway&Date`, 2025. [Dostęp: 24 maja 2025].

[30] Jenkov. Maven commands. `https://jenkov.com/tutorials/maven/maven-commands.html`, 2025. [Dostęp: 20 maja 2025].

[31] joke2k. Faker: Python package for generating fake data. `https://pypi.org/project/Faker/`, 2025. [Dostęp: 20 maja 2025].

[32] C. Jones. *Applied Software Measurement: Global Analysis of Productivity and Quality*. McGraw-Hill, New York, NY, 2008.

[33] Z. Kaluba, M. Nyirenda. Database migration service with a microservice architecture. *ResearchGate*, 2024.

[34] G. Kim, J. Humble, P. Debois, J. Willis, N. Forsgren. *The DevOps handbook: How to create world-class agility, reliability, & security in technology organizations*. It Revolution, 2021.

[35] Liquibase. How liquibase works. `https://www.liquibase.org/get-started/how-liquibase-works`. [Dostęp: 24 maja 2025].

[36] Liquibase. Changelog. `https://docs.liquibase.com/concepts/changelogs/home.html`, 2025. [Dostęp: 22 maja 2025].

[37] Liquibase. Databasechangelog table. `https://docs.liquibase.com/concepts/tracking-tables/databasechangelog-table.html`, 2025. [Dostęp: 24 maja 2025].

[38] Liquibase. Databasechangeloglock table. `https://docs.liquibase.com/concepts/tracking-tables/databasechangeloglock-table.html`, 2025. [Dostęp: 24 maja 2025].

[39] Liquibase. Liquibase. `https://github.com/liquibase/liquibase`, 2025. [Dostęp: 24 maja 2025].

[40] Liquibase. Liquibase commands. `https://docs.liquibase.com/commands/home.html`, 2025. [Dostęp: 20 maja 2025].

[41] Liquibase. Liquibase community. `https://www.liquibase.com/community`, 2025. [Dostęp: 24 maja 2025].

[42] Liquibase. Liquibase editions & pricing. `https://www.liquibase.com/pricing`, 2025. [Dostęp: 24 maja 2025].

[43] Liquibase. Liquibase enterprise product summary. `https://www.liquibase.com/resources/whitepapers/liquibase-enterprise-product-summary`, 2025. [Dostęp: 24 maja 2025].

[44] Liquibase. Liquibase university. `https://learn.liquibase.com`, 2025. [Dostęp: 24 maja 2025].

[45] Liquibase. Supported databases. `https://www.liquibase.com/supported-databases`, 2025. [Dostęp: 22 maja 2025].

[46] A. Maatuk, A. Ali, N. Rossiter. Relational database migration: A perspective. *Database and Expert Systems Applications: 19th International Conference, DEXA 2008, Turin, Italy, September 1-5, 2008. Proceedings 19*, strony 676–683. Springer, 2008.

[47] T. J. McCabe. A complexity measure. *IEEE Transactions on Software Engineering*, SE-2(4):308–320, 1976.

[48] P. Netinant, N. Saengsuwan, M. Rukhiran, S. Pukdesree. Enhancing data management strategies with a hybrid layering framework in assessing data validation and high availability sustainability. *Sustainability*, 15(20):15034, 2023.

[49] S. Overflow. Tag: flyway. https://stackoverflow.com/tags/flyway/, 2025. [Dostęp: 24 maja 2025].

[50] S. Overflow. Tag: liquibase. https://stackoverflow.com/tags/liquibase/, 2025. [Dostęp: 24 maja 2025].

[51] T. pandas development team. pandas: powerful python data analysis toolkit. https://pandas.pydata.org/, 2025. [Dostęp: 24 maja 2025].

[52] O. Polska. Czym jest etl? https://www.oracle.com/pl/integration/what-is-etl/, 2021. [Dostęp: 22 maja 2025].

[53] O. Polska. Czym jest system oltp? https://www.oracle.com/pl/database/what-is-oltp/, 2023. [Dostęp: 22 maja 2025].

[54] red gate. Example ci/cd pipelines - redgate flyway. https://documentation.red-gate.com/fd/example-ci-cd-pipelines-138347137.html, 2025. [Dostęp: 24 maja 2025].

[55] Redgate. Flyway github action. https://github.com/red-gate/FlywayGitHubAction. [Dostęp: 24 maja 2025].

[56] Redgate. Using flyway on your ci/cd agents. https://documentation.red-gate.com/fd/using-flyway-on-your-ci-cd-agents-153027261.html. [Dostęp: 24 maja 2025].

[57] Redgate. Flyway by redgate. https://github.com/flyway/flyway, 2025. [Dostęp: 24 maja 2025].

[58] Redgate. Redgate university. https://www.red-gate.com/hub/university/, 2025. [Dostęp: 24 maja 2025].

[59] Redgate Software. Repeatable migrations - redgate flyway - product documentation. https://documentation.red-gate.com/fd/repeatable-migrations-273973335.html, 2024. [Dostęp: 22 maja 2025].

[60] Redgate Software. Undo migrations - redgate flyway - product documentation. https://documentation.red-gate.com/fd/undo-migrations-273973334.html, 2024. [Dostęp: 22 maja 2025].

[61] Redgate Software. Versioned migrations - redgate flyway - product documentation. https://documentation.red-gate.com/fd/versioned-migrations-273973333.html, 2024. [Dostęp: 22 maja 2025].

[62] Redgate Software. Flyway schema history table - redgate flyway - product documentation. https://documentation.red-gate.com/fd/flyway-schema-history-table-273973417.html, 2025. [Dostęp: 22 maja 2025].

[63] Redgate Software. Supported databases and versions. `https://documentation.red-gate.com/fd/supported-databases-and-versions-143754067.html`, 2025. [Dostęp: 22 maja 2025].

[64] R. Software. Exploring the flyway schema history table. `https://www.red-gate.com/hub/product-learning/flyway/exploring-the-flyway-schema-history-table`, 2021. [Dostęp 24 maja 2025].

[65] R. Software. Commercial licensing faq. `https://documentation.red-gate.com/fd/commercial-licensing-faq-181633028.html`, 2024. [Dostęp: 24 maja 2025].

[66] R. Software. *Flyway Command-line Tool*, 2025. [Dostęp: 20 maja 2025].

[67] R. Software. Flyway community – database migrations made easy. `https://www.red-gate.com/products/flyway/community`, 2025. [Dostęp: 24 maja 2025].

[68] R. Software. Flyway enterprise – automate database deployments across teams and technologies. `https://www.red-gate.com/products/flyway/enterprise/`, 2025. [Dostęp: 24 maja 2025].

[69] M. S. S. Team. Backup and restore of sql server databases. *Microsoft Docs*, 2024. [Dostęp: 1 czerwca 2025].

[70] Tianzhou. Database change management. `https://www.bytebase.com/blog/what-is-database-change-management/` [Dostęp: 05.01.25], 2023.

[71] UCSB CS48. Spring boot: Database migrations. `https://ucsb-cs48.github.io/javatopics/spring_boot_database_migrations/`, 2025. [Dostęp: 22 maja 2025].

[72] P. Virtanen, R. Gommers, T. E. Oliphant, M. Haberland, T. Reddy, D. Cournapeau, E. Burovski, P. Peterson, W. Weckesser, J. Bright, S. J. van der Walt, M. Brett, J. Wilson, K. J. Millman, N. Mayorov, A. R. J. Nelson, E. Jones, R. Kern, E. Larson, C. J. Carey, İlhan Polat, Y. Feng, E. W. Moore, J. VanderPlas, D. Laxalde, J. Perktold, R. Cimrman, I. Henriksen, E. A. Quintero, C. R. Harris, A. M. Archibald, A. H. Ribeiro, F. Pedregosa, P. van Mulbregt, S. . Contributors. Scipy 1.0: Fundamental algorithms for scientific computing in python. *Nature Methods*, 17:261–272, 2020.

Spis rysunków

2.1 Potok przepływu danych DevOps [7] 7

3.1 Strona główna aplikacji demonstracyjnej 20
3.2 Strona podglądu aktualnego schematu bazy danych 21
3.3 Diagram - scenariusz 1. 24
3.4 Diagram - scenariusz 2. 25
3.5 Uproszczony schemat dziennika zmian (*changelog*) [36] 29
3.6 Wykres popularności Flyway i Liquibase wg ilości gwiazdek na platformie Github [29]. 34

4.1 Zestawienie średnich czasów migracji i cofania zmian dla każdego z przebiegów. 38
4.2 Histogram różnic [ns] scenariusza 1. w środowisku lokalnym. 39
4.3 Zestawienie średnich czasów migracji i cofania zmian dla każdego z przebiegów w środowisku CI/CD. 42
4.4 Histogram różnic [ns] scenariusza 1. w środowisku CI/CD. 43
4.5 Zestawienie średnich czasów migracji i cofania zmian dla każdego z przebiegów. 46
4.6 Histogram różnic [ns] scenariusza 2. w środowisku lokalnym. 47
4.7 Zestawienie średnich czasów migracji i cofania zmian dla każdego z przebiegów w środowisku CI/CD. 50
4.8 Histogram różnic [ns] scenariusza 2. w środowisku CI/CD. 51

Spis tabel

3.1 Zawartość tabeli `flyway_schema_history` [64]. 30
3.2 Zestawienie obsługiwanych baz danych i platform [45] [63]. 31
3.3 Zawartość tabeli `databasechangelog` [37]. 32
3.4 Zawartośc tabeli `databasechangeloglock` [38]. 32

4.1 Przykładowe wyniki scenariusza 1. na maszynie lokalnej 40
4.2 Wyniki Scenariusza 1. 41
4.3 Przykładowe wyniki scenariusza 1. na Github Actions 44
4.4 Wyniki Scenariusza 1. w środowisku Github Actions. 45
4.5 Przykładowe wyniki scenariusza 2. na maszynie lokalnej 48
4.6 Wyniki Scenariusza 2. w środowisku lokalnym. 49
4.7 Przykładowe wyniki scenariusza 2. na Github Actions 52
4.8 Wyniki Scenariusza 2. w środowisku Github Actions. 53
4.9 Metryki dla poszczególnych changeSet w narzędziu Liquibase. 54
4.10 Metryki skryptów migracyjnych i cofających zmiany dla narzędzia Flyway. 55
4.11 Wyniki wskaźnika skomplikowania skryptów. 57

5.1 Obliczone znormalizowane wartości wraz ze stopniami efektywności. 62

Listings

3.1 Potok wdrożeniowy `ci-frontend.yml` 16
3.2 Przykładowe uruchomienie migracji w potoku wdrożeniowym `ci-backend.yml` 17
3.3 Kod pliku konfiguracyjnego `application.yml` 19
3.4 Schemat bazy danych wykorzystywanej w scenariuszu 1. 21
3.5 Zastosowana konstrukcja dodająca dane 24
3.6 Konwencja nazwy plików Flyway 27
3.7 Przykładowy zestaw zmian (*changeset*) w fomacie XML. 30
7.1 Struktura projektu 69